

# 我是人工智能

GYF

2026 年春季学期

## Contents

|                             |          |
|-----------------------------|----------|
| <b>我是人工智能</b>               | <b>4</b> |
| 前言                          | 4        |
| 第一章 绪论                      | 4        |
| 本章概要                        | 4        |
| 1.1 人工智能究竟研究什么              | 4        |
| 1.2 从思想史到形式化：人工智能为什么离不开逻辑   | 5        |
| 1.3 符号主义：规则、推理与专家系统         | 6        |
| 1.4 联结主义与进化主义：另外两条重要路线      | 7        |
| 1.5 人工智能的典型应用领域与整门课的主线      | 8        |
| 本章小练习                       | 8        |
| 参考解答                        | 8        |
| 第二章 联结主义与人工神经网络             | 9        |
| 本章概要                        | 9        |
| 2.1 联结主义的基本想法与深度学习的兴起       | 9        |
| 2.2 人工神经元：从加权求和到激活输出        | 9        |
| 2.3 从线性模型到感知机：最早的可学习分类器     | 10       |
| 2.4 感知机的局限与多层感知机的出现         | 12       |
| 2.5 Sigmoid、均方误差、梯度下降与反向传播  | 13       |
| 2.6 竞争学习、赢家通吃与 Kohonen 网络   | 15       |
| 2.7 从模式识别到完整分类系统            | 16       |
| 本章小练习                       | 16       |
| 参考解答                        | 17       |
| 第三章 深度学习、表示学习与序列建模          | 17       |
| 本章概要                        | 17       |
| 3.1 为什么需要深度学习               | 17       |
| 3.2 分类、损失函数与表示空间            | 18       |
| 3.3 自编码器、深层表示与卷积网络          | 20       |
| 3.4 序列数据、中文分词与 HMM          | 22       |
| 3.5 从 HMM 到 CRF：条件建模与标注偏置   | 24       |
| 3.6 循环网络、上下文表示与 Transformer | 24       |
| 3.7 PCA、自编码器与生成模型           | 25       |
| 3.8 深度学习在更大图景中的位置           | 26       |
| 本章小练习                       | 27       |
| 参考解答                        | 27       |

|                               |    |
|-------------------------------|----|
| 第四章 知识表示                      | 27 |
| 本章概要                          | 27 |
| 4.1 为什么人工智能离不开知识表示            | 28 |
| 4.2 语义网络：把概念放进关系图中            | 28 |
| 4.3 框架系统：把对象写成带槽位的结构          | 29 |
| 4.4 概念图：把语义关系画成更精细的结构         | 29 |
| 4.5 知识图谱与三元组                  | 30 |
| 4.6 从句法分析到语义解释                | 31 |
| 4.7 不同知识表示方式的比较               | 31 |
| 本章小练习                         | 31 |
| 参考解答                          | 32 |
| 第五章 不确定性推理                    | 32 |
| 本章概要                          | 32 |
| 5.1 不确定性从哪里来                  | 32 |
| 5.2 演绎、归纳与非单调推理               | 33 |
| 5.3 经典概率、显著性检验与它们的局限          | 34 |
| 5.4 贝叶斯定理：用证据更新信念             | 35 |
| 5.5 贝叶斯决策：不仅问“信什么”，还问“做什么”    | 36 |
| 5.6 贝叶斯方法的困难与贝叶斯网络            | 37 |
| 5.7 MYCIN 与确定性因子              | 38 |
| 5.8 证据组合与 CF 的局限              | 39 |
| 5.9 时间推理、随机过程与马尔可夫链           | 41 |
| 5.10 模糊集合：处理“部分为真”的概念         | 42 |
| 5.11 模糊规则、语言修饰词与模糊合成          | 43 |
| 本章小练习                         | 46 |
| 参考解答                          | 46 |
| 第六章 遗传算法                      | 47 |
| 本章概要                          | 47 |
| 6.1 从生物进化到搜索算法                | 47 |
| 6.2 一个最基本的例子：最大化 $f(x) = x^2$ | 47 |
| 6.3 种群、适应度与轮盘赌选择              | 48 |
| 6.4 交叉与变异：两种不同的搜索力量           | 49 |
| 6.5 遗传算法的标准流程                 | 49 |
| 6.6 遗传算法适合解决什么问题              | 50 |
| 6.7 旅行商问题中的编码、交叉与变异           | 51 |
| 6.8 Gray 编码与编码方式的重要性          | 51 |
| 6.9 进化硬件、随机搜索与遗传算法的优势         | 52 |
| 6.10 模式、阶与定义长度                | 52 |
| 6.11 模式定理                     | 53 |
| 本章小练习                         | 54 |
| 参考解答                          | 55 |
| 第七章 逻辑与 Prolog                | 55 |
| 本章概要                          | 55 |
| 7.1 逻辑在人工智能中的位置               | 55 |
| 7.2 命题逻辑：从真假命题到复合公式           | 56 |
| 7.3 从命题逻辑到一阶逻辑                | 57 |
| 7.4 一阶逻辑的语义：解释、满足、模型、有效       | 58 |
| 7.5 自然语言到逻辑表达                 | 59 |

|                                            |    |
|--------------------------------------------|----|
| 7.6 推理、逻辑后继与证明过程                           | 60 |
| 7.7 合一：让两个表达式“对上”                          | 61 |
| 7.8 归结反驳法的基本思想                             | 62 |
| 7.9 从自然语言到子句形式                             | 64 |
| 7.10 两个经典归结例子                              | 65 |
| 7.11 归结策略与 Herbrand 观点                     | 67 |
| 7.12 逻辑系统的层次与扩展                            | 67 |
| 7.13 Horn 子句：让逻辑变成程序                       | 68 |
| 7.14 Prolog 的执行机制                          | 69 |
| 7.15 Prolog 的基本语法与简单例子                     | 69 |
| 7.16 列表匹配与递归程序                             | 70 |
| 7.17 子句顺序、目标顺序与回溯                          | 71 |
| 7.18 九宫格马步搜索与路径问题                          | 71 |
| 7.19 cut：控制搜索的剪枝                           | 72 |
| 7.20 农夫、狼、羊、菜与状态空间搜索                       | 72 |
| 本章小练习                                      | 73 |
| 参考解答                                       | 74 |
| 第八章 CLIPS 与基于规则的专家系统                       | 74 |
| 本章概要                                       | 74 |
| 8.1 规则系统的基本结构                              | 74 |
| 8.2 CLIPS 是什么：从语法外形看规则系统                   | 75 |
| 8.3 事实与模板：让知识有结构地存起来                       | 75 |
| 8.4 模板属性：类型、取值范围、基数与默认值                    | 76 |
| 8.5 有序事实与显式模板                              | 77 |
| 8.6 事实操作：添加、显示、删除与修改                       | 77 |
| 8.7 规则：左部匹配，右部动作                           | 78 |
| 8.8 变量、事实地址与规则中的绑定                         | 79 |
| 8.9 单字段通配、多字段通配与未指定槽位                      | 80 |
| 8.10 约束：否定、选择、合取与测试                        | 80 |
| 8.11 条件元素：or、and、not、exists、forall、logical | 81 |
| 8.12 规则系统中的决策树程序                           | 82 |
| 8.13 推理引擎、agenda 与冲突消解                     | 83 |
| 8.14 salience、控制模式与模块化                     | 83 |
| 8.15 监控问题：一个完整规则系统的拼装方式                    | 85 |
| 8.16 模式匹配网络                                | 85 |
| 8.17 模式顺序                                  | 85 |
| 8.18 多字段通配与匹配爆炸                            | 86 |
| 8.19 test 条件与内建约束的效率                       | 86 |
| 8.20 initial-fact、过程函数与交互控制                | 87 |
| 本章小练习                                      | 88 |
| 参考解答                                       | 88 |
| 第九章 强化学习                                   | 88 |
| 本章概要                                       | 88 |
| 9.1 从机器学习的整体图景看强化学习                        | 89 |
| 9.2 强化学习的基本框架                              | 89 |
| 9.3 奖赏、回报与折扣因子                             | 90 |
| 9.4 多臂老虎机问题与探索利用矛盾                         | 90 |
| 9.5 贝尔曼方程：把当前与未来联系起来                       | 91 |

9.6 广义策略迭代与策略迭代 . . . . .

9.7 一个直观例子：为什么局部动作要服从长期价值 . . . . .

9.8 蒙特卡洛方法：从完整经历中学习 . . . . .

9.9 时序差分学习：边走边学 . . . . .

9.10 Sarsa 与 Q-learning：控制问题的两条典型路线 . . . . .

9.11 基于模型的方法与现实挑战 . . . . .

9.12 策略梯度：直接优化策略 . . . . .

9.13 n 步 TD、 $\lambda$ -回报与资格迹 . . . . .

9.14 深度强化学习与多智能体方向 . . . . .

本章小练习 . . . . .

参考解答 . . . . .

92

93

93

93

94

94

95

96

96

97

97

我是人工智能

前言

人工智能是一门内容跨度较大、方法类型丰富、概念层次分明的课程。初学时，读者往往会同时接触搜索、知识表示、不确定性推理、机器学习、神经网络与深度学习等多个方向，容易在名词、方法和应用之间感到割裂。因此，学习这门课时，既需要理解每一种方法各自在解决什么问题，也需要逐步建立起不同知识点之间的联系。

本资料面向第一次接触人工智能的初学者，旨在帮助读者根据人工智能学科的主要内容建立较为清晰的知识框架，理解基本概念、主要方法与常见模型，并在后续学习、整理与查阅时能够较为顺畅地使用相关内容。资料既可作为入门材料，也可作为系统梳理知识时的参考。

第一章 绪论

本章概要

人工智能并不是一门只讨论“如何写几个新模型”的课程，也不是若干零散算法的堆积。更准确地说，它研究的是：怎样让一个人工系统能够表现出某种意义上的**智能行为**，例如表示知识、进行推理、从经验中学习、在复杂环境中搜索与决策，以及和外部世界进行交互。要真正理解这门课，第一步不是急着记算法名称，而是先看清人工智能的发展主线：它为什么会出现，最早在解决什么问题，后来又为什么逐渐分化出符号主义、联结主义和进化主义等不同路线。

这一章的任务，就是为整门课建立一个总框架。读完后，读者应当能够回答这些基础问题：人工智能究竟在研究什么；为什么逻辑、数学、认知科学和计算理论都与人工智能密切相关；规则、神经网络、进化算法分别代表怎样的思想；以及人工智能常见的应用领域有哪些。只要这一章把握住了，后面学习知识表示、推理、机器学习、神经网络和深度学习时，就不会总觉得知识点是分散的。

1.1 人工智能究竟研究什么

**人工智能** (Artificial Intelligence, AI) 通常可以理解为：研究如何构造能够表现出某种智能行为的人工系统。这里的“智能行为”并不神秘，它往往体现在几个可分析、可实现的方面，例如理解问题、表示信息、进行推理、从数据中学习、

在大量可能性中搜索可行方案、根据目标做决策，以及在环境中执行行动。换句话说，人工智能关心的不是“机器有没有灵魂”这种哲学问题，而是：一个系统究竟应当具备哪些机制，才能在复杂问题上表现得像一个有能力的行动者。

初学人工智能时，最容易产生的误解之一，是把它直接等同于“机器学习”或者“深度学习”。这种理解并不完整。机器学习当然是人工智能的重要组成部分，但人工智能的范围更大。早在大规模数据驱动方法流行之前，人们已经在研究定理证明、规则推理、博弈、规划、专家系统、自然语言理解和机器人控制等问题。因此，人工智能更像一个总学科，而机器学习、知识表示、逻辑推理、神经网络等，则是这个总学科中的不同分支。

如果再往深一层看，人工智能实际上始终在围绕三个最根本的问题展开。第一，**什么叫知识**，以及知识怎样被表示。第二，**什么叫推理**，也就是系统如何从已知事实推出未知结论。第三，**什么叫学习**，也就是系统怎样利用数据或经验不断调整自己的行为。整门课后续的许多章节，本质上都只是对这三件事做更细致的展开。

这一点也解释了为什么人工智能常常会提出一些看似很大的问题，例如：什么是思维，什么是机器，什么是智能。表面上看，这些问题很哲学；一旦进入具体实现，它们就会立刻落到十分务实的层面，例如一个系统内部有没有可操作的表示形式，有没有可执行的规则，有没有能够优化目标函数的机制，有没有处理不确定性的办法。也正因为如此，人工智能这门学科从一开始就带有强烈的跨学科色彩，它既离不开数学，也离不开逻辑、心理学、计算机科学和工程实现。

通常人们会把 1956 年达特茅斯会议（Dartmouth Workshop）视为人工智能作为独立学科的标志性起点。这个时间点之所以重要，不只是因为“Artificial Intelligence”这一名称被明确提出，更因为从那时起，人们开始系统地把一个问题摆在台面上：若人的某些智能活动本身可以被分解、被描述、被形式化，那么它们是否也就有机会被机器部分实现。后面几十年的发展，不同路线虽然彼此竞争，但都可以看作对这个总问题的不同回答。

## 1.2 从思想史到形式化：人工智能为什么离不开逻辑

人工智能并不是从计算机出现那一天才突然诞生的。它真正的思想根源，要追溯到人类很早就开始思考的一件事：**思维是否可以被分析、被刻画、被规则化**。如果答案是否定的，那么人工智能几乎无从谈起；如果答案在某种程度上是肯定的，那么人工智能就至少有了理论出发点。

在这条思想脉络中，亚里士多德的重要性尤其突出。因为他最早系统地研究了**三段论**，也就是从一般命题和个别事实中推出结论的规则。例如，“所有人都会死；苏格拉底是人；因此苏格拉底会死”。这个例子之所以在人工智能史上反复出现，不是因为它本身多么复杂，而是因为它清楚地说明了一件事：有些推理不是凭直觉跳出来的，而是遵循某种相对稳定的形式结构。只要这种结构能够被抽象出来，人类的部分思维活动就可能被形式化处理。

从现代符号写法看，上述推理可以被压缩成一种非常经典的结构：若已知  $A$  与  $A \rightarrow B$  为真，则可以推出  $B$  也为真。这种推理方式通常被称为**肯定前件**，也是最基本的推理模式之一。它启发了后来的逻辑系统、规则系统和自动推理系统。读者需要特别理解这里的核心思想：人工智能早期并不是先去模仿大脑，而是先去追问“正确推理有没有清晰的规则”。

不过，单有逻辑还不够。近代科学革命又补上了另一条重要线索。哥白尼、伽利略、笛卡尔等人的工作，使人们逐渐意识到：世界的真实结构未必和人的直观感受完全一致，要真正理解世界，必须依靠抽象的理论和数学化描述。这个转变对于人工智能非常关键，因为它暗示了这样一种可能性：人类对世界的理解，并不等于直接复制世界本身，而是通过某种内部表示与推理机制来完成。换句话说，智能活动也许可以被看作一种信息处理过程。

到了霍布斯、莱布尼茨、布尔、弗雷格、罗素、怀特海等人的时代，逻辑与数学的形式化程度进一步提高。尤其是布尔代数的建立，使逻辑不再只是哲学论辩的工具，而成为可以精确运算的形式系统；弗雷格和后继者则推动了谓词逻辑的发展，使“对象”“性质”“关系”能够被更精细地表达出来。人工智能后来的知识表示、命题逻辑、一阶谓词逻辑、规则推理，正是建立在这些形式化成果之上的。

在这里，读者需要特别分清**语法**和**语义**。语法关心的是“式子写得是否合规、推理步骤是否符合规则”；语义关心的是“这些式子到底在说什么，它们在某个解释下是不是真的”。这也是塔斯基语义理论的重要贡献所在。对于人工智能而言，一个系统不能只会机械地搬动符号，它还必须有办法把符号和对象、事实、世界状态联系起来。所谓**模型** (model)，可以简单理解为一种解释方式：在这种解释下，某些命题是真的，某些命题是假的。于是，推理就不再只是字符串变换，而是和“真值”“满足”“解释”这些概念联系起来了。

从这个角度看，人工智能早期一项极其重要的工作，就是把“思维”从模糊的心理印象转化成“可被规则操纵的符号结构”。这也是为什么一阶谓词逻辑、形式语言、证明、模型、可满足性等概念在人工智能中始终占有基础地位。哪怕后来的机器学习方法已经极为强大，逻辑路线所提供的思想框架仍然没有失效，因为任何真正复杂的智能系统，几乎都绕不开表示、约束、解释与推理这些问题。

最后还必须提到图灵。图灵的重要性不只在于他提出了“机器能否思考”的问题，更在于他把“可计算”这件事本身变成了严格的数学对象。人工智能之所以是一门计算学科，而不是泛泛讨论智能的哲学随笔，关键就在于它默认这样一个立场：至少有一部分智能活动，是能够落入计算过程之内的。这个立场并没有回答全部问题，但它给了整门学科最重要的可操作起点。

### 1.3 符号主义：规则、推理与专家系统

如果说上一节解决的是“为什么思维有可能被形式化”，那么这一节要回答的就是：当人们真的尝试去实现一个会推理的系统时，最自然的路线是什么？历史上最早成型、也最符合直觉的一条路线，就是**符号主义** (symbolicism)。它的基本信念是：智能活动可以通过明确的符号表示和规则操作来实现。系统内部先用符号表示事实、概念和关系，再依据推理规则不断推出新结论。

这种思想在认知科学中也有很强的影响力。纽厄尔和西蒙等人认为，人类的许多问题求解活动可以表述成某种 **IF-THEN** 型规则。比如说，若已知“汽车无法启动”并且“油表显示为空”，那么就可以推出“应先加油”。这种表示方式之所以重要，是因为它把复杂思考拆成了相对清楚的局部单元：前件给出条件，后件给出结论或动作。只要规则足够多，系统就可以沿着规则网络不断前进。

一个典型的规则系统，通常至少包括几个核心部分。首先是**知识库** (knowledge base)，里面存放规则；其次是**工作记忆** (working memory)，里面存放当前已知事实；再次是**推理机** (inference engine)，它负责检查哪些规则当前可以触发，并把新结论加入系统；此外往往还会配有解释模块与知识获取模块，用来说明“系统为什么这样判断”，以及让领域专家把知识逐步录入系统。对于初学者来说，这里最重要的不是死记模块名称，而是理解它们之间的角色分工：规则负责“会什么”，事实负责“当前知道什么”，推理机负责“接下来该做什么”。

这种系统后来发展成了经典的**专家系统** (expert system)。所谓专家系统，并不是说它真的等同于某位人类专家，而是说它试图在一个范围较窄、知识较明确的领域内，利用大量规则模拟专家的判断过程。医学诊断系统 MYCIN、矿产分析系统 PROSPECTOR、计算机配置系统 XCON 都是历史上很著名的例子。它们的成功说明：当领域知识足够清晰、规则可以被专家较稳定地总结出来时，基于规则的系统确实能解决实际问题。

符号主义的优点很明显。第一，它的知识表示往往是**可解释的**，系统为什么得到某个结论，通常可以追溯到具体规则链条。第二，它非常适合处理那些结构明确、约束清晰、逻辑关系强的问题。第三，它和人们日常理解“推理”的方式比较接近，因此在知识表示、自动证明、规划、法律规则、医学规则等场景中很自然。

但它也有明显局限。最突出的问题是：现实世界并不总能被写成干净整齐的规则。很多知识是模糊的、不完整的、带噪声的、依赖大量经验的。若一个领域变化很快，或者知识本身难以精确表述，那么靠人工不断写规则就会变得异常困难。这

也就是所谓的“知识获取瓶颈”。因此，符号主义并不是人工智能的终点，但它为人工智能建立了极其重要的第一代框架：**知识可以被表示，推理可以被规则化，系统可以在符号层面操作知识**。后面的不确定性推理、逻辑程序设计和专家系统章节，本质上都会继续展开这条路线。

#### 1.4 联结主义与进化主义：另外两条重要路线

如果符号主义把智能理解成“符号和规则的操作”，那么后来的研究者很快就会追问：人类真的只是按一条条显式规则来思考吗？许多能力，例如识别图像、听懂语言、从经验中逐渐改进判断，似乎并不完全像在手工调用规则。这种怀疑推动了另一条重要路线的发展，即**联结主义** (connectionism)。

联结主义的基本想法是：与其先把知识写成清晰规则，不如构造一个由大量简单单元相互连接而成的网络，让系统通过调整连接权重来表现复杂行为。这里的灵感很大程度上来自神经系统。单个神经元并不神奇，但大量神经元的连接与激活可以形成丰富的功能。于是，在人工智能中，联结主义更强调**分布式表示、数值计算和从数据中学习参数**。后面要学习的感知机、神经网络、反向传播和深度学习，正是这一路线逐步成熟后的结果。

对初学者来说，最需要抓住的一点是：联结主义并不是完全抛弃“知识”，而是改变了知识的存在方式。在符号主义中，知识往往以明确规则或逻辑公式的形式存在；而在联结主义中，知识往往分散地“藏在”参数里，体现在网络结构和权重之中。因此，这一类方法通常更擅长从样本中提取模式，却也更容易出现“结果不错但解释较难”的问题。

除了联结主义之外，人工智能中还有第三条非常重要的路线，即**进化主义** (evolutionism)。它的思想来源于生物进化：如果一个问题很难直接写出解析解，那么是否可以像自然选择那样，通过“生成候选解、评估适应度、保留更优个体、反复变异和重组”的方式逐步逼近更好的结果？这就是遗传算法等方法的基本思路。

遗传算法 (genetic algorithm) 的核心对象不是单个公式，而是一群候选解，也就是一个**种群** (population)。每个候选解都对应一个**适应度** (fitness)，用于衡量它当前有多好。算法通常先随机生成一批个体，然后根据适应度进行选择，让更优个体更有机会被保留下来；接着通过**交叉** (crossover) 把两个个体的部分结构重新组合，再通过**变异** (mutation) 引入小幅随机改变。经过多代迭代后，种群整体质量往往会提高。

若把第  $i$  个个体的适应度记作  $f_i$ ，总共有  $n$  个个体，那么最常见的一种选择概率可以写为

$$p_i = \frac{f_i}{\sum_{j=1}^n f_j}$$

这里， $p_i$  表示第  $i$  个个体被选中的概率，分子是它自己的适应度，分母是全体适应度之和。这个公式表达的思想非常朴素：个体越优，越应该更可能参与繁殖。但也要注意，这并不意味着最差个体一定毫无机会。保留少量随机性，恰恰有助于系统跳出局部最优。

联结主义与进化主义都在提醒我们：智能未必只能通过显式逻辑规则体现。前者强调通过大量连接和参数学习去形成能力，后者强调通过群体搜索和选择机制去逐步优化。它们与符号主义并不是简单的替代关系，而更像三种观察智能的不同视角。后面的课程中，你会看到它们在具体方法上不断交织：有时靠逻辑和知识表示，有时靠参数学习，有时靠搜索和优化。

顺便说一句，现代人工智能中还经常讨论**多智能体系统** (multi-agent systems)。它关注的不是单个系统独自完成任务，而是多个具有一定自主性的智能体如何在同一环境中协作、竞争、通信和形成整体行为。这个方向在绪论里只需建立印象即可：当系统规模扩大后，智能不一定只体现在单个体内部，也可能体现在群体互动所产生的整体结构之中。

## 1.5 人工智能的典型应用领域与整门课的主线

理解一门学科，除了看它的理论基础，还要看它到底在解决哪些问题。人工智能的典型应用领域非常广，但作为初学者，最值得先建立印象的是几类经典方向。第一类是**博弈与搜索**，例如棋类对弈、路径规划、状态空间搜索等。第二类是**自动推理**，也就是依据逻辑规则和知识库推导结论。第三类是**专家系统**，强调在特定领域中模仿专家判断。第四类是**自然语言理解**，研究系统怎样处理文本和语言。第五类是**规划与机器人**，关注如何在环境中制定行动方案并执行。第六类则是**机器学习**，研究系统怎样从数据中改进自身。

把这些方向放在一起看，整门人工智能课程的真实主线就会变得比较清楚。它并不是让读者背下一串互不相干的方法名，而是引导读者逐步回答几个层层递进的问题：当问题是离散而结构化的，如何用搜索和逻辑来解决；当知识需要长期保存和推理时，怎样表示它；当环境带有不确定性时，又怎样做判断；当规则难以写清时，如何让系统从数据中学习；当单个模型不够时，又怎样借助进化、群体或多层网络去增强能力。

因此，学习这一章时，最重要的不是把历史人物和系统名称孤立地记住，而是要把它们放回人工智能的总体图景中。亚里士多德、布尔、弗雷格、图灵等人，帮助我们理解“思维能否被形式化”；纽厄尔、西蒙以及专家系统的发展，帮助我们理解“规则能否承载知识与推理”；神经系统与联结主义的发展，帮助我们理解“学习能否由参数和网络结构体现”；进化计算则提醒我们，“优化与搜索”本身也可以构成智能的一部分。只有把这些线索合到一起，我们才会真正明白：人工智能不是单一路线的胜利，而是一门不断吸收不同思想、不断扩展方法边界的综合学科。

### 本章小练习

#### 1. 判断题

人工智能与机器学习是完全等价的概念，因此学习人工智能时只要掌握数据驱动模型即可。(\_\_\_\_)

#### 2. 填空题

在规则系统中，存放规则的部分通常称为 \_\_\_\_\_，存放当前事实的部分通常称为 \_\_\_\_\_。

#### 3. 简答题

请说明符号主义、联结主义和进化主义三条路线分别是如何理解“智能”的。

### 参考解答

1. 错。机器学习只是人工智能的重要组成部分之一。人工智能还包含搜索、知识表示、逻辑推理、规划、专家系统、机器人等内容。若把人工智能简单等同于机器学习，就会忽略这门学科中大量与表示、推理和搜索有关的经典问题。
2. 存放规则的部分通常称为**知识库**，存放当前事实的部分通常称为**工作记忆**。前者回答系统“知道哪些规则”，后者回答系统“当前掌握哪些事实”，而推理机则在二者之间不断匹配并产生新结论。
3. 符号主义认为智能主要体现为对符号的表示和规则化操作，因此强调逻辑、知识表示、规则推理和解释能力。联结主义认为智能可以由大量简单单元之间的连接结构和参数学习产生，因此强调神经网络、分布式表示和从数据中学习。进化主义则认为不必先精确写出求解步骤，可以通过候选解群体的选择、交叉和变异逐步逼近更优结果，因此更强调搜索、优化和适应过程。三条路线关注的侧面不同，但都在尝试解释和实现智能行为。



## 第二章 联结主义与人工神经网络

### 本章概要

如果说第一章主要建立了人工智能的总体地图，那么这一章要真正进入联结主义路线的内部。联结主义的核心观点是：智能不一定非要写成一条条显式规则，也可以通过大量简单计算单元之间的连接、加权、激活和学习来逐步形成。沿着这条思路，人工神经元、感知机、多层感知机、梯度下降、反向传播以及竞争学习等方法逐步发展起来，并构成了后来深度学习的重要基础。

这一章要解决的问题主要有四类。第一，为什么神经网络会被看成一种与符号主义不同的智能实现方式。第二，人工神经元、McCulloch-Pitts 神经元与感知机是什么，它们怎样完成分类。第三，单层感知机为什么有能力也有局限，多层网络又为什么能够突破这些局限。第四，梯度下降、均方误差和反向传播究竟在做什么，以及竞争学习和 Kohonen 网络又代表了怎样的无监督学习思想。读完后，读者应当能够从结构、公式和直观图像三个层面，对联结主义方法形成比较完整的初步认识。

### 2.1 联结主义的基本想法与深度学习的兴起

联结主义之所以重要，首先在于它对“智能”的理解方式与符号主义明显不同。符号主义更强调用明确的规则、命题和逻辑结构来表达知识；联结主义则认为，一个系统的能力未必非要先写成清清楚楚的规则，它也可能来自大量简单单元之间的连接方式，以及连接权重在学习过程中的逐步调整。简单地说，联结主义更关心“能力是怎样被学出来的”，而不仅仅是“规则是怎样被写出来的”。

这种想法与神经系统的启发密切相关。生物神经元本身并不是一个会推理的小机器，但大量神经元通过连接形成网络之后，却能支持感知、记忆、学习和决策等复杂功能。于是，人工智能研究者便尝试构造一种人工版本的神经网络：每个节点只完成简单的加权求和与激活操作，而复杂行为则由整体网络涌现出来。正是在这一意义上，联结主义强调的是**分布式表示与参数化学习**。

后来所谓的**深度学习** (deep learning)，本质上可以看作联结主义在更大规模数据、更深网络结构和更强算力条件下的进一步发展。它之所以在近十余年中重新崛起，并不是因为“神经网络”这个概念突然被发明出来，而是因为 GPU、数据规模、训练技巧和工程能力共同推动，使深层网络真正能够稳定训练并在视觉、语音、自然语言处理等任务上达到极高水平。ImageNet 竞赛、语音识别、深度强化学习和围棋系统的成功，都说明联结主义方法已经不再只是理论上的设想，而是能够在复杂任务中表现出强大能力的现实工具。

不过，在真正理解深度学习之前，必须先回到它最简单、也最基础的起点：**人工神经元**。如果最基本的单元都没有看清，那么后面的多层网络、误差反传、表示学习就会显得像一堆来路不明的公式。因此，这一章接下来的内容，实际上就是沿着“单个神经元 -> 单层网络 -> 多层网络 -> 学习算法 -> 更广义的联结主义模型”这条路线逐步展开。

### 2.2 人工神经元：从加权求和到激活输出

最基本的人工神经元模型可以概括成一个非常简单的流程：输入信号进入系统后，先分别乘上各自的权重，再把所有结果加总起来，形成一个总激活量，最后再经过某种激活函数输出结果。若输入向量记为  $x = (x_1, x_2, \dots, x_n)$ ，权重向量记为  $w = (w_1, w_2, \dots, w_n)$ ，偏置记为  $b$ ，那么神经元的净输入通常写为

$$\sigma = \sum_{i=1}^n w_i x_i + b$$

这里， $x_i$  表示第  $i$  个输入分量， $w_i$  表示该输入对应的重要程度， $b$  表示偏置项。偏置的作用可以理解为“平移阈值”，它决定了神经元在没有足够输入时是否容易被激活。若把这些量写成向量形式，则上式也可以记作

$$\sigma = w^T x + b$$

随后，神经元输出并不一定直接等于  $\sigma$ ，而通常写成

$$y = f(\sigma)$$

其中， $f$  称为**激活函数** (activation function)。激活函数的意义在于，它决定了神经元对输入总和如何作出反应。若没有这一层非线性变换，多个线性层叠加后本质上仍然只是线性映射，模型表达能力就会非常有限。这一点在后面讨论多层感知机时会再次成为关键。

从直觉上看，一个神经元实际上很像一个带阈值的判别器。它先看所有输入的加权和是否足够大，再决定是否输出某种激活结果。因此，权重控制“每个输入有多重要”，偏置控制“门槛大致在哪里”，激活函数控制“通过门槛之后怎样输出”。这三者共同决定了神经元的行为。

为了更好理解这一点，可以先看最早期、最经典的 McCulloch-Pitts 神经元模型。它把输入通常取成 0 或 1，把输出也看成离散状态，并通过阈值函数来判断是否激活。这个模型虽然非常简单，却已经能够实现某些基本逻辑功能。例如，若令两个输入分别为  $x$  和  $y$ ，并考虑净输入

$$\sigma = x + y - 2$$

若规定只有当  $\sigma \geq 0$  时输出 1，否则输出 0，那么只有在  $x = 1$  且  $y = 1$  时，输出才为 1，这就对应了逻辑与运算。类似地，若把偏置改成较小的值，例如考虑

$$\sigma = x + y - 1$$

则只要两个输入中至少有一个为 1，输出就会变成 1，这对应了逻辑或运算。由此可以看出，神经元并不只是“一个数学表达式”，它其实已经可以作为逻辑判别单元使用。也正因为如此，神经网络从一开始就与逻辑、决策和分类问题紧密相关。

### 2.3 从线性模型到感知机：最早的可学习分类器

理解感知机之前，可以先把它放在线性模型的框架中来看。在线性代数中，我们已经知道一个线性表达式  $w^T x + b$  可以表示一个超平面。所谓超平面，就是一个维度可能超过人类可视化范围（3 维）的空间，它可以将空间分割为两部分。

例如，若输入是二维向量  $x = (x_1, x_2)$ ，那么方程

$$w_1x_1 + w_2x_2 + b = 0$$

在平面中表示一条直线；若输入维度更高，它就表示更高维空间中的超平面。这个超平面会把空间切成两侧，因此天然适合作为分类边界。这就是线性模型最根本的几何意义。

日常生活中，我们经常要做的工作是回归 (Regression)。回归的目标是预测一个连续的数值（例如房价、温度、股票价格）。它通过寻找输入变量（特征）与输出变量（目标值）之间的数学关系来进行预测。

如果我们只是做回归，那么常见写法是令模型输出

$$f(x, w) = w^T x + b$$

并用训练数据去拟合参数  $w$  和  $b$ 。这时一个常见的目标函数是**残差平方和** (residual sum of squares, RSS):

$$\text{RSS}(w) = \frac{1}{2} \sum_{n=1}^N (y_n - f(x_n, w))^2$$

这里， $N$  表示样本数， $x_n$  表示第  $n$  个输入样本， $y_n$  表示对应的目标值。这个目标函数的含义非常直接：希望模型输出与真实值之间的平方误差尽量小。后面无论是梯度下降还是反向传播，本质上都离不开“先定义误差，再沿着误差减小的方向更新参数”这一思路。也就是说，Linear-to-neural 中出现的线性模型并不是额外的插曲，而是理解神经网络学习机制的自然前置。

在分类问题中，最经典的单层联结主义模型是**感知机** (perceptron)。它由 Rosenblatt 在 1958 年提出，可以看作在人工神经元基础上加入了明确学习规则的线性分类器。若采用二分类设定，并把类别输出记为  $+1$  或  $-1$ ，则感知机的输出可写为

$$y = \begin{cases} 1, & \sum_{i=1}^n w_i x_i \geq t \\ -1, & \sum_{i=1}^n w_i x_i < t \end{cases}$$

其中， $t$  是阈值。若把偏置写成  $b = -t$ ，则也可写成更紧凑的形式

$$y = \text{sign}(w^T x + b)$$

这里的  $\text{sign}$  函数表示看符号取值：正数输出  $+1$ ，负数输出  $-1$ 。从几何角度看，这意味着感知机的任务就是寻找一个超平面，把两类样本尽量分开。于是，感知机的本质就非常清楚了：它是一个**线性分类器**。也就是说，我们要时刻牢记：感知机并不是在记忆单个样本，而是在**学习一个整体判别边界**。

感知机真正关键的地方不在于输出公式，而在于它有一条非常简单的学习规则。若真实标签记为  $d$ ，模型当前输出记为  $y$ ，学习率记为  $r$ ，则权重更新可以写为

$$\Delta w_i = r(d - y)x_i$$

这个式子表达的思想非常朴素。若当前输出和真实标签相同，即  $d = y$ ，则不需要更新；若输出偏小或偏大，就沿着输入方向去修正对应权重。由于这里的输出只取  $+1$  和  $-1$ ，所以错误分类时， $d - y$  的取值只会是  $2$  或  $-2$ 。因此可以进一步得到两个很直观的结论：若模型实际输出为  $-1$ ，但正确答案应为  $+1$ ，那么权重应沿输入方向增加  $2rx_i$ ；若模型实际输出为  $+1$ ，但正确答案应为  $-1$ ，那么权重应沿输入方向减少  $2rx_i$ 。

这非常容易直观理解：假设我当前“预估小了”，那么肯定就需要把权重增大。但并不是所有  $w_i$  都一样程度地加，而是要结合各个“输入方向”  $x_i$ ，如果  $x_i$  大，说明这里贡献得多，这个方向的  $w_i$  就要加得更多。

这一规则之所以重要，是因为它把“模型做错了该怎么办”用一个极其清晰的方式表达了出来。假设当前参数给出的分类边界不对，那么每遇到一个被误分的样本，就把边界稍微朝正确方向推一点。经过很多轮迭代后，只要数据本身是线性可分的，感知机往往能够收敛到某个可行解。

## 2.4 感知机的局限与多层感知机的出现

感知机虽然重要，但它有一个根本局限：它只能表示线性可分的分类边界。所谓**线性可分**，是指存在某个超平面，可以把不同类别样本完全分开。如果数据的分布结构本身不是线性的，那么单层感知机无论怎样训练，也无法得到正确结果。

最经典的例子就是异或 (exclusive-or, XOR) 问题。异或的真值表是：当两个输入不同的时候输出  $1$ ，相同的时候输出  $0$ 。若把四个输入点  $(0, 0)$ 、 $(0, 1)$ 、 $(1, 0)$ 、 $(1, 1)$  画到平面中，就会发现需要被归为同一类的两个点恰好落在对角线上，而另一类则落在另一条对角线上。这样的分布不可能用一条直线切开。换句话说，异或问题是**非线性可分**的。

如果坚持用单层感知机来处理异或，就会得到一组彼此矛盾的不等式条件。证明思路很清晰：根据四个点应落在分类边界两侧的要求，可以推出对同一组参数  $w_1$ 、 $w_2$  和阈值  $t$  的几条不等式，而这些不等式不可能同时成立。因此，问题不在于“训练得不够久”，而在于“模型能力本身不够”。这也是联结主义发展史上极其著名的转折点：单层感知机不能解决所有分类问题。

要突破这个限制，就必须引入**非线性和多层结构**。最自然的办法，是在输入层和输出层之间加入一个或多个隐藏层，让网络不再只做一次线性判别，而是先把输入映射到新的表示空间，再在新的表示上继续做计算。这样得到的模型就是**多层感知机** (multilayer perceptron, MLP)。

从结构上看，多层感知机一般包含输入层、一个或多个隐藏层以及输出层。每一层中的节点都接收前一层的输出，进行加权求和并通过激活函数后，把结果传给下一层。若以一个隐藏层为例，隐藏层第  $j$  个节点的输出可以写成

$$h_j = f \left( \sum_{i=1}^n w_{ji} x_i + b_j \right)$$

其中， $x_i$  是输入层第  $i$  个分量， $w_{ji}$  是从输入层第  $i$  个节点连到隐藏层第  $j$  个节点的权重， $b_j$  是隐藏层该节点的偏置。而这里的  $f$  就是一个非线性函数（如 Sigmoid 函数）。

若输出层第  $k$  个节点记为  $o_k$ ，则有

$$o_k = f \left( \sum_{j=1}^t v_{kj} h_j + c_k \right)$$

这里， $v_{kj}$  表示从隐藏层第  $j$  个节点到输出层第  $k$  个节点的权重， $c_k$  是输出层偏置。这个公式形式看起来和单个神经元很像，但意义已经完全不同：网络不再直接对原始输入做一次切分，而是先在隐藏层中构造新的中间表示，再在这些表示上完成最后任务。

于是，之前的异或问题就有了新的解法。单层感知机只能给出一条直线，而带隐藏层的网络可以通过多个隐藏节点先构造若干中间判别区域，再把这些区域组合起来，从而形成非线性边界。因此我们就能理解其中的核心含义：**多层结构 + 非线性激活** 使网络具备了表达非线性分类边界的能力。

## 2.5 Sigmoid、均方误差、梯度下降与反向传播

单层感知机之所以不适合继续向深层推广，一个重要原因在于它使用的是硬阈值函数。硬阈值函数虽然直观，但在阈值点附近不可导，因此不便于使用基于导数的优化方法。

如何理解“硬阈值”会让深层网络彻底瘫痪呢？假设你手头有一个拥有 10 层、包含几千个权重的深层网络。你给它输入了一张猫的照片，模型经过复杂的层层计算，最后输出说：“这是狗”。你现在想要调整网络里的权重，让它下次聪明点。这时候你就会遇到硬阈值函数带来的致命绝望：它无法提供方向和试错反馈：假设某个神经元当前的输入得分是 -10，因为小于 0，硬阈值函数输出 0。你把权重稍微调整了一点点，让得分变成了 -9.9。因为还是小于 0，硬阈值函数的输出依然是 0。你根本不知道这次调整是让结果变好了还是变坏了，因为输出没有任何变化！在数学上，它的导数（斜率）是 0：除了中间那个断崖点，硬阈值函数的左右两边都是绝对水平的直线。水平线的斜率就是 0。

因此，在现代 AI 中，我们用导数来当做“指南针”。导数会告诉我们：“权重应该增加还是减少，才能让误差变小”。为了解决这一问题，联结主义中常引入更平滑的激活函数，其中最经典的一种就是 **Sigmoid 函数**。其形式为

$$f(\sigma) = \frac{1}{1 + e^{-\sigma}}$$

这个函数的输出范围在 0 到 1 之间，能够把任意实数压缩到一个连续区间内。更重要的是，它处处可导，且导数形式十分简洁：

$$f'(\sigma) = f(\sigma)(1 - f(\sigma))$$

这个导数公式在后面的误差反传中非常重要，因为它使链式法则展开后仍能保持相对简单的形式。

有了可导激活函数之后，下一步就要回答：网络究竟怎样知道自己做得好不好。对此，最常见的做法是先定义一个**损失函数**或**误差函数**。对于输出层，若第  $i$  个输出节点的真实值记为  $d_i$ ，网络实际输出记为  $O_i$ ，则均方误差可以写为

$$\text{Error} = \frac{1}{2} \sum_i (d_i - O_i)^2$$

这个公式表示，把每个输出节点的误差平方之后求和，再乘上  $1/2$ 。前面的  $1/2$  主要是为了后续求导更方便，并不改变

最优点的位置。这个误差函数的意义很清楚：希望网络输出尽量接近期望输出。

一旦误差函数确定下来，训练的目标就转化为一个优化问题：调整所有权重，使误差尽量小。此时就要用到**梯度** (gradient) 和**梯度下降** (gradient descent)。简单地说，梯度给出了函数在当前位置增长最快的方向，因此若我们想让误差减小，就应当沿着负梯度方向移动参数。若某个权重记为  $w$ ，学习率记为  $r$ ，则最基本的更新形式可以写成

$$w \leftarrow w - r \frac{\partial \text{Error}}{\partial w}$$

其中，学习率  $r$  决定每次更新走多远。它太小会导致学习很慢，太大则可能在误差曲面上来回震荡，甚至无法稳定下降。我们经常能见到的“误差曲面”和“局部极小值”图，想表达的就是这一优化视角：训练网络，本质上是在高维参数空间里寻找较低误差的区域。

那么，如何计算这里的  $\frac{\partial \text{Error}}{\partial w}$  呢？我们可以通过链式法则逐步展开，即将其写为两项相乘：

$$\frac{\partial \text{Error}}{\partial w} = \frac{\partial \text{Error}}{\partial O_i} \frac{\partial O_i}{\partial w}$$

先有

$$\frac{\partial \text{Error}}{\partial O_i} = -(d_i - O_i)$$

再结合 Sigmoid 的导数公式，以及输出节点输入和权重之间的关系，我们就不难得到最终的输出层权重更新公式

$$\Delta w_{ji} = r(d_i - O_i)O_i(1 - O_i)O_j$$

这里， $O_j$  即为前一层节点传来的输出， $w_{ji}$  表示从隐藏层第  $j$  个节点到输出层第  $i$  个节点的权重。这个公式有非常鲜明的结构，非常便于我们理解：1. 误差项  $(d_i - O_i)$  说明“错了多少”。显然，实际和理想差得越多，权重调整的步伐就越大；2. 导数项  $O_i(1 - O_i)$  说明“当前激活函数在这一点有多敏感”，即当前处于“斜坡”的什么位置？这是 Sigmoid 的导数。如果  $O_i$  接近 0 或 1（已经在平地了），说明网络很确信，这一项会变得很小，权重几乎不调；如果在 0.5 附近，则说明模型很犹豫，这一项最大，权重调整最剧烈。3. 输入项  $O_j$  说明“前一层传来的信号有多大”。显然，谁传过来的信号大，谁就要为这次的错误负更多的责任。

上述的公式已经不简单了，然而它只是最后一步的权重更新公式。但是实际上的层数可能更多，中间的某个隐藏层的权重该如何更新呢？这就是更难的一步了。

隐藏层本身没有直接对应的真实标签，没有一个具体的标准答案按进行比对。此时，就需要使用**反向传播** (backpropagation) 算法。它的核心思想是：输出层的误差信息虽然没有直接标注隐藏层，但有果必有因，我们可以利用最后的标签差距，去分析“前面某一层的参数要负多大责任”。我们可以借助链式法则，通过网络连接一层一层向后传递。于是，隐藏层权重的更新公式会带上来自后一层的加权误差信号。隐藏层权重更新形式也就可以写为

$$\Delta w_{kj} = rO_j(1 - O_j)O_k \sum_i w_{ji}O_i(1 - O_i)(d_i - O_i)$$

这里的推导步骤是与先前类似的，但比较冗长，这里就省略了。我们直接关注公式表达的根本含义：隐藏层节点的更新幅度，取决于它对后续误差到底贡献了多少。可以拆解为：1.  $O_j(1 - O_j)$  我们可以理解为“中层车间当前的迷茫度”，如果中层车间  $j$  当时的输出非常笃定（要么极度兴奋接近 1，要么极度冷淡接近 0），公式里这一项会变得极小，它几乎不受惩罚。但如果它当时算出来的结果是 0.5 左右，说明它摇摆不定。这时候这一项最大，它需要被狠狠地推一把，重新调整方向。2.  $O_k$  说明“前一层传来的信号有多大”。谁传过来的信号大，谁就要为这次的错误负更多的责任，这永远是不言而喻的。3.  $\sum_i w_{ji} O_i (1 - O_i) (d_i - O_i)$  是最终的最终的“总受灾信号”。由于中间产生的结果被传给了多个维度，我们要将所有受牵连的后续灾情合并起来。其中， $(d_i - O_i)$  说明“错了多少”，最显然； $O_i(1 - O_i)$  说明当前这个方向的“犹豫程度”； $w_{ji}$  说明当前这个方向的权重，权重越大，说明从  $j$  到  $i$  的影响越大，这条路当然就要负更多责任。

也就是说，反向传播并不是某种神秘技巧，而只是把“误差如何经过网络传播”这件事，用链式法则系统地计算了出来。

从整体流程看，反向传播训练一个多层感知机大致分为四步。先做一次前向传播，计算每层输出；再根据真实标签计算误差；接着用链式法则把误差信息从输出层往隐藏层传回去；最后更新全部权重。不断重复这一过程，网络就会逐渐学到较好的参数。

## 2.6 竞争学习、赢家通吃与 Kohonen 网络

前面讨论的感知机和反向传播，大多属于**有监督学习** (supervised learning)。也就是说，训练数据会明确给出输入以及它应对应的正确标签或目标输出。但联结主义并不只研究有监督学习。若数据只有输入，没有明确标签，网络是否仍能自己发现其中的结构？这就引出了**无监督学习** (unsupervised learning) 中的竞争学习思想。

没有标签，还怎么学习呢？我们可以举一个形象的例子：假设有一个社区，里面住着成千上万的居民（输入数据  $X$ ）。这时候，有 3 家外卖店（竞争节点  $w_1, w_2, w_3$ ）准备入驻。一开始，三家店的老板都不知道居民分布在哪里，他们的店址是随机乱选的。第一天一个住在 A 区的居民点外卖。系统一看，发现  $w_1$  这家店离他最近。于是  $w_1$  成了赢家，赚到了钱，就决定把店面往 A 区居民的方向搬迁一点点（权重向输入靠拢）。另外两家店没抢到单，原地不动。第二天一个住在 B 区的居民点外卖，发现  $w_2$  离他近。 $w_2$  成了赢家，也把店面往 B 区居民的方向搬迁一点点。随着成千上万个居民（数据点）不断点外卖，神奇的事情发生了：最终这 3 个节点的权重向量，完美变成了这 3 个居民聚集区的聚类中心，它们自动把整个庞大的社区划分成了 3 个势力范围。这就是无监督竞争学习：它通过自我调整，让每个节点分别去代言数据中的一个“自然聚集团”。

从中可见，竞争学习最经典的机制之一，就是**赢家通吃** (winner-take-all)。其基本做法是：给定一个输入向量  $X = (x_1, x_2, \dots, x_n)$ ，网络中多个竞争节点各自拥有一个权重向量，系统先比较谁与当前输入“最接近”，然后只更新那个获胜节点的权重。也就是说，并不是所有节点一起学习，而是谁最像当前输入，谁就被进一步拉向这个输入。

为了衡量“谁更像输入”，通常会使用距离度量。最常见的是欧氏距离。若某个节点的权重向量记为  $W$ ，输入向量记为  $X$ ，则二者的距离可以写成

$$\|X - W\| = \sqrt{\sum_{i=1}^n (x_i - w_i)^2}$$

在实际比较时，也常直接比较平方距离，因为平方根不会改变大小规律。选出获胜节点之后，其权重常按如下方式更新：

$$W_t = W_{t-1} + c(X_t - W_{t-1})$$

这里， $W_{t-1}$  是更新前的权重向量， $X_t$  是当前输入样本， $c$  是学习率。这个式子很好理解：新权重等于旧权重加上 “一小步朝向当前输入” 的位移。因此，若某个节点经常赢得某一类样本，它的权重就会逐渐移动到那一类样本的代表位置附近，形成一个**原型** (prototype)。

Kohonen 型竞争学习网络正是这一思路的简单体现。假设初始有两个原型点，输入数据分布在平面上形成两个簇，那么随着训练进行，一个原型会逐渐靠近其中一个簇，另一个原型会逐渐靠近另一个簇。于是，即使没有显式标签，网络也能根据输入分布自动形成某种聚类结构。这说明联结主义并不只是 “给定标签然后逼近答案”，它也可以通过竞争和自组织，从数据本身的分布中提取结构。

从更高层次看，竞争学习与前面讨论的感知机、反向传播代表了两种不同的学习观。前者更强调 “根据相似性形成原型和类别中心”，后者更强调 “根据标签误差反向调整参数”。二者都属于联结主义，但适用任务不同。一个偏向聚类和表示组织，一个偏向监督预测和分类。把这一点看清后，读者就会明白：联结主义并不是单一算法，而是一整类围绕网络连接、参数学习和分布式表示展开的方法体系。

## 2.7 从模式识别到完整分类系统

学到这里，可以把这一章的内容放回更完整的任务链条中来看。现实中的模式识别系统，通常并不是直接把原始世界交给某个分类器就结束了。更完整的过程往往包括：先通过传感器或换能器采集原始数据，再做特征提取，把原始数据转成较稳定、较紧凑的特征表示，最后再由分类器给出结果。也就是说，分类器通常只是整个识别系统的后半段。

这条链路之所以重要，是因为它提醒我们：神经网络不是孤立存在的。它既可以担当分类器，也可以承担特征提取器，甚至在深层网络中同时完成这两件事。传统模式识别更常把 “特征提取” 和 “分类” 明确分开，而深度学习则常常把它们合并进一个端到端模型中，让网络自己学出有效表示。联结主义的发展方向，也正是在不断增强这种自动表示学习能力。

从任务类型看，联结主义方法常见于分类、模式识别、记忆回忆、预测、优化和噪声过滤等问题。分类强调判断输入属于哪一类；模式识别强调发现数据中的结构；记忆回忆强调从部分线索恢复完整信息；预测强调由症状推测疾病、由原因推测结果；优化强调在约束下寻找较优方案；噪声过滤则强调从混杂信号中分离出有效成分。把这些任务并列起来看，就会发现联结主义真正强大的地方不在于某一个特定模型，而在于它提供了一种统一视角：**通过网络结构与参数学习，把输入映射成所需输出或内部表示。**

### 本章小练习

#### 1. 判断题

单层感知机之所以不能解决异或问题，是因为训练迭代次数还不够多，而不是模型本身能力受限。(\_\_\_\_)

#### 2. 计算题

设感知机当前输入为  $x = (1, 0)$ ，真实标签为  $d = 1$ ，模型当前输出为  $y = -1$ ，学习率为  $r = 0.2$ 。求权重增量  $\Delta w$ 。

#### 3. 简答题

请说明为什么多层感知机能够突破单层感知机的局限。回答时应包含 “线性可分” “隐藏层” “非线性激活” 这三个关键词。



## 参考解答

1. 错。异或问题的根本困难不在于训练不充分，而在于它不是线性可分的。单层感知机对应一个线性分类边界，因此无论如何更新参数，都不可能用一条直线把异或数据完全分开。
2. 由感知机更新公式

$$\Delta w = r(d - y)x$$

代入  $r = 0.2$ 、 $d = 1$ 、 $y = -1$  和  $x = (1, 0)$ ，得到

$$\Delta w = 0.2(1 - (-1))(1, 0) = 0.2 \times 2 \times (1, 0) = (0.4, 0)$$

因此，本次更新应把第一个权重增加 0.4，第二个权重不变。若把偏置也视作与常数输入 1 相连，则偏置同样需要按相同原则更新。

3. 单层感知机只能形成一个线性分类边界，因此只适合处理**线性可分**问题。多层感知机通过加入**隐藏层**，先把原始输入映射到新的中间表示空间；再配合**非线性激活函数**，使网络不再只是简单的线性变换叠加。这样一来，网络就能表示复杂的非线性决策边界，从而处理像异或这类单层感知机无法解决的问题。

---

## 第三章 深度学习、表示学习与序列建模

### 本章概要

进入深度学习之后，人工智能的视野会一下子变得非常宽。前一章主要讨论的是人工神经元、感知机、多层感知机与反向传播，那时我们关心的重点还是“网络怎样训练起来”。而这一章要继续向前走，进一步回答：为什么需要更深的结构，为什么“表示”本身比分类器还重要，分类任务中常用的损失函数应该怎样设计，卷积与池化为什么适合处理图像和序列，序列数据为什么会催生 HMM、CRF、RNN、Transformer 等不同模型，以及现代生成模型又是怎样发展的。

这一章的知识跨度很大，因此学习时尤其不能只盯着模型名称。更重要的是抓住其中几条主线。第一条主线是**表示学习**：深层网络之所以强，不只是因为层数多，而是因为它能逐层构造更有用的特征。第二条主线是**优化与目标函数**：无论是逻辑回归、Softmax 还是更复杂的深层网络，最终都离不开损失函数、梯度和正则化。第三条主线是**结构与数据类型的匹配**：图像、文本、语音、序列、生成任务，它们各自适合怎样的网络结构。只要把这几条主线看清楚，整章就不会显得支离破碎。

### 3.1 为什么需要深度学习

深度学习最常见的一个问题是：既然浅层模型已经可以做分类，为什么还需要更深的网络？这个问题表面上是在比较“层数”，实际上是在比较**表示能力**。浅层结构通常只能在单层次上组织特征，而很多现实问题并不是单层结构就能自然表达的。例如，一张图像中的边缘可以组合成简单纹理，简单纹理再组合成局部形状，局部形状再组合成完整物体；一句话中的字可以组合成词，词组合成短语，短语再表达更高层语义。若模型只能停留在单层变换上，那么它就很难高效地利用这种层次结构。

深层结构的关键优势，在于它允许特征进行**逐层分解与逐层组合**。较低层先识别简单模式，较高层再把这些简单模式重新组织成更抽象的表示。这样做往往比一次性用一个很大的浅层模型去拟合复杂关系更经济，因为同一组低层特征可以被许多高层结构反复复用。

你可能会想：一层神经元的目的不就是传向下一层吗，怎么能复用呢？实际上，神经网络的层与层之间，通常是“全连接”或“多通道连接”的。这意味着，低层的一个神经元学到的特征，可以同时把信号传给高层的多个不同神经元。

我们来看一个例子：假设低层有几个神经元非常聪明，它们合力学会了如何识别一个特征：圆形的轮子。在深层网络中，这个圆形的轮子特征会被后面的高层网络疯狂地压榨和复用：高层神经元 A（负责识别自行车）：它会跑过来调用这个特征。高层神经元 B（负责识别小汽车）：它也会跑过来调用这同一个特征。高层神经元 C（负责识别飞机）：飞机降落也需要起落架，它也跑过来调用这个特征。发现了吗？低层网络只需要花一份力气，把“轮子”这个基础特征训练好；高层网络在识别自行车、汽车、飞机（它们完全可能在不同层数）时，直接拿来用就行了。也正因为如此，人们常说深层网络能够以更节省神经元的方式表达复杂函数。

进一步说，深度学习真正改变的并不只是分类器，而是“特征从何而来”这件事。在传统模式识别中，人们往往先手工设计特征，再把这些特征送进分类器；而深度学习则试图让网络自己从数据中学出多层表示。于是，重点就从“怎样在固定特征上做分类”转向“怎样同时学出好的特征与好的分类器”。这也是为什么常说：若表示学得足够好，后端分类器的重要性反而会下降。并不是分类器不重要，而是一个好的表示已经把问题简化了很多。

深度学习在图像识别、语音识别、自然语言处理等任务中的突破，都可以从这个角度理解。图像中的局部边缘、语音中的短时频谱模式、语言中的上下文语义，都不是单一步骤可以直接抓住的，它们更适合由层层抽象的结构去逐级提取。因此，深度学习并不是简单地“把网络堆深”，而是在利用现实数据中本来就存在的层次性。

### 3.2 分类、损失函数与表示空间

深度学习虽然很复杂，但很多任务最终仍可归结为一个基本问题：给定输入  $x$ ，输出一个预测结果  $y(x)$ ，并希望这个预测尽量接近真实目标。要做到这一点，首先需要明确什么叫“预测得好”，这就要定义**损失函数** (loss function)。

若输出是一个连续值，最常见的损失之一是平方误差：

$$\text{Error} = \frac{1}{2} \sum_{i=1}^n (t_i - y(x_i))^2$$

其中， $t_i$  表示第  $i$  个样本的真实目标， $y(x_i)$  表示模型在该样本上的输出。这个公式的思想很直接：预测值和真实值差得越大，损失就越大。平方的作用是让正负误差都转化成非负值，同时放大大误差。

但是在分类问题中，尤其是输出代表概率时，平方误差往往不是最自然的选择。更常见的做法是使用**交叉熵** (cross entropy)。在二分类情形中，若标签  $t_i \in \{0, 1\}$ ，模型输出  $y(x_i)$  可以解释为样本属于正类的概率（1 表示正类，0 表示负类），那么对数似然对应的损失可以写为

$$J(\theta) = -\frac{1}{n} \sum_{i=1}^n [t_i \ln y(x_i) + (1 - t_i) \ln (1 - y(x_i))]$$

这里， $\theta$  表示全部模型参数， $n$  表示样本数。这个公式之所以重要，是因为它和概率模型天然匹配：当真实标签为 1 时，希望  $y(x_i)$  越接近 1 越好；当真实标签为 0 时，希望  $y(x_i)$  越接近 0 越好。若模型把真实类别赋予了极小概率，那么

对数项会带来很大惩罚。公式开头加上负号表示，模型希望这个损失  $J(\theta)$  越小越好。

二分类中最经典的概率模型是**逻辑回归** (logistic classification)。它并不是“回归成一个连续值”，而是先构造一个线性函数

$$f_{\theta}(x) = \theta_0 + \theta_1 x_1 + \cdots + \theta_d x_d$$

再通过 Sigmoid 函数把它变成 0 到 1 之间的概率：

$$y(x) = \frac{1}{1 + \exp(-f_{\theta}(x))}$$

因此，逻辑回归虽然最终输出的是概率，但它的核心仍然是在建模一个线性打分函数，只是这个打分结果经过了非线性压缩。对应的一个重要概念是 **logit**，也就是对数几率：

$$g(y(x)) = \ln \frac{y(x)}{1 - y(x)} = f_{\theta}(x)$$

前面的式子是在用  $f_{\theta}(x)$  来计算  $y(x)$ ，即用线性结果来映射到概率空间；而后面的式子则是反向的过程，将在  $[0, 1]$  范围的概率映射到  $(-\infty, \infty)$  范围的对数几率。

这也说明，逻辑回归真正线性建模的，其实不是概率本身，而是概率对应的对数几率。理解这一点很有帮助，因为它说明逻辑回归并不是凭空出现的技巧，而是“线性模型 + 概率解释”的自然结合。

若类别不止两个，而是  $k$  个，那么就需要从二分类推广到 **Softmax 回归**。这时模型不再输出一个概率，而是输出一个  $k$  维概率向量，每一维表示属于某一类的概率，并满足总和为 1。若第  $j$  类的打分函数记为  $f_{\theta_j}(x)$ ，则有

$$p(t = j | x; \theta) = \frac{\exp(f_{\theta_j}(x))}{\sum_{\ell=1}^k \exp(f_{\theta_{\ell}}(x))}$$

其中的  $p(t = j | x; \theta)$  就表示：在输入内容为  $x$ 、模型参数为  $\theta$  的条件下，模型预测“推测目标  $t$  恰好是第  $j$  类”的概率是多少。

这就是 Softmax 的本质：先对每个类别给出一个实数打分，再通过指数归一化把它们转换成概率分布。这里用指数函数有两个好处。第一，它能保证分子分母都为正；第二，打分高的类别会在概率上得到更明显的放大。

若把真实类别写成指示函数  $I$

$t_i = j$ ，则 Softmax 的交叉熵损失可以写成

$$J(\theta) = -\frac{1}{n} \sum_{i=1}^n \sum_{j=1}^k I t_i = j \ln p(t_i = j | x_i; \theta)$$

这说明：多分类问题的训练，本质上仍是在最大化真实类别的概率，只不过现在要在多个类别之间同时竞争。

在实际训练中，仅仅把训练误差压低还不够，因为模型可能会记住训练集中的偶然噪声，导致对新样本表现变差，这就是**过拟合**。为缓解这一问题，常见做法是在损失函数中加入参数惩罚项，也就是**权重衰减** (weight decay) 或  $L_2$  正则化：

$$J(\theta) = J_0(\theta) + \frac{\lambda}{2} \sum_j \theta_j^2$$

其中， $J_0(\theta)$  是原始损失， $\lambda$  是正则化系数。这个额外项会惩罚过大的参数，从而鼓励模型保持更平滑、更简单的解。其梯度也会相应多出一项：

$$\nabla_{\theta} J(\theta) = \nabla_{\theta} J_0(\theta) + \lambda \theta$$

这一步在直觉上可以理解为：模型不仅要拟合数据，还要避免把参数拉得过于极端。

这里还需要补充一个非常重要的思想，即**基函数展开** (basis expansion) 与表示空间变换。原始输入空间中如果边界是弯曲的、复杂的，那么我们完全可以先把输入  $x$  映射到一个新的特征空间  $\phi(x)$ ，再在新空间上使用线性模型。这也是多项式曲线拟合、Gaussian 基函数、混合 Gaussian 特征等方法共同思路。深度学习进一步把这个过程自动化：它不再依赖人工写出固定的基函数，而是由网络自己逐层学习表示。于是，“特征工程”就被部分转化为“表示学习”。

### 3.3 自编码器、深层表示与卷积网络

在前一节，我们已经了解了如何“在已有表示上分类”。比方说，我们要让 AI 认识一件衣服。原始表示  $x$  就是照相机拍下的  $100 \times 100$  个像素点的颜色数字。对计算机来说，这 10,000 个数字极其混乱，根本看不出规律。在深度学习中，照片到了中间某层，这些像素数字已经浓缩成了 3 个新数字，比如 [袖子长度, 衣服颜色, 有无领子]，它们并没有丢失衣服的核心灵魂，反而把原始混乱的像素提炼成了精简的特征。这 3 个数字组成的向量就是更好、更新的“表示”。随后，我们再通过交叉熵损失来调参，让模型分类得更好。

那么这一节，我们要进一步讨论“表示本身怎样被学出来”。在这方面，自编码器 (autoencoder) 是最典型的入门模型之一。

自编码器的结构看上去很简单：输入先进入一个**编码器** (encoder)，被压缩成一个中间表示；然后这个中间表示再经过**解码器** (decoder)，尽量恢复出原始输入。若输入记为  $x$ ，编码器输出隐变量表示  $z$ ，则可以写成

$$z = f_{\text{enc}}(x)$$

解码器再给出重构结果

$$\hat{x} = f_{\text{dec}}(z)$$

训练目标通常是让  $\hat{x}$  尽量接近  $x$ 。表面上看，这像是在“把输入抄一遍”，但真正关键的地方在于中间层  $z$  往往被限制得更小，或者被施加某种结构约束。于是，网络就不得不学会保留输入中最重要的部分，而丢掉冗余信息。这样得到的  $z$  就是一种紧凑的表示。拿前面的“让 AI 识别一件衣服”作为例子：网络被逼着要把 10,000 个数字压缩成 3 个数字，而

且还要用这 3 个数字把原图完美还原出来。这说明，这 3 个数字必须要把衣服最核心、最不可丢失的特征死死抓住。所有无关紧要的背景噪声和杂质，在压缩过程中都会被迫丢弃。

自编码器的重要意义在于，它把“如何构造有用特征”变成了一个可训练问题。早期深层网络的有效训练，很大程度上依赖于这种逐层无监督预训练思想：先用自编码器或深度信念网络（DBN）逐层学习表示，再在此基础上做监督微调。DBN 是什么呢？我们知道自编码器坚信世界上有一种明确的数学公式，能把像素压缩成特征。而 DBN 则不觉得世界上有确定的公式。它认为数据背后隐藏着一种“看不见的概率分布”。它是把一种叫 RBM（受限玻尔兹曼机）的微型概率网络像搭积木一样，一层一层往上叠。假设输入是一堆猫的照片。第一层 DBN 会去猜测：“这些猫的照片背后，是由哪些隐藏的‘概率因子’（隐藏层）组合出来的？”它通过不断调整参数，让隐藏层能够以最大的概率“解释并生成”输入层的猫照片。学完第一层，把第一层的输出当成新的输入，往上再盖第二层，继续去猜更高级的概率因子。

这就是早期 AI 的训练套路了。当时网络深了训不动，科学家们就想出了这个“套娃”预训练的妙招。虽然现代大规模训练方法已经不再像早期那样强依赖逐层预训练，也不用逐层 AE 或 DBN 了，但自编码器所体现的表示学习思想仍然非常核心。

接下来要讨论的，是图像处理中最重要的一类结构：**卷积神经网络**（convolutional neural network, CNN）。卷积之所以适合图像，是因为图像中的模式通常具有局部性和平移相似性。同一个边缘、纹理或角点，不会只出现在图像固定位置，而可能在不同区域重复出现。若为每个位置都单独学习一套参数，既浪费，也不符合问题本身的结构。

卷积的基本思想，就是让一个小窗口在输入上滑动，并在每个位置重复使用同一组参数。若一维输入为  $x$ ，卷积核为  $w$ ，则离散卷积的一种常见写法是

$$(x * w)(t) = \sum_{\tau} x(\tau)w(t - \tau)$$

在图像任务中，卷积核会变成二维窗口；在语音或文本中，也常使用一维卷积。卷积之后得到的一张张输出图，称为**特征图**（feature map）。每一个卷积核都会专门响应某一类局部模式，因此多组卷积核就能产生多张不同的特征图。

为什么要不断移动一个窗口呢？特征图又到底在表示什么呢？读者千万不要被那个看似复杂的公式吓到。在图像处理中，卷积的实际动作极其直观。假设我们的卷积核  $w$  是一个  $3 \times 3$  的权重矩阵。这个卷积核通过之前的反向传播，已经学会了识别“左斜线”。所以它的参数长这样：

$$w = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

现在，我们让这个  $3 \times 3$  的小窗口，像一个放大镜一样，从左到右、从上到下地在整张大照片上滑行。每移动到一个位置，就做一件事情：将窗口覆盖下的 9 个图像像素值，和卷积核的 9 个权重，对应位置相乘，最后全部加起来。此时，如果当前滑到的这个局部区域全是一片空白，或者是别的东西（比如一条横线），那么算出来的最终得分会非常低。但如果当前滑到的这个局部区域，刚好有一条小猫胡须（左斜线），那么算出来的最终得分就会非常高（比如 3.0）。所以，这个计算的本质是：卷积核在全图扫描，每到一个地方就打得分，测一测这个地方跟自己长得像不像。

那么什么是特征图呢？当这个卷积核在整张大照片上全部滑完一遍后，它在每一个停留的位置都会留下一个得分。把这些按照空间顺序排列好的得分组合在一起，形成的一张全新的二维数字矩阵，它就是特征图。在上一节，我们学到了“表

示“的概念。而特征图，就是图像在 CNN 这一层学出来的“表示”。如果原始照片是一张黑白的猫咪图，那么经过卷积计算后，输出的特征图在人类眼里看起来，会是一张黑乎乎的背景，只有猫咪身上所有带有“左斜线”方向的胡须和毛发，散发着明亮的白色高光（得分高的地方）。

在卷积后，我们还要进行非线性激活。这里的原因和之前的多层感知机是相同的：如果卷积核只学习到线性关系，那么无论多少层的叠加，最后输出的特征图仍然是线性的。因此，我们一定要加入激活函数。在现代 CNN 中，最常用的就是 ReLU 函数：

$$f(x) = \max(0, x)$$

它的动作极其简单：只要卷积算出来的得分是负数，一律变成 0；正数则原样保留。当它紧跟在卷积后面时，其工作可以被形象地理解为：提取真正有用的特征信号，清除无关的背景杂质。也就是说，负分代表这里根本没有我们要的特征，只是些干扰的背景噪声，ReLU 就直接把它们全部变成 0。经过这一步，特征图上的噪声被成片地抹去，只留下了真正有价值的重点，网络就具备了极强的特征选择能力。

在这之后，常常还会接**池化层**（pooling layer）。池化并不重新学习复杂参数，而是对局部区域做压缩汇总。常见的池化方式有最大池化、平均池化等，其中最大池化最常用。池化的作用主要有两个：第一，降低空间分辨率，从而减少后续计算量；第二，在局部区域内引入一定程度的不变性，使模型对小幅平移和细小扰动不那么敏感。

因此，一个典型的卷积网络通常会经历这样的过程：输入图像先经过卷积提取局部模式，再经过非线性激活，随后通过池化进行压缩，之后不断重复，直到高层获得较抽象的表示，最后再交给全连接层或分类头输出结果。随着层数加深，网络看到的“感受野”越来越大，识别的特征也从局部边缘逐渐上升到纹理、部件和整体对象。

在实际训练卷积网络时，参数初始化、学习率、mini-batch、大致的偏置范围、归一化、Dropout 和优化器选择都很重要。这些内容虽然带有较强工程色彩，但本质上仍然服务于同一个目标：让梯度传播更加稳定、让训练更加高效、让模型更不容易过拟合。对于初学者而言，至少应当记住一点：深度学习的成功并不只靠“模型想法好”，还依赖大量训练细节的配合。

### 3.4 序列数据、中文分词与 HMM

前面讨论的卷积与图像，重点在于空间结构；而一旦处理语音、文本这类数据，问题就会变成**序列建模**。在序列中，当前位置的含义往往与前后位置密切相关，因此模型不仅要看局部内容，还要考虑顺序关系。

中文分词是一个非常典型的序列标注问题。英文天然以空格分词，而中文句子写出来往往是连续字符，因此系统需要判断哪里应该切开、哪里不应切开。这个任务看似简单，实际上困难很多。首先，同一串字可能有不同切分方式，例如组合歧义、交叉歧义、全局歧义都会出现。其次，大量新词、人名、地名、机构名、时间数量表达和领域专名都会不断产生，这使得“未知词”问题始终存在。也就是说，中文分词并不是单纯查字典，而是一个需要结合上下文进行判断的结构化预测问题。

为把分词问题转化成模型可处理的形式，一种常见方法是**字标注**。例如对每个字赋予 B、I、E、S 等标签，分别表示“词首”“词中”“词尾”“单字成词”。于是，分词任务就转化为：给定一个字序列，预测对应的标签序列。只要标签序列确定，分词结果也就随之确定。

在传统序列模型中，最经典的方法之一是**隐马尔可夫模型**（Hidden Markov Model, HMM）。HMM 的思想是：真实标签序列看不见，是隐藏状态；我们实际观察到的是字、词或语音特征等观测序列。模型通过状态转移概率与发射概率，

把“状态如何变化”和“状态如何产生观测”联系起来。

一个 HMM 通常由五元组描述：

$$\mu = (S, K, \Pi, A, B)$$

其中， $S$  是状态集合， $K$  是观测符号集合， $\Pi$  是初始状态概率， $A$  是状态转移概率矩阵， $B$  是发射概率。对于分词任务，可以把状态理解为标签，例如 B、I、E、S；观测则是字序列。

HMM 中最基本的三个问题值得反复记住。第一，**评估问题**：给定模型参数，怎样计算一个观测序列出现的概率  $P(O | \mu)$ 。第二，**解码问题**：给定观测序列和模型，怎样找到最可能的隐藏状态序列。第三，**学习问题**：给定观测数据，怎样反推出更合适的模型参数。

评估问题通常用**前向算法**解决。定义前向变量

$$\alpha_i(t) = P(o_1 o_2 \cdots o_{t-1}, X_t = i | \mu)$$

它表示：到时刻  $t$  为止，已经观测到前面的序列，并且当前处于状态  $i$  的联合概率。通过递推计算全部  $\alpha_i(t)$ ，最后再求和，就能得到整个观测序列的概率。与之对应，还可以定义后向变量

$$\beta_i(t) = P(o_t \cdots o_T | X_t = i, \mu)$$

前向与后向结合，不仅能高效求概率，也能为后面的参数估计提供中间量。

第二个问题是解码问题，它和之前自编码器中的解码的含义截然不同。这里要做的工作是：给定观测序列和模型，找到最可能的隐藏状态序列。比如你连续 5 天在朋友圈看到你朋友的动态是 [宅, 宅, 出门, 宅, 出门]，要在脑海里推测这几天最可能的天气序列是 [雨, 雨, 晴, 雨, 晴]。

解码问题通常由 **Viterbi 算法** 解决。它和前向算法很像，也是在网格状状态图上做动态规划，但目标不是求所有路径概率之和，而是找概率最大的那条路径。若定义

$$\delta_j(t) = \max_{X_1, \dots, X_{t-1}} P(X_1, \dots, X_{t-1}, o_1, \dots, o_{t-1}, X_t = j | \mu)$$

那么它表示在第  $t$  个位置处于状态  $j$  时的最优路径分数。配合回溯指针，就能恢复出最可能的完整状态序列。

学习问题则通常借助 EM (Expectation-Maximization) 思想来完成，也就是 Baum-Welch 类型的方法。其核心是通过前向后向变量计算各状态与转移的期望出现次数，再据此更新初始概率、转移概率和发射概率。

具体流程是：1. 初始化：一开始既然什么都不知道，算法先随机填一组参数。2. E 步 (Expectation, 算期望)：拿着这组参数，配合前向后向变量，去对着成千上万条数据做推算，即计算各状态与转移的期望出现次数。3. M 步 (Maximization, 重新估计参数)：根据记下的结果，重新数数、算百分比，来更新这些参数。4. 循环：拿着第三步刚修正好的、变聪明了一点的参数，重新回到第二步去更精准地记账，再去更新参数，如此往复。

随着这个过程重复几十次，收敛就会发生：一开始随机瞎猜的参数会一点点漂移，最终慢慢契合数据的真实规律，从而把真正的转移概率和发射概率全部自动“学”了出来。

EM 思想的实际的推导较长，这里不做展开。但从思想上看，就是先根据当前模型估计哪些状态和转移更可能出现，再据此重新估计参数，如此反复迭代。

### 3.5 从 HMM 到 CRF：条件建模与标注偏置

HMM 虽然经典，但它有一个明显限制：它必须对“状态如何生成观测”作出较强假设，因此往往很难灵活地加入丰富特征。现实中的文本任务常常不只是依赖当前字本身，还依赖左右上下文、字串组合、词形模式乃至领域知识。若继续坚持生成式建模，模型就会显得很僵硬。

这时就需要引入**条件随机场** (Conditional Random Fields, CRF)。CRF 与 HMM 的根本区别在于：HMM 建模的是联合概率或生成过程，而 CRF 直接建模条件概率  $P(Y | X)$ ，也就是“给定观测序列时，标签序列的条件分布”。这里的  $X$  可以理解为包含了当前字、左右上下文、乃至全局统计特征的超级信息体，而  $Y$  就是一串类别标签。

正因为不必显式描述观测是怎样被生成的，CRF 就可以更加自由地使用特征函数。例如，在中文分词中，我们完全可以定义这样的特征：当前位置字是否为“就”，左边一个字是否为“小”，右边一个字是否为“开”，当前位置与前一个标签的组合是否满足某种模式，等等。于是，模型不再局限于“转移概率 + 发射概率”两种固定结构，而是可以使用大量**特征模板**去刻画局部上下文。

这类特征常写成函数  $f(S_0, C_0)$ 、 $f(S_{-1}, S_0, C_0)$  这类形式。这里的  $S$  表示状态或标签，负下标和正下标分别表示当前位置左边和右边的相对位置；而  $C$  则表示字符或观测。比方说上面的第二个例子就是指“在前一个标签是  $S_{-1}$ 、当前标签是  $S_0$  且观测到特征  $C_0$  的情况下，这种状态转移是否合理”。通过模板系统，我们可以灵活地把单字特征、相邻字特征、标签转移特征以及更复杂的组合特征都纳入模型中。

CRF 之所以在序列标注中很受重视，还有一个重要原因，即它能缓解**标注偏置问题** (label bias problem)。某些局部归一化模型在做决策时，容易过度依赖当前状态的局部转出概率，从而忽视后续观测对整体路径的影响。直观地说，如果某个状态的出口很少，那么它可能因为“局部归一化的便利”而被错误地偏爱（因为出口多的状态的转移概率被稀释掉了）。CRF 通过对整条序列进行全局归一化，能更好地让不同路径在整体上竞争，而不是只做局部比较。

因此，从 HMM 到 CRF 的转变，体现的并不仅是“换了个模型”，而是思考方式发生了变化：从生成式的概率分解，转向条件式、特征化、全局归一化的结构化预测。对于中文分词、词性标注、命名实体识别等任务，这种转变尤其重要。

### 3.6 循环网络、上下文表示与 Transformer

虽然 HMM 和 CRF 仍然非常有价值，但它们主要依赖人工设计的离散特征。深度学习进入自然语言处理之后，一个核心目标变成了：能否让模型自己学出上下文表示，而不必过度依赖手工模板。

为此，首先发展起来的是**循环神经网络** (recurrent neural network, RNN)。RNN 的特点在于，它会把前一时刻的隐藏状态传递到后一时刻，因此天然适合处理序列。简单说，网络读到当前位置时，不仅看当前输入，还会“记住”前面的一些信息。

但普通 RNN 在长序列上容易出现梯度消失或梯度爆炸，导致长期依赖难以学习。为缓解这一问题，出现了 **LSTM** 与 **GRU** 等门控结构。它们通过输入门、遗忘门、输出门或更简化的门控机制，控制信息该保留多少、忘记多少、输出多少。



其中，LSTM 有 3 个门（遗忘、输入、输出）和 2 条信息传输通道（细胞状态  $c_t$  和隐藏状态  $h_t$ ）。在每个时刻，遗忘门评估之前的信息，决定哪些从旧状态中丢弃；输入门筛选当前的新输入，把有价值的写入记忆中；输出门则基于最终记忆加工出当前时刻的隐藏状态。GRU 只有 2 个门（更新、重置）和 1 条信息传输通道（隐藏状态  $h_t$ ）。它只用一个“更新门”来决定保留多少旧记忆、吸收多少新信息，将输入和输出的工作打包在了更新门之中，因此其结构比 LSTM 更精简、运算速度更快。

初学时我们先不必记住所有门的细节，更重要的是理解它们的目标：让网络在更长的时间跨度上保留有用信息。

随着发展，人们进一步发现，仅靠从左到右或从右到左的递推，还不足以充分表达复杂上下文。于是，注意力机制（attention）开始兴起。注意力的核心思想是：处理当前位置时，不必一视同仁地参考所有位置，而是可以对更相关的部分赋予更大权重。这样，模型就能根据任务需要动态聚焦于真正重要的上下文。

在这一基础上，Transformer 出现了。Transformer 的关键突破在于，它基本抛开了传统递归结构，而主要依赖注意力机制来建模序列中的依赖关系。这样做有两个明显优点。第一，任意两个位置之间的关联都可以更直接地建立，而不必经过很长的递推路径。第二，计算更适合并行化，因此在大规模训练中效率更高。

后来 GPT、BERT 等模型正是在 Transformer 框架上发展起来的。BERT 通过掩码语言模型学习双向上下文表示，GPT 则更强调自回归（Autoregressive）生成能力。

BERT 的核心动作就是做“完形填空”题。训练方法很简单：拿来一句话，比如“小米手机今天公开测试”。BERT 会随机把中间的几个字用一个特殊的遮罩 [MASK] 盖住，变成：“小米 [MASK] 机今天公开测试”，此时，BERT 的任务就是猜出被盖住的字是“手”。因为左右两边的信息同时无阻碍地流向中间，就叫“双向上下文表示”。通过大量这样的完形填空，BERT 练就了极强的“阅读理解”能力，能深刻理解语境的中间特征表示。但它的局限在于不擅长写长文章，因为现实里说话并不是完形填空。

而 GPT 的核心动作则是玩“文字接龙”游戏。训练方法是给出一句话的前半句，让它去预测下一个字词是什么。比方说给 GPT 输入“小”，GPT 预测“米”，进而连成“小米”。然后再把“小米”喂回当前的输入，继续预测下一个字词。因此，这里自回归的意思就极其纯粹了：过去说出来的话，重新变成自己下一步的输入。在计算概率时，它用的是我们前面学过的多分类 Softmax 公式。

它们二者的共同点在于：都把“学表示”这件事推到了前所未有的重要地位。与其为每个任务单独设计特征，不如先训练一个强大的语言模型，让它内部已经拥有丰富的语言知识，再通过微调或提示来适应下游任务。而其核心区别是：BERT 能够看到左右两边的上下文，但 GPT 在算“下一个字的概率”时，绝对看不到右边（未来）的字，只能依赖左边（过去）已经生成的字。因此，GPT 的优势就在于：由于这种单向接龙的机制和人类写字、说话的顺序一模一样，所以 GPT 天生具备了长文本生成能力。今天和 ChatGPT 聊天，就是这样一个字一个字“接”出来的。

由此又进一步发展出提示学习（prompt-based learning）、混合专家模型（mixture-of-experts）等新框架。对于初学者来说，这些方法的细节可以后续再深入，但至少应先理解它们的共同逻辑：深度学习正在把“先学通用表示，再适配具体任务”变成主流路线。

### 3.7 PCA、自编码器与生成模型

除了判别任务，深度学习还大量用于表示压缩与数据生成。在这一方向上，一个经典的线性方法是**主成分分析**（principal component analysis, PCA）。PCA 的目标，是在尽量保留数据方差的前提下，把高维数据投影到低维空间。更准确地说，它寻找一组新的正交坐标轴，使得第一主成分方向上的投影方差最大，第二主成分在与第一主成分正交的条件下方差最大，依此类推。

为什么要看方差呢？这是因为方差越大，说明这个方向上的数据越丰富、越没有被埋没；方差越小，说明这个方向上的数据全缩成了一团，变成了毫无用处的死水。

我们以相亲挑选特征的场景为例：要把一堆相亲对象的无数特征，压缩成一个最重要的“主成分”，即只能留一个指标。现在我们手里有两个备选特征。特征 A 是所有人抽烟的频率：但调查发现，现在大家都不抽烟，所有人全都是 0。这意味着，这个特征的方差为 0，没有任何波动。特征 B 是每个人的年收入：在这里，有人年入 5 万，有的人年入 100 万。这个特征的方差极大，波动非常剧烈。请问，如果要“找重点”去区分这群人，你会选特征 A 还是特征 B？你一定会选年收入。因为特征 A 中所有人都是 0，它对于区分这群人没有提供任何有用的信息，数据在这个方向上全部缩成了一个点。所以，PCA 寻找方差最大的方向，就是为了不让数据的这种区分度在降维时被抹杀掉。

我们再看一个二维空间里的例子。假设我们有一堆散落的数据点，它们在空间里呈现出一个斜着的椭圆形分布。现在，要求是必须把二维数据变成一维。此时，如果顺着水平方向把所有点拍扁到 Y 轴上，可能就会造成一些投影过去的落点几乎重合在了一起。它们原本在二维空间里离得很远，但现在它们变得无法区分了。而 PCA 的聪明选择则是寻找主成分轴：它发现，这个椭圆形的长轴方向（斜线方向），是数据延展得最长的方向。因此，这个长轴方向，就是第一主成分（PC1）。

PCA 的价值在于，它告诉我们：表示学习并不一定都靠深层网络，早期统计学习已经在研究怎样找到“最有信息量的方向”。后来自编码器可以看作把这种“降维与重构”的思想推广到了非线性情形。若编码器和解码器都是线性的，并且一些条件满足，自编码器学出的子空间与 PCA 甚至会有密切联系。

再往前一步，就是各种**生成模型**。生成模型不只是判断“输入属于哪一类”，而是试图学习数据本身的分布。我们希望模型能够学得  $p(x)$ ，从而能够采样生成新样本，或直接估计某个样本出现的可能性。

**变分自编码器**（variational autoencoder, VAE）就是一个非常典型的例子。普通自编码器只是把输入压成一个向量再重构，而 VAE 会把隐变量看成随机变量，并试图学习一个潜在分布。如何理解呢？我们知道，普通的自编码器只能死记硬背。它可能在中间层记住了 1.5 和 3.5，如果在中间层随机摇出了一个 2.5，由于中间层没有规律，解码器大概率没法给出正常的结果。然而，VAE 要求所有特征必须符合最标准的正态分布。它让编码器去学的不是一个死数字，而是去学数学里的均值  $\mu$  和方差  $\sigma$ 。由于整个中间特征空间被强行约束得非常规范、连续且没有空隙，后续想要生成时，只需要在标准的正态分布里随便采样，解码器都能稳稳当地把它做出来。

在其中，通常引入编码分布  $q(z|x)$  和生成分布  $p(x|z)$ ，并通过变分推断去优化一个可计算的下界。完整推导较长，我们在初学时可以先把它理解为：希望学出一个结构良好的潜在空间，使得在潜在空间中采样得到的点，经解码器后能够生成合理数据。训练中的一个关键技巧是**重参数化技巧**（reparameterization trick），它把随机采样写成可微形式，从而允许梯度传播。

另一类非常著名的生成模型是 **GAN**（generative adversarial network）。GAN 的思想极具代表性：让一个生成器负责造假样本，让一个判别器负责区分真假，两者在对抗中共同提升。若生成器越来越强，最后产生的样本就会越来越逼真。这个思路后来被广泛用于图像生成、图像编辑、文本到图像合成等任务。

近年来又出现了**扩散模型**（diffusion model）等更稳定、更高质量的生成方法。它们大致通过“逐步加噪”和“逐步去噪”的逆过程来学习数据分布。虽然技术细节比 VAE 和 GAN 更复杂，但从大方向上看，它们都属于同一条主线：不仅要识别世界，还要学会模拟、重建甚至创造世界中的数据。

### 3.8 深度学习在更大图景中的位置

从更宏观的视角看，深度学习并不是把人工智能中其他方法全部取代，而是进一步嵌入到了更大的知识图谱中。可以把整条发展链条理解为：先有规则系统和知识驱动方法，随后有经典机器学习，再进一步出现表示学习，而深度学习则是在

表示学习基础上的一次大规模推进。

它最重要的贡献，并不只是“模型变深了”，而是让特征层次能够被自动学习，让不同数据类型都能找到更合适的结构表达方式，并把分类、检测、识别、翻译、生成等任务放入一个越来越统一的优化框架中。与此同时，深度学习也并非没有挑战。例如训练往往依赖大量数据与算力，模型可解释性常常不足，对抗样本、泛化稳健性、跨领域适应等问题依然很突出。

因此，学习这一章时，最需要的不是背下很多新名词，而是一个更成熟的判断：面对不同任务时，究竟是表示不够、结构不够、数据不够，还是目标函数与训练方式不够合适。只有把这些问题分开来看，深度学习才不会被误解成一种万能黑箱。

## 本章小练习

### 1. 判断题

Softmax 回归的输出是一个多维概率向量，各分量之和等于 1。(\_\_\_\_)

### 2. 填空题

在 HMM 中，计算观测序列概率的经典动态规划方法是 \_\_\_\_\_ 算法；寻找最可能状态序列的经典动态规划方法是 \_\_\_\_\_ 算法。

### 3. 简答题

请说明为什么深层网络通常比浅层网络更适合做表示学习。回答时应包含“层次化特征”“复用”“自动学习表示”这几个要点。

## 参考解答

1. 对。Softmax 会把各类别的打分通过指数归一化变成概率，因此输出向量中的每一维都表示一个类别的概率，且所有概率之和为 1。
2. 计算观测序列概率的经典方法是**前向算法**，寻找最可能状态序列的经典方法是 **Viterbi 算法**。前者关注“所有可能路径的总概率”，后者关注“概率最大的那一条路径”。
3. 深层网络更适合表示学习，首先是因为现实数据常具有明显的层次结构，低层简单模式可以逐步组合成高层复杂模式，因此使用**层次化特征**更自然。其次，低层特征在深层结构中可以被多个高层模式反复**复用**，从而更经济地表达复杂函数。最后，深度学习最大的优势之一在于它能够通过数据**自动学习表示**，不再严重依赖人工手工设计特征，这使它在图像、语音、文本等复杂任务中往往更有优势。

---

## 第四章 知识表示

### 本章概要

人工智能如果只会做数值计算，而不会把“世界中有哪些对象、对象之间有什么关系、哪些规则通常成立、哪些事实只是默认值”这些内容清楚地存起来，那么它就很难真正进行稳定推理。知识表示这一章研究的，正是如何把人类头脑中那些带有结构的知识，转化成机器可以存储、检索、继承和推演的形式。

这一章的内容看起来比较分散，实际上围绕的是同一个核心问题：**怎样把知识放进系统里，并让系统能围绕这些知识继续思考**。为此，我们会依次看到语义网络、框架、概念图、知识图谱等不同表示方式，并理解它们分别适合表达什么信

息、怎样支持推理，以及它们和语言理解、问答、数据库查询之间有什么联系。读完这一章后，读者应当能够理解：知识表示并不是“换一种写法记笔记”，而是在为推理、解释和应用系统搭建内部世界模型。

#### 4.1 为什么人工智能离不开知识表示

在程序设计中，我们常说“程序 = 算法 + 数据结构”。这个说法强调的是：算法负责规定处理步骤，数据结构负责存放对象与状态。而在专家系统中，一个更贴切的表达是：**专家系统 = 知识 + 推理**。这里的“知识”并不只是零散数据，而是关于对象、属性、类别、关系、规则和经验的组织化内容；“推理”则负责根据已有知识得到新结论。

这也说明了一个很重要的事实：知识表示、知识获取和知识使用，是人工智能中彼此紧密相连的三个环节。其中，知识表示处在最基础的位置。因为如果连知识该怎样存、怎样组织都没有想清楚，那么后面的知识录入、自动推理、解释系统行为等工作都会变得非常困难。换句话说，知识表示是后续一切知识型智能活动的出发点。

为什么知识表示这么重要？因为现实世界中的知识不是一堆互不相干的句子，而往往带有层次、联系和默认规律。比如我们说“金丝雀是鸟，鸟是动物，动物有皮肤”，那么系统就应当能自然推出“金丝雀有皮肤”；再比如我们说“鸟会飞”，但又知道“鸵鸟是鸟但不会飞”，这就说明知识中还会同时出现默认规律与例外情况。一个好的知识表示方式，必须能较自然地承载这些结构。

从心理学角度看，人类在处理不同句子时，反应时间并不完全一样。像“金丝雀是金丝雀”这样几乎同义反复的句子，判断起来最容易；“金丝雀是鸟”“金丝雀是动物”这样的句子也容易，但若层级越远、关系越间接，判断往往就会慢一点。这类现象启发研究者思考：人脑中的概念可能并不是平铺存放的，而是以某种层次网络组织起来的。人工智能中的许多知识表示方法，正是对这种“概念之间存在结构关系”的一种形式化模仿。

#### 4.2 语义网络：把概念放进关系图中

最直观的一种知识表示方法，就是**语义网络** (semantic network)。它的基本思想很简单：把概念、对象或事件表示成节点，把它们之间的关系表示成带标签的边。这样一来，知识就不再是一串孤立句子，而是变成了“谁和谁有什么关系”的网络结构。

例如，可以把 CANARY、BIRD、ANIMAL 看成三个概念节点，并用 is-a 边表示“是某类的一个实例”或“属于某上位类”的关系。这样，CANARY is-a BIRD、BIRD is-a ANIMAL 就形成了一个概念层次。与此同时，还可以用 has 表示属性关系，如 ANIMAL has SKIN；用 can 表示能力关系，如 BIRD can FLY；用 color 表示颜色属性，如 CANARY color YELLOW。

一旦知识以这种方式组织起来，推理就会变得自然很多。最常见的一种推理叫做**继承** (inheritance)。如果金丝雀是鸟，鸟又会飞，那么金丝雀通常也应当继承“会飞”这一属性；如果鸟是动物，而动物有皮肤，那么金丝雀也应当继承“有皮肤”。这种沿着层级从上往下传播属性的机制，就是语义网络最核心的优势之一。

不过，继承并不是机械拷贝，现实知识里常常有默认和例外。例如，一般来说鸟会飞，但鸵鸟虽然也是鸟，却不能飞。这意味着知识表示不能只靠“看到上位类就全部继承”，还必须允许特殊节点覆盖更一般的默认属性。也正因为如此，语义网络虽然直观，却并不是简单画几条线就够了，真正关键的是要想清楚哪些关系是可继承的，哪些关系会被例外打断。

在语义网络中，推理通常可以理解为沿图搜索关系链。比如若已知 TWEETY is-a CANARY, CANARY is-a BIRD, BIRD has property CAN-FLY, 那么系统就可以推出 CAN-FLY(TWEETY)。再比如若已知 SYLVESTER owns TWEETY, 且 TWEETY is-a CANARY, 那么系统可以进一步推出 SYLVESTER owns a CANARY, 甚至在更粗层次上推出 SYLVESTER owns a BIRD。这些结论不是显式存进去的，而是通过网络中的路径和继承关系得到的。

因此，语义网络最适合帮助初学者建立这样的直觉：知识并不是孤立事实的堆积，而是一个由“类别”“属性”“关系”“能力”“个体实例”共同构成的图结构。只要图结构搭好了，很多原本需要一条条硬编码写出的结论，就可以通过网络上的继承和搜索自然推出。

### 4.3 框架系统：把对象写成带槽位的结构

语义网络强调“点与边”，而**框架**（frame）更强调“一个对象本身应当包含哪些典型信息”。可以把框架理解成一种带槽位的结构化模板，它很像程序设计里的对象或记录。每个框架都围绕某个概念、对象或场景展开，并把相关属性、组成部分、默认值、使用方式等放进不同的槽位（slot）中。

例如，若我们建立一个 hotel room 框架，就可以把它的上位概念写成 room，把位置写成 hotel，把常见包含物写成 hotel chair、hotel phone、hotel bed。再进一步，hotel chair 本身也可以是一个框架，它是 chair 的一种，常见属性包括高度、腿数和用途；hotel phone 是 phone 的一种，可以带有“呼叫客房服务”“通过房费计费”等属性；hotel bed 还可以继续分解出 mattress 和 frame 等组成部分。

从这里可以看到，框架非常适合表达一种“对象内部结构比较丰富”的知识。与语义网络相比，它不只是告诉你“某物和某物有关系”，还更系统地说明“某一类对象通常具备哪些特征、包含哪些部件、缺省情况下是什么样子”。这种表示方式尤其适合常识知识、场景知识以及复杂对象描述。

框架中的槽位类型也很丰富。最基本的有标识信息，用来说明这是哪个框架；有和其他框架的关系信息，用来说明上位类、下位类、组成关系等；有描述性信息，如颜色、大小、用途；也可以有过程性信息，例如当访问某个槽位时触发一个计算过程；还可以有默认信息，用来表示“如果没有特别说明，就按通常情况处理”；甚至可以有创建新实例时需要填入的信息。

**默认值** 是框架系统中非常重要的思想。现实世界中的知识不可能每次都说得面面俱到，因此很多信息是“通常如此，但不保证总是如此”。例如，我们提到“房间”时，往往默认里面有可供休息的设施；提到“椅子”时，往往默认有腿、可坐；提到“床垫”时，往往默认有一定软硬程度。系统不必把每条实例都写全，而可以在上位框架里先设置默认值，具体实例再按需覆盖。

框架的推理方式，和语义网络有相通之处，但更接近“填槽、继承与覆盖”。例如，若已知 John isa human，再有一个 fire engine 框架包含属性 color: red、activity: high、volume: very high，那么当我们读到“John likes a fire engine”时，系统就可以把 fire engine 的某些突出属性迁移到 John 当前的心理或偏好刻画中。例如他喜欢高活动度、高音量、显眼颜色等。这里的要点不是具体例子本身，而是框架允许系统围绕一个对象，把相关槽位组织成一整块知识，并在此基础上作出较丰富的联想与补全。

因此，若语义网络更像“概念之间的关系图”，那么框架更像“围绕一个概念建立的说明书或模板”。二者并不冲突，反而经常互补：前者擅长表现整体关系网络，后者擅长表现单个概念的内部结构。

### 4.4 概念图：把语义关系画成更精细的结构

除了语义网络和框架，人工智能中还有一种很有代表性的知识表示方式，叫做**概念图**（conceptual graph）。概念图的目标，是把自然语言中的语义结构表示得更清楚，尤其适合表达“谁对谁做了什么、谁和谁具有什么关系”这类信息。

概念图中最重要的两类元素，是**概念节点**和**关系节点**。概念节点表示对象、事件、属性等实体；关系节点表示这些概念之间的联系。按关系所连接的参数个数不同，可以区分一元关系、二元关系、三元关系等。

例如，“鸟会飞”可以看成是一个一元关系：flies 只描述 bird 这一概念的某种性质。又如“狗是棕色的”，这里的 color 是一个二元关系，它把 dog 和 brown 连起来。再如“一个孩子有父母”，parents 就可以被看成连接 child、father、mother 的三元关系。通过这种方式，概念图可以非常自然地表达谓词和论元之间的结构。

在概念图中，还会区分**具体个体**和**泛化变量**。例如 dog:#007 这样的记法，可以理解为一个带唯一标识的具体个体；而 dog:\*X 则更像一个泛指变量，表示某个尚未具体命名的狗。这样做的好处是，系统既能表示“那只特定的棕色狗叫 Emma”，也能表示“某只狗抓自己的耳朵”这种带变量的普遍结构。

概念图还有一个很重要的能力，是能表达“命题本身成为另一个命题的对象”。例如，“Tom believes that Jane likes pizza”这句话里，likes(Jane, pizza) 本身就作为 believe 的宾语出现。普通的简单三元组很难自然表达这一层嵌套，而概念图通过引入命题性概念，可以把“一个命题被相信、被知道、被怀疑”这类更高层语义表示出来。

为了支持推理，概念图还定义了一些基本操作。**限制操作** (restriction) 可以把较一般的概念进一步收紧，例如把 animal:"emma" 限制成 dog:"emma"; **连接操作** (join) 可以把两个图中共享的部分合并起来，形成更完整的信息；**化简操作** (simplify) 则用于消去冗余结构，使结果更紧凑。初学时不一定要把每个图都画出来，但应当理解：这些操作的实质，就是围绕概念与关系做匹配、合并与精化，从而得到新的表示。

从语义表达能力来看，概念图比简单语义网络更细，尤其适合承载自然语言语义分析结果。它让我们看到：知识表示不仅仅是在存“事实清单”，也可以是在存一整套接近句子深层语义结构的内部图式。

#### 4.5 知识图谱与三元组

如果把知识表示往更现代、也更工程化的方向推进，就会遇到**知识图谱** (knowledge graph)。知识图谱的核心思想，可以看成对“实体 - 关系 - 实体/属性值”这一结构的大规模组织。最基本的表示单位通常是**三元组** (triple)：

(subject, predicate, object)

也就是“主语 - 谓词 - 宾语”。例如，可以把“Sean Penn 是 Person”“Sean Penn 的邮箱是某地址”“Sean Penn 的职业是 Actor”分别表示成多个三元组。这样一来，知识就不再埋在长句子里，而是被拆成大量可组合、可查询、可链接的小结构。

在 RDF 风格的表示中，常见写法会用 URI 来唯一标识资源，再用谓词描述关系。例如某个实体 URI 经过 rdfs:type 指向 Person，经过 fullName 指向具体姓名字符串，经过 mailBox 指向邮箱地址，经过 occupation 指向职业。这样做的关键目的，是让不同来源的数据都能尽量指向统一对象，而不是因为写法不同就彼此隔离。

知识图谱中的另一个重要特点，是它非常强调**对齐与融合**。现实世界中的数据往往来自多个数据库、多个机构、多个语言环境。一个地方叫 author，另一个地方可能叫 name；一个价格用欧元表示，另一个价格用美元表示；一个实体在两个数据库里对应两套不同 URL。若想把它们并起来，就必须处理“这些关系其实等价吗”“这些实体其实是同一个对象吗”之类的问题。

因此会出现像 owl:sameAs、owl:equivalentProperty 这样的机制。前者用于表达两个不同标识实际上指向同一个实体，后者用于表达两个不同谓词在语义上等价。通过这些链接，不同来源的知识就能被合并成更大的一张图，而不是各自孤立。

一旦知识以三元组形式组织起来，就可以使用专门的查询语言进行检索。最典型的是 SPARQL。比如一个查询可能希望找出某本书的作者、标题和主页信息，这种查询本质上就是在图上找某种关系模式的匹配结果。与关系数据库中的表查询相比，知识图谱更强调实体之间的链接结构以及语义层次。

因此，知识图谱可以看作知识表示在大规模互联网数据环境中的一种自然发展。它保留了“知识是结构化关系”的核心思想，但把这种思想推进到了跨来源整合、统一标识、可查询与可推理的层面。

#### 4.6 从句法分析到语义解释

知识表示并不是孤立存在的，它与自然语言理解有非常紧密的联系。因为很多知识最初都来自语言，而系统若想真正“读懂一句话”，就必须把句子的表面文字转换成较稳定的内部结构。

以一句简单的话“Tarzan kissed Jane.”为例，系统首先面对的是原始字符串。第一步通常是**句法分析** (parsing)，也就是识别句子由哪些成分构成，例如句子由名词短语和动词短语组成，动词短语内部又包含动词和宾语名词短语。这个阶段得到的往往是语法树，它说明的是结构，而不是最终语义。

接下来是**语义解释** (semantic interpretation)。在这一步里，系统需要把句法结构映射成更接近含义的表示。例如，它可能把 person: tarzan、person: jane、kiss、agent、object、instrument 等成分组织起来，从而表示“Tarzan 作为施事，Jane 作为受事，发生了亲吻这一事件”。这里的关键，是把单词和句法位置转化成语义角色。

但仅有语义结构还不够。真实理解往往还依赖更广义的**上下文知识或世界知识**。例如，若系统还知道 Tarzan 生活在丛林里，Tarzan 养了一只猎豹，或者“亲吻”通常与“喜爱”存在较强语义关联，那么它就能把原始语义结构进一步扩展成一个更丰富的内部表示。这说明语言理解的后半段，实际上又回到了知识表示本身：只有系统内部已有某种世界模型，才能对语言内容做更深层解释。

再往后，基于这些内部表示，系统才可能去做问答、数据库查询、翻译、文本生成、语音合成等更高级任务。换句话说，知识表示在这里扮演着中介角色：一边连接语言，一边连接推理和应用。

#### 4.7 不同知识表示方式的比较

学到这里，容易出现一个疑问：既然有这么多表示方法，它们到底是什么关系？其实可以把它们看成从不同角度切入同一问题的工具。

语义网络适合表现概念之间的关系与层级，尤其擅长做继承和联想式推理。框架适合围绕某个对象或场景组织较完整的属性与默认值，适合表达常识模板。概念图更强调语义结构，适合刻画事件、角色、命题嵌套等复杂关系。知识图谱则把这种结构化表达推进到大规模数据整合与查询环境中，更适合现代网络知识组织。它们彼此之间并不是非此即彼，而是经常互相借鉴、互相转化。

除此之外，知识表示领域还有许多其他方法。例如命题逻辑、一阶逻辑、描述逻辑、模态逻辑强调形式推理能力；产生式规则适合表达“如果……那么……”的知识；贝叶斯网络适合表达不确定性依赖；对象 - 属性 - 值三元组适合做轻量级结构化存储；脚本、决策表、Petri 网等也都在不同问题中发挥作用。重要的不是一次性记全这些名词，而是理解：不同任务会偏好不同的表示方式，因为它们对“结构”“不确定性”“默认值”“可查询性”“可解释性”的需求并不相同。

#### 本章小练习

##### 1. 判断题

知识表示只是把知识换一种方式存下来，对后续推理和应用没有本质影响。(\_\_\_\_)

##### 2. 填空题

在知识图谱中，最基本的表示单位通常是 \_\_\_\_\_，其基本形式可以写成 \_\_\_\_\_。

### 3. 简答题

请说明语义网络与框架系统各自更擅长表示什么类型的知识，并说明它们为什么都能够支持一定程度的推理。

### 参考解答

1. 错。知识表示并不只是存储格式的变化，而是直接影响系统能否进行继承、匹配、补全、解释和推理。表示方式不同，系统能方便做出的推理类型也会不同，因此它对后续应用有本质影响。
2. 最基本的表示单位通常是**三元组**，其基本形式可以写成 **(subject, predicate, object)**，也就是主语、谓词、宾语三部分。
3. 语义网络更擅长表示概念之间的层次关系、属性关系和实例关系，因为它把知识组织成“节点 + 关系边”的图结构，因此很适合做继承式推理，例如从上位类向下位类传递属性。框架系统更擅长表示某个对象、场景或概念内部较完整的属性结构，因为它把相关信息集中放入槽位中，并允许默认值和覆盖机制，因此适合做属性补全和基于默认知识的推理。二者都能支持推理，但推理方式有所不同：语义网络偏向沿关系图搜索和继承，框架偏向围绕槽位做填充、继承和覆盖。

---

## 第五章 不确定性推理

### 本章概要

前面几章讨论的许多方法，无论是知识表示、逻辑推理还是神经网络学习，都默认系统至少能拿到某种相对清楚的输入与规则。但真实世界远不是这样。信息可能不完整，测量可能不准确，证据之间可能互相冲突，过去经验未必还能适用于未来，而系统往往又必须在这种条件下继续行动。于是，人工智能不能只研究“在完全确定的条件下怎样推理”，还必须研究：当信息带有不确定性时，系统应当怎样表达信念、更新判断、比较行动方案并承受风险。

这一章的内容虽然很多，但可以沿着一条非常清楚的主线来理解。首先，我们要弄清楚不确定性究竟来自哪里，以及它会导致哪些典型错误。接着，我们会看到：单靠传统演绎逻辑并不足以应对现实世界，因此需要引入归纳推理、非单调推理与概率推理。之后，我们会学习贝叶斯公式如何更新信念，贝叶斯决策如何把“信念”进一步转化成“行动”。再往后，还会看到早期专家系统如何用 certainty factor 处理不确定证据，时间相关问题如何引出随机过程与马尔可夫链，自然语言中的模糊概念又如何通过模糊集合与模糊规则来处理。读完这一章后，读者应当能够理解：面对不确定世界，人工智能并不是只能说“我不知道”，而是要学会在不完全确定的条件下，尽可能做出合理判断。

### 5.1 不确定性从哪里来

**不确定性**可以简单理解为：系统手头没有足够、精确、可靠的信息，因而无法完全确定地做出判断。之所以要认真研究它，是因为现实决策往往恰恰发生在这种条件下。若一个系统只能在所有信息都完整无误时工作，那么它几乎无法在真实环境中长期生存。事实上，生物之所以能在复杂环境中存活，很大程度上就是因为它们能够在不确定条件下行动，而不是等到一切都确定无疑以后才开始做决定。

不确定性常常会以各种“错误”形式表现出来。最先出现的一类，是**信息本身的问题**。有的信息是**歧义的**，例如一句指令可能有不止一种解释；有的信息是**不完整的**，例如只告诉系统去操作某个阀门，却没有说明向哪个方向操作；还有的信息



甚至是**错误的**，即给出的要求本身就 and 真实应执行的动作相反。对于系统来说，这三种问题虽然都表现为“做不对事”，但来源并不一样：歧义来自解释不唯一，不完整来自信息缺失，错误则来自内容本身不真。

另一类常见不确定性来自**测量**。这里又可以细分出几个不同概念。第一是**精密度** (precision) 问题，即读数太粗，无法反映真实值的细节。例如正确值是 5.4，但系统只能把它读成 5。第二是**准确度** (accuracy) 问题，即读数看起来很精细，但偏离真实值很远。例如系统读出 5.4，而真实值其实是 9.2。第三是**随机误差** (random error)，即误差没有固定方向，而是随测量波动不断变化。第四是**系统误差** (systematic error)，即设备、方法、校准方式本身存在偏差，导致误差长期朝同一方向偏移。

还有一类错误不是来自输入，而是来自**判断机制本身**。例如系统可能产生**假阳性** (false positive, Type I error)，也就是本来没有故障，却判断成有故障；也可能产生**假阴性** (false negative, Type II error)，也就是明明有故障，却判断成没有故障。这两种错误在医学检测、设备监控和异常识别中都极其重要，因为不同场景对它们的容忍程度往往完全不同。

最后，还有一种特别值得警惕的错误来源，即**推理错误**。有时系统会做出不合理的归纳，例如“过去从没出过问题，所以以后也不会出问题”；也可能做出不合理的演绎，例如“若阀门状况正常，则输出正常；现在输出正常，所以阀门一定正常”。后者之所以错，是因为输出正常并不一定只有一个原因，例如阀门也可能卡在打开位置。也就是说，从结果反推原因时，常常会出现“一个结果对应多个可能原因”的情况。

因此，所谓不确定性，并不是单一的一种东西，而是多种不同问题的统称。它可能来自输入的歧义与缺失，来自测量的不稳定，来自统计波动，来自模型的简化，也来自推理结构本身的不充分。也正因为来源多样，处理不确定性的方法也不可能只有一种。

## 5.2 演绎、归纳与非单调推理

在讨论不确定性时，首先必须区分两种很基础但经常被混淆的推理方式：**演绎推理** (deduction) 和**归纳推理** (induction)。

演绎推理是我们在逻辑中最熟悉的形式。若前提为真、推理结构有效，那么结论就一定为真。最经典的例子就是：

```
All men are mortal.
Socrates is man.
Therefore Socrates is mortal.
```

这类推理之所以强大，就在于它具有有一种“真值保真”的性质：只要前提没有问题，形式又正确，那么结论不会出错。从这个角度看，演绎推理是“总是正确的”，至少在逻辑结构层面如此。

归纳推理则完全不同。比如：

```
My disk has never crashed.
Therefore my disk will never crash.
```

这个结论显然不能保证一定为真。过去的经验只能给出某种**信心程度**，而不能像演绎那样提供绝对保证。也正因为如此，归纳推理非常接近机器学习和经验学习的本质：我们不是从公理推到必然结论，而是在有限样本、有限历史记录基础上，对未来做一个有风险的判断。

这里还要特别注意一种常见逻辑谬误。若我们知道  $p \rightarrow q$ ，又观察到  $q$ ，并不能反推出  $p$ 。例如：

```
If the valve is in good condition, then the output is normal.
The output is normal.
```

Therefore the valve is in good condition.

这个推理是无效的。因为“输出正常”只是结果，它可能由多种原因造成；阀门状态良好只是其中一种解释，而不是唯一解释。这类错误在诊断问题中尤其常见，因为从症状反推病因几乎从来都不是单值映射。

一旦进入现实环境，人们还会进一步发现：即使某个结论原先看起来合理，后来也可能被新证据推翻。这就引出了**非单调推理** (nonmonotonic reasoning)。所谓“非单调”，指的是推理系统的结论集合不会永远只增不减；新信息出现后，旧结论可能被撤回。

一个很典型的例子是：一开始我们看到某物看起来是红色的，于是支持“它是红色的”；但后来又知道它正处在红灯照射下，而红灯恰好会让本来不是红色的物体也显得发红。这时，新证据就会**击败** (defeat) 原来的支持关系。换句话说，不是说旧推理过程本身出错了，而是说它原本只是“在当前知识条件下合理”，一旦知识条件变化，结论也应跟着变化。

非单调推理会引出一系列经典困难。第一类问题是像 **Buridan's ass** 这样的“对称困境”：若两个选择在当前知识下完全对称，系统可能迟迟无法行动。第二类问题是**医疗诊断**这类多证据竞争场景：新症状到来之后，旧诊断会被削弱，新的诊断又会暂时变强。第三类问题是**彩票悖论**：每张彩票单独看都几乎不会中奖，所以似乎每张都不该相信会中；但又知道一定有一张会中奖，于是整体信念会出现冲突。第四类问题是**序言悖论**：作者对书中每条陈述都各自有信心，但也同时承认一本足够长的书几乎必然含有至少一个错误。第五类则是**击败循环**：A 削弱 B，B 又削弱 C，C 反过来又削弱 A，从而形成循环依赖。

这些现象共同说明：一旦知识不再完全确定，传统逻辑中“前提固定不变、结论单调累积”的理想状态就会被打破。于是，人工智能系统不仅要能推理，还要能在新证据出现时合理地修改原先的信念状态。

### 5.3 经典概率、显著性检验与它们的局限

当人们希望把“不确定”变成可以计算的量时，最自然的工具就是**概率**。概率可以从不同角度理解。**实验概率** (experimental probability) 强调反复实验后频率的统计意义，例如抛硬币长期接近 1/2；**主观概率** (subjective probability) 则强调某个主体对事件发生的相信程度。人工智能中两种观点都会出现，因为有些问题依赖长期频率，有些问题则更像基于经验和先验知识作判断。

概率论中有一些基本概念必须清楚。若两个事件一起发生，就涉及**复合概率**；若我们关心“在某条件已经发生的前提下，另一个事件发生的概率”，就涉及**条件概率**。正是条件概率，使概率论能够描述“在观察到新证据之后，信念应当怎样变化”。

在统计推断中，一个常见工具是**假设检验** (hypothesis test)。它的思路是：先提出零假设，再根据观测数据计算“若零假设为真，当前结果有多罕见”，然后决定是否拒绝零假设。这个工具在很多科学研究和工程检测中都很重要。

然而，假设检验也有明显局限。一个很有代表性的例子是三个“十次全对”的实验：一位女士声称自己能区分奶茶中是先倒茶还是先倒奶；一位音乐专家声称能区分海顿与莫扎特的谱页；一位醉酒朋友声称自己能预测公平硬币的正反面。若只做古典显著性检验，并令未知参数  $\theta$  表示“每次回答正确的概率”，零假设取  $H_0: \theta = 0.5$ ，那么三种情况中“十次全对”的单侧显著性水平都是  $2^{-10}$ 。也就是说，若只看这个计算结果，三者都足以拒绝零假设。

但人的直觉会立即察觉：这三个例子其实并不等价。音乐专家的说法显然比醉酒朋友预测硬币更可信。那么这暴露出了什么问题呢？由于检验的逻辑是“反证法”，我们必须都先假装音乐家根本没有真本事（猜对概率  $\theta = 0.5$ ），然后考察是否要“拒绝”他没本事的说法，承认他有真本事。让醉汉去猜硬币，也是一样的预先“假装”逻辑。如果他恰好连续猜对了 10 次，在传统检验看来，由于其没本事的概率也是  $2^{-10}$ ，我们也必须承认醉汉有“超能力”。这就是局限所在：传

统检验不允许我们把“音乐家懂音乐”和“醉汉在胡扯”这两个先验常识算进公式里。它把二者完全等同对待，这违背了人类的直觉。

也就是说，仅靠显著性水平，并没有把我们头脑中那些关于背景知识、先验可信度和机制合理性的判断充分表达出来。这说明，纯粹的显著性检验虽然有用，但它并不能独立解决所有不确定性推理问题。

除此之外，假设检验本身还面临两个常见困难。

第一，所谓“点零假设”往往几乎不可能在现实中精确成立。例如硬币的绝对公平  $\theta = 0.5$  在现实中几乎不存在，可能因为磨损导致正面概率是 0.50001。此时只要样本足够大，哪怕偏离极其微小，也可能被检验出来。比方说偏离了百万分之一，软件也会报出  $p < 0.05$ ，认为需要拒绝零假设（这枚硬币不公平）。

第二，拟合优度检验也有类似问题。在概率论课程中，我们就学过：拟合优度就是在检验实际数据是否完美符合某个数学模型，如正态分布。然而，在模型在现实中几乎不可能完全正确，因此当样本越来越大时，系统几乎总有机会把模型拒掉。

于是，人们逐渐意识到：仅仅问“是否拒绝零假设”还不够，更重要的是建立一种能把信念、证据、行动和偏好统一起来的理论。

#### 5.4 贝叶斯定理：用证据更新信念

贝叶斯方法正是朝这个方向迈出的一大步。它最核心的思想可以概括成一句话：**在看到新证据后，应该用概率系统地更新原有信念**。若有一组假设  $H_i$  和观察到的证据  $E$ ，则贝叶斯公式写为

$$P(H_i | E) = \frac{P(E | H_i)P(H_i)}{P(E)}$$

这个公式中，每一项都有清楚含义。 $P(H_i)$  称为**先验概率** (prior)，表示在观察证据之前，对假设  $H_i$  的原始相信程度。 $P(E | H_i)$  称为**似然** (likelihood)，表示若假设  $H_i$  为真，看到证据  $E$  的概率有多大。 $P(H_i | E)$  称为**后验概率** (posterior)，表示结合证据之后，对该假设的新相信程度。分母  $P(E)$  起归一化作用，保证所有后验概率加起来仍然构成合法分布。

从直觉上说，贝叶斯公式其实非常自然。若一个假设本来就比较可信，而且它还能很好地解释当前证据，那么它的后验概率就应当升高；反之，若一个假设本来可信度不高，或者它根本解释不了当前证据，那么它的后验概率就会下降。贝叶斯公式只是把这一直觉严格量化了而已。

一个非常典型的例子，是资源勘探问题。假设某地有油的先验概率是 0.6，没有油的概率是 0.4。又假设地震测试为条件的条件概率满足：

$$P(+ | O) = 0.8, \quad P(- | O) = 0.2$$

$$P(+ | O') = 0.1, \quad P(- | O') = 0.9$$

其中， $O$  表示“有油”， $O'$  表示“无油”。先把条件概率与先验相乘，就能得到联合概率：

$$P(- \cap O') = 0.36, \quad P(+ \cap O') = 0.04$$

$$P(- \cap O) = 0.12, \quad P(+ \cap O) = 0.48$$

于是总边缘概率为

$$P(-) = 0.48, \quad P(+) = 0.52$$

接下来再用贝叶斯公式求后验：

$$P(O | -) = \frac{0.12}{0.48} = \frac{1}{4}, \quad P(O' | -) = \frac{3}{4}$$

$$P(O | +) = \frac{0.48}{0.52} = \frac{12}{13}, \quad P(O' | +) = \frac{1}{13}$$

这个结果非常有启发性：测试前我们只知道“有油”的概率是 0.6；若测试为正，这个概率立刻提高到 12/13；若测试为负，则降到 1/4。这说明贝叶斯方法并不是凭空给出新结论，而是在已有先验和新证据之间建立一条明确的更新通道。

### 5.5 贝叶斯决策：不仅问“信什么”，还问“做什么”

仅有后验概率还不够。真正的决策问题常常不仅关心“哪个假设更可能为真”，还关心“在不同情况下采取什么行动最划算”。这就进入了**贝叶斯决策** (Bayesian decision making)。

继续上面的资源勘探为例。假设系统可以先做一次地震测试，也可以不做测试；测试之后还可以选择退出或钻探。若成功采到油，收益是 \$1,250,000；钻探成本是 -\$200,000；地震调查成本是 -\$50,000。此时，一个合理决策流程就不再只是比较  $P(O | +)$  和  $P(O | -)$ ，而是要把这些后验概率与行动收益结合起来，计算**期望收益**。

例如，若测试结果为负，那么“钻探”这一行动的收益就需要在“有油”和“无油”两种结果下加权平均。由于此时  $P(O | -) = 1/4$ ， $P(O' | -) = 3/4$ ，所以它的期望收益大约为 \$62,500。若测试为正，则由于  $P(O | +) = 12/13$  非常高，钻探的期望收益会升到约 \$846,153。再把“测试本身的代价”考虑进去，整条“先测试，再根据结果决定是否钻探”的策略，总体期望收益约为 \$470,000。

这个例子说明了一个极其重要的思想：**后验概率本身不是决策终点，行动价值才是决策终点**。有时一个假设虽然更可能为真，但采取对应行动的收益不一定更高；反过来，有些行动虽然成功概率不算最高，但一旦成功回报巨大，也可能更值得执行。于是，贝叶斯方法便把“相信什么”和“做什么”真正连接在了一起。

另一个很有代表性的例子，是农民根据降水情况决定种植哪种作物。假设有两种行动： $a_1$  表示种抗旱作物， $a_2$  表示种高产作物；自然状态  $\theta$  表示来年的降水量。若损失函数写为

$$l(\theta, a_1) = 200 - 2\theta$$

$$l(\theta, a_2) = 3000 - 10\theta$$

那么农民的任务就是：根据天气预报对降水量作估计，再选择损失更小的行动。若进一步给出天气预报的误差模型、降水量的先验分布以及相应的决策风险函数  $R(\theta, \delta)$ ，就可以继续求出某一决策规则的 **Bayes risk**。在这个例子里，若农民采用“预报低于 400mm 种抗旱作物，否则种高产作物”的阈值策略，那么在给定的先验和误差模型下，这个策略最终被证明是合理的。这说明贝叶斯决策理论并不是只会处理纸面上的小概率题，它也能自然地处理“信息有误差、行动有损失、主体有偏好”的现实决策问题。

此时，我们也可以回顾前面的“女士、音乐家和醉汉”的问题了。现代机器学习引入贝叶斯推理和决策后，就可以带来以下改观：1. 引入“先验概率”：机器学习不仅仅看当下的数据，还要看历史经验。比如，AI 会给音乐家一个很高的先验信任度（例如 0.8），给醉汉一个极低的先验信任度（例如 0.001）。结合数据后，算出的“后验概率”就会非常符合直觉：音乐家是真的，醉汉是运气好。2. 从“非黑即白”到“概率分布”：传统检验只回答“拒绝”或“不拒绝”。但机器学习需要知道具体的确定性程度（Softmax 概率输出）。3. 强化学习中的“信念与行动”：在本节的决策讨论中，我们就知道：还必须根据当前不确定的证据，结合它对环境的信念，做出收益最大的行动。

## 5.6 贝叶斯方法的困难与贝叶斯网络

贝叶斯方法很优雅，但它并不意味着现实中所有问题都会变得轻松。一个很大的困难，是当证据越来越多时，所需要的条件概率会迅速膨胀。

例如在疾病诊断中，若  $D_i$  表示第  $i$  种疾病， $E$  表示某组症状，则有

$$P(D_i | E) = \frac{P(E | D_i)P(D_i)}{\sum_j P(E | D_j)P(D_j)}$$

这一步还不算太难。但如果后续证据不断加入，就必须继续计算类似

$$P(D_i | E_1, E_2, \dots)$$

的量。这时会出现大量条件概率项，例如  $P(E_2 | D_i \cap E_1)$ 。证据越多，相关概率项就越多，参数获取、存储与维护都会急剧变难。换句话说，贝叶斯公式本身并没有错，难的是现实问题中概率结构过于复杂。

为缓解这一困难，人们提出了**贝叶斯网络** (Bayesian network)。贝叶斯网络用有向图结构来表达随机变量之间的依赖关系，从而把一个庞大的联合分布拆解成许多局部条件分布。若两个变量在图结构上被判定为条件独立，就不必再强行估计它们之间的冗余联合概率。

例如在医疗诊断图中，年龄、天气、疾病、腺体状态、声音质量、腹痛、发热、毒性外观等变量可能彼此有方向性的影响关系。通过把这些变量组织成图，系统就能更清楚地表达“哪些因素直接影响哪些症状”，以及“哪些变量只需要在局部条件下考虑”。因此，贝叶斯网络可以看作一种把概率推理与知识结构结合起来的方法：它既保留了贝叶斯概率更新的严格性，又在结构上大幅减少了不必要的复杂度。

## 5.7 MYCIN 与确定性因子

在早期医学专家系统中，并不是所有人都愿意完全采用严格概率模型。一个著名例子就是 **MYCIN**。它使用规则系统来做诊断，同时引入了 **certainty factor**（确定性因子，简称 CF）来表达“不完全确定的支持程度”。

一个典型规则是：

IF

the stain of the organism is gram positive,  
and the morphology is coccus,  
and the growth conformation is chains

THEN

there is suggestive evidence (0.7)  
that the identity of the organism is streptococcus.

翻译成中文就是：

// IF 阳性 且 球菌 且 链状

// THEN 有 0.7 的暗示性证据表明是链球菌。

表面看起来，这个 0.7 很像条件概率，于是似乎可以写成

$$P(H \mid E_1 \cap E_2 \cap E_3) = 0.7$$

但问题恰恰在这里。若这样写，就会让人不由自主地联想到

$$P(\neg H \mid E_1 \cap E_2 \cap E_3) = 0.3$$

而很多医学专家会对这种解释感到不舒服。因为“对链球菌有 0.7 的支持”并不等于“对非链球菌只有 0.3 的支持”。剩下那部分并不一定是明确反对，它也可能只是“还不知道”“还有别的可能”“证据还不够”。因此，MYCIN 采用了不同于经典概率的表达方式，把“支持”和“反对”分开。

具体地说，它定义：

$$CF(H, E) = MB(H, E) - MD(H, E)$$

其中， $MB(H, E)$  表示证据  $E$  对假设  $H$  的**增加相信程度** (measure of increased belief)， $MD(H, E)$  表示证据  $E$  对假设  $H$  的**增加不信程度** (measure of increased disbelief)。于是，CF 的取值范围变为

$$0 \leq MB \leq 1, \quad 0 \leq MD \leq 1, \quad -1 \leq CF \leq 1$$

这样一来， $CF = 1$  表示证据让假设几乎确定为真， $CF = -1$  表示证据让假设几乎确定为假， $CF = 0$  则表示当前证据并没有改变信念状态。

在前面的链球菌的例子中, 在这个体系下, 我们就可以说: 证据让链球菌的可信度增加了 0.7。然而仍然有  $MD(H, E) = 0$ , 也就是说这组证据没有提供任何反对它是链球菌的理由。最终代入公式计算确定性因子, 就可以得到

$$CF(H, E) = MB(H, E) - MD(H, E) = 0.7 - 0 = 0.7$$

这样的解释为什么在医学中更好? 因为它完美容纳了“未知”。同时, 支持和反对也可以“分别积累”。假设医生接着又做了一个新的皮肤化验 (证据  $E_4$ ), 这个化验结果通常在非链球菌感染中才会出现。专家认为它对链球菌有 0.2 的反对力度。此时新证据结合之前的证据后, 最终的  $CF$  就会从 0.7 降到 0.5。

从直觉上看, 这种拆分是有意义的。因为现实中的许多证据确实不是“支持某假设”就一定等价于“反对其否定”。很多时候, 支持和反对是两种可以分别积累、分别削弱的力量。

### 5.8 证据组合与 CF 的局限

既然专家系统中往往会出现复杂前件, 就必须规定多个证据如何组合。MYCIN 对此采用了一套相当直接的规则:

$$CF(E_1 \wedge E_2) = \min(CF(E_1), CF(E_2))$$

$$CF(E_1 \vee E_2) = \max(CF(E_1), CF(E_2))$$

$$CF(\neg E) = -CF(E)$$

例如, 若

$$E = (E_1 \wedge E_2 \wedge E_3) \vee (E_4 \wedge \neg E_5)$$

那么它的确定性因子可以按逻辑结构逐层合成。若具体给出

$$CF(E_1, e) = 0.5, \quad CF(E_2, e) = 0.6, \quad CF(E_3, e) = 0.3, \quad CF(E_4, e) = 0.8, \quad CF(E_5, e) = 0.9$$

且规则本身的结论强度是

$$CF(H, E) = 0.7$$

那么前件整体的确定性因子就是

$$CF(E, e) = \max(\min(0.5, 0.6, 0.3), \min(0.8, 1 - 0.9)) = 0.3$$

注意，这里的  $e$  是指当前面对的具体病人的化验报告综合。 $CF(E_1, e) = 0.5$  的意思就是：“在当前病人已有的化验报告  $e$  下，医生对 ‘化验 1 ( $E_1$ ) 为真’ 的把握是 0.5”。正因如此，我们在进行最终计算时，不仅要考虑规则本身的强度（已知化验报告为真，能够推出结论的信度），还要把本身 “化验报告为真” 的概率一起算入。

于是，结论的 CF 便计算为

$$CF(H, e) = CF(E, e) \cdot CF(H, E) = 0.3 \times 0.7 = 0.21$$

这说明：前件支持越弱，最终结论被推出的强度也会相应减弱。

当两条规则都支持同一个假设时，还需要进一步做**组合函数**。设两个规则给出的确定性因子分别为  $CF_1$  和  $CF_2$ ，则经典组合形式可写为

$$CF_{COMB}(CF_1, CF_2) = \begin{cases} CF_1 + CF_2(1 - CF_1), & \text{两者都大于 0} \\ \frac{CF_1 + CF_2}{1 - \min(|CF_1|, |CF_2|)}, & \text{一正一负} \\ CF_1 + CF_2(1 + CF_1), & \text{两者都小于 0} \end{cases}$$

这个组合函数满足交换性，也就是先合并哪条规则都一样。但它虽然实用，却并不等于严格概率推理。

事实上，certainty factor 最大的问题，恰恰在于它是一种工程启发式而不是严密概率模型。一个明显困难是：CF 对假设的排序可能与后验概率排序相反。

MYCIN 的设计者为了让 MB 有概率影子，曾给出过这样一个定义：若  $P(H|E) \geq P(H)$ ，即证据让可信度增加了，那么：

$$MB(H, E) = \frac{P(H|E) - P(H)}{1 - P(H)}$$

这个公式的直观含义是：证据帮你消除了剩下那部分不确定性  $1 - P(H)$  中的百分之几。

那么，会出现什么问题呢？比如：

$$P(H_1) = 0.8, \quad P(H_1 | E) = 0.9$$

$$P(H_2) = 0.2, \quad P(H_2 | E) = 0.8$$

在这里， $H_1$  的后验概率显然更高，但若按 CF 规则算出来，却可能有



$$CF(H_1, E) = 0.5, \quad CF(H_2, E) = 0.75$$

于是,  $H_2$  的 CF 反而更高。这意味着如果系统把 CF 直接当作“最可信诊断”的排序指标, 就会出现与概率判断不一致的情况。

第二个困难, 是推理链上的简单乘法并不符合一般概率论。例如通常并没有

$$P(H | e) = P(H | i)P(i | e)$$

这样的等式成立, 但 CF 系统在推理链上却往往采用

$$CF(H, e) = CF(i, e) \cdot CF(H, i)$$

这种近似形式。因此, certainty factor 的地位应当理解为: 它是早期专家系统中一种便于与专家交流、便于工程实现的折中办法, 而不是对概率推理的严格替代。

## 5.9 时间推理、随机过程与马尔可夫链

有些不确定性并不来自“证据真假不清”, 而来自“状态会随时间变化”。这就引出**时间推理** (temporal reasoning)。在许多问题中, 系统不仅要判断当前状态, 还要关心状态未来如何演化。例如病情如何发展、交通流如何变化、市场份额会怎样转移、用户下一步会做什么等。

描述这类问题时, 一个很重要的工具是**随机过程** (stochastic process)。若系统状态在离散时间上一步步变化, 并且下一步只依赖当前状态, 而不依赖更久远历史, 就得到了**马尔可夫链** (Markov chain)。

设系统只有两个状态  $S_1$  和  $S_2$ , 则转移矩阵的一般形式为

$$T = \begin{bmatrix} P_{11} & P_{12} \\ P_{21} & P_{22} \end{bmatrix}$$

其中,  $P_{mn}$  表示从当前状态  $m$  转移到下一时刻状态  $n$  的概率。矩阵每一行的和应为 1, 因为系统从某个状态出发, 总会转移到某个可能的下一状态。

以消费者电脑品牌选择为例, 设  $C_1$  表示购买 Lenovo,  $C_2$  表示购买 Apple, 转移矩阵为

$$T = \begin{bmatrix} 0.1 & 0.9 \\ 0.6 & 0.4 \end{bmatrix}$$

若初始状态分布为

$$S_1 = [0.8, 0.2]$$

表示一开始 80% 的人买 Lenovo, 20% 的人买 Apple, 那么下一步分布就是

$$S_2 = S_1 T = [0.8, 0.2] \begin{bmatrix} 0.1 & 0.9 \\ 0.6 & 0.4 \end{bmatrix} = [0.2, 0.8]$$

继续迭代, 就会得到

$$S_3 = [0.5, 0.5], \quad S_4 = [0.35, 0.65], \quad S_5 = [0.425, 0.575], \dots$$

最终状态逐步收敛到一个稳定分布

$$S = [0.4, 0.6]$$

这个稳定分布满足

$$[P_1, P_2]T = [P_1, P_2], \quad P_1 + P_2 = 1$$

这说明: 如果转移矩阵满足适当条件, 那么系统经过足够长时间后, 会稳定在某种长期比例上, 而与初始状态的影响逐渐减弱。

马尔可夫链之所以重要, 不只是因为它会做矩阵乘法, 而是因为它提供了一种极其清楚的时间建模思想: 状态数量有限, 任一时刻只处于一个状态, 系统按时间步前进, 而下一步只依赖当前状态。这种“无后效性”假设虽然是简化, 但在很多问题中非常有效。

PageRank 就是一个很有代表性的例子。网页之间的超链接关系可以看作状态转移图, 用户在页面间随机跳转的过程可以看作马尔可夫过程。通过不断迭代状态分布, 就能得到网页的重要性评分。对没有外链的悬挂节点, 还需要对转移矩阵做适当修正, 以保证随机游走过程可以持续进行。

### 5.10 模糊集合: 处理“部分为真”的概念

前面讨论的概率、贝叶斯和马尔可夫链, 主要处理的是随机性、信念和时间演化。但现实世界里还有另一类问题: 很多概念本身就不是清清楚楚的边界, 而是带有渐变性。比如“高”“热”“危险”“有点亮”这些词, 并没有一个绝对界限。若继续坚持传统二值逻辑中“属于”或“不属于”的严格分法, 就会显得非常生硬。

这正是**模糊集合** (fuzzy set) 要解决的问题。模糊集合并不是要否认逻辑, 而是要描述这样一种现实: 一个对象对某个概念的符合程度, 往往可以介于 0 和 1 之间, 而不一定只能取 0 或 1。

模糊集合最基本的工具是**隶属函数** (membership function)。若集合  $A$  的隶属函数记为  $\mu_A(x)$ , 则有

$$\mu_A(x) : x \rightarrow [0, 1]$$

$$0 \leq \mu_A(x) \leq 1$$

这里， $\mu_A(x)$  的值表示对象  $x$  属于模糊集合  $A$  的程度。例如在模糊集合 TALL 中，身高越高，隶属度通常越接近 1；身高越矮，则越接近 0；而在中间地带，隶属度会逐步变化，而不是突然跳变。

这正对应了现实中的很多语言变量。例如 height 可以有 dwarf、short、average、tall、giant 等典型值；light 可以有 dim、faint、normal、bright、intense 等取值。模糊集合的优势在于：它允许系统直接操作这些自然语言概念，而不是强行把它们切割成僵硬的界限。

此外，我们一定要注意的是：隶属度并不是概率。这是一个常见误解。在模糊逻辑中，千万不能乱用“概率”这个词。概率代表着随机性，里面有“赌”的成分。如果说 180cm 有 0.8 的概率是高个子，意思是如果把很多人叫过来，有 80% 的真的是高个子，有 20% 的是矮个子。但实际上，这个人就在这里，不属于任何随机事件。

而隶属度则代表“模糊性”。模糊集合说 180cm 属于 tall 的程度是 0.8，意思是，180cm 这个人的身高，有 80% 的属性符合人类对‘高’的定义，有 20% 不符合。这才是正确的理解。

### 5.11 模糊规则、语言修饰词与模糊合成

有了模糊集合之后，就可以进一步建立**模糊规则**。例如：

```
IF the TV is too dim THEN turn up the brightness
IF it is too hot THEN add some cold
IF the pressure is too high THEN open the relief valve
```

这些规则与传统规则系统的区别在于：前件不要求绝对为真，而是以某种程度成立。于是，后件也会以相应强度被激活。这样一来，系统就能处理“有点暗”“非常热”“略微高”等更接近人类表达方式的情况。

那么，读者可能自然会产生一个疑问：前面的条件是模糊的，那么它是如何“往后推送”的呢？以空调为例，假设我们有一条模糊规则：

```
IF it is too hot THEN add some cold
```

现在传感器测到房间温度是 26°C。假设根据模糊集合的定义，这个温度属于“有点热”的隶属度是 0.6。此时，系统不是以 60% 的概率去开大风速，而是必然 100% 执行这条规则，但把输出的强度砍到只剩 0.6。具体在数学和工程上，有两种主流的处理方式。一是 Mamdani 模糊推理，其中后件的“把风速调大”本身也是一个模糊集合，有一个分布图。比如风速从 0 到 100 档，那么规则激活后，系统就会算出一个输出值，比如将风速开到 75 档。还有一种方式是 Sugeno 模糊推理，常见于现代工业控制，如洗衣机、微波炉。它的后件直接是一个明确的数学公式，比如一个风速和温度的函数关系。

模糊系统中还有一类非常有趣的工具，叫做**语言修饰词** (linguistic hedges)。像 very、more or less、not 这类词，其实可以理解为对隶属函数做某种数学变换。常见定义包括：

$$\text{Very } F = F^2$$

$$\text{More or Less } F = F^{0.5}$$

$$\text{Plus } F = F^{1.25}$$

$$\text{Not } F = 1 - F$$

$$\text{Not Very } F = 1 - F^2$$

这些公式很有直观意义。若一个对象原本对 TALL 的隶属度是 0.5，那么平方以后就变成 0.25，说明 VERY TALL 比 TALL 更严格；而开平方以后会变大，说明 MORE OR LESS TALL 更宽松。也就是说，语言修饰词不只是语文现象，它们完全可以被形式化为对隶属度曲线的变形。

模糊集合之间还可以定义**模糊关系**。例如，可以用一个矩阵来表达“体重近似相等”的程度。若  $R_1(x)$  表示 HEAVY,  $R_2(x, y)$  表示 APPROXIMATELY EQUAL，则可以通过合成得到新的模糊集合 MORE OR LESS HEAVY:

$$R_3(y) = R_1(x) \circ R_2(x, y)$$

在这里， $x$  表示已知体重的那些人；而  $y$  则表示陌生人，我们不知道他的具体体重  $R_1(y)$ ，只知道他和其他人在肥胖上的“相似程度”  $R_2(x, y)$ 。

其中常见的合成规则是 **max-min 合成**:

$$R_3(y) = \max_x \min(R_1(x), R_2(x, y))$$

这个算子的含义可以这样理解：对候选  $y$ ，先看它和各个  $x$  的近似程度，再结合这些  $x$  本身“有多 heavy”来取最小值。最后再选择支持度最大的那条路径。于是，模糊关系不只是描述“像不像”，还可以借助合成产生新的模糊概念。

在图像识别这类任务中，模糊集合也能发挥作用。某一张图像未必能被百分之百判成某一类，它可能同时具有 0.2/Missile + 0.3/Fighter + 0.5/Airliner 这样的隶属结构。若多个规则分别给出不同目标集合，则系统可以对每一类取最大隶属度，从而得到最终判断。这种做法特别适合处理边界不清晰、视觉类别彼此有重叠的情形。

更一般地说，若有一组模糊规则

IF E1 THEN H1

IF E2 THEN H2

...

IF  $E_n$  THEN  $H_n$ 

则总体输出隶属度常写为

$$\mu_H = \max(\mu_{H_1}, \mu_{H_2}, \dots, \mu_{H_n})$$

读者初学时可能会在这里产生困惑。之前学 MYCIN 时，我们知道  $E$  是证据， $H$  是结论，算是“有了  $E$ ，推出  $H$  的把握”，即 CF 值。那么，这里的  $\mu_H$ 、 $\mu_E$  表示什么含义？实际上，在模糊规则里，它们就不再是“证据和结论”了，而是被具体化成了两个“模糊集合”。

比方说，规则的条件是  $E = \text{“Tall”}$ 。此时输入数据是  $x = 180\text{cm}$ 。那么此时，这里的  $\mu_E$  其实就表示“180cm 符合‘身材高’的程度是 0.8”。当然，如果条件变成“身材高并且体重重”，那就是把两个符合度用 min 组合起来。组合得到的最终结果，就是  $\mu_E$ ，即这条规则前件的总体符合度。而  $\mu_H$  则是这条规则决定输出的“动作强度”，例如规则的后件是  $H = \text{“衣服尺码选 XL”}$ 。如果前件的符合度  $\mu_E$  算出来是 0.8，那么后件  $H$  的动作强度也被限死在了 0.8。

在现实中，有时很多条规则在同时争夺同一个控制权。比如开车时，系统有两条规则在决定“刹车力度  $H$ ”，一是和前车的距离，二是路滑的程度。此时，就可能出现多个  $\mu_{E_i}$  和对应的  $\mu_{H_i}$ 。我们需要从这些  $\mu_{H_i}$  中选择最大值，即符合度最高的一项，来作为最终的  $\mu_H$ ，即是否要踩刹车的依据。

其中，若前件  $\mu_E$  本身又是模糊表达式，则需先对前件求值。例如当

$$E_1 = E_A \wedge (E_B \vee \neg E_C)$$

时，有

$$\mu_{E_1} = \min(\mu_{E_A}, \max(\mu_{E_B}, 1 - \mu_{E_C}))$$

这就体现了模糊逻辑中三条最基本的对应关系：AND 对应 min，OR 对应 max，NOT 对应  $1 - \mu$ 。初学时一定要牢牢记住这三条，因为它们几乎是所有模糊推理计算的起点。

到这里可以回头看整章的主线。人工智能一旦进入现实世界，就几乎无法避免不确定性。它可能来自模糊词语，来自测量误差，来自信息不全，来自统计波动，也来自证据本身的冲突。于是，系统就需要不同层面的工具去应对它。

当重点在于“新证据可能推翻旧结论”时，需要非单调推理；当重点在于“根据证据更新信念”时，需要概率与贝叶斯公式；当重点在于“结合收益和损失做选择”时，需要贝叶斯决策；当重点在于“在规则专家系统中方便表达支持和反对的程度”时，可以使用 certainty factor；当重点在于“状态随时间演化”时，可以使用马尔可夫链；当重点在于“处理自然语言中的渐变概念”时，则需要模糊集合与模糊规则。

这也说明，现实中的不确定性不是一种方法可以包打天下的。不同问题有不同结构，不同结构就需要不同工具。真正成熟的人工智能系统，并不是死守某一种推理方式，而是能够根据问题的类型，在逻辑、概率、启发式和模糊推理之间做出恰当选择。

**本章小练习**

## 1. 判断题

若观察到结果  $q$  为真, 并且已知  $p \rightarrow q$ , 就可以推出  $p$  为真。(\_\_\_\_)

## 2. 计算题

设资源勘探问题中  $P(O) = 0.6$ ,  $P(O') = 0.4$ ,  $P(+ | O) = 0.8$ ,  $P(+ | O') = 0.1$ 。求测试为正时有油的后验概率  $P(O | +)$ 。

## 3. 简答题

请说明为什么 certainty factor 不能被简单地看成严格概率, 并指出它相对于直接概率表示的一个优点和一个局限。

**参考解答**

1. 错。这种推理属于肯定后件谬误。由  $p \rightarrow q$  和  $q$  不能推出  $p$ , 因为同一个结果  $q$  可能由多个不同原因导致。

2. 先计算联合概率:

$$P(+ \cap O) = P(+ | O)P(O) = 0.8 \times 0.6 = 0.48$$

$$P(+ \cap O') = P(+ | O')P(O') = 0.1 \times 0.4 = 0.04$$

因此

$$P(+) = 0.48 + 0.04 = 0.52$$

再由贝叶斯公式得到

$$P(O | +) = \frac{P(+ | O)P(O)}{P(+)} = \frac{0.48}{0.52} = \frac{12}{13}$$

所以测试为正时有油的后验概率为  $12/13$ 。

3. certainty factor 不能被简单视为严格概率, 原因在于它的组合和传递规则主要是工程启发式, 而不是从标准概率论直接推出的。例如它可能出现“后验概率更高的假设, CF 却更低”的排序矛盾, 也常用简单乘法来近似推理链传播。它的一个优点是: 能较自然地把“支持”和“反对”分开表达, 便于专家系统与领域专家交流; 它的一个局限是: 数学一致性弱, 和严格概率模型之间可能不一致。

---

## 第六章 遗传算法

### 本章概要

前面几章中，我们已经看到，人工智能并不只有一种思路。符号主义强调知识和推理，联结主义强调神经网络与学习，而**进化主义**则从生物进化中获得启发，试图让系统通过“保留较优个体、淘汰较差个体、不断产生新候选解”的方式逐步逼近更好的答案。遗传算法正是这一思路中最经典、最具代表性的方法之一。

这一章的重点，不只是记住“选择、交叉、变异”这三个词，更重要的是理解：为什么很多复杂优化问题很难直接穷举，为什么遗传算法要维护一个种群而不是只维护一个解，轮盘赌选择是在模拟什么，交叉和变异各自提供了什么搜索能力，Gray 编码为什么会出现，旅行商、背包、可满足性问题为什么都可以转化成遗传算法来做，以及模式定理为什么常被看作遗传算法力量的理论基础。只要把这些问题想明白，整章就会非常清楚。

### 6.1 从生物进化到搜索算法

遗传算法的思想来源于生物进化。生物并不是一次就生成最完美的形态，而是在繁殖、遗传、变异和自然选择的长期作用下，逐渐形成更适应环境的结构。若把这个过程抽象成计算思想，就会得到一个非常有吸引力的想法：面对一个难以直接求最优解的问题，不如先随机生成许多候选解，把它们看成一个“种群”，再根据解的质量进行选择，让较优个体更有机会留下后代，同时通过交叉和变异不断产生新解。这样经过很多代之后，种群中出现高质量解的概率就会越来越高。

从这个角度看，遗传算法本质上是一种**随机化全局搜索方法**。它不像梯度下降那样依赖可导结构，也不像穷举法那样要把所有可能解逐一检查，而是在一个较大的搜索空间中，通过群体进化的方式不断试探、保留、组合和扰动候选解。正因为如此，它非常适合那些搜索空间巨大、目标函数复杂、缺少良好解析结构的问题。

遗传算法的一个重要特点，是它把“解”编码成一种统一的表示形式，通常是二进制串、整数串、排列或其他固定长度结构。算法本身并不直接关心“这个问题到底是选物品、排路径还是设计电路”，它更关心“当前个体怎样编码、怎样评价、怎样交叉、怎样变异”。因此，遗传算法具有很强的通用性：换一个评价函数，它就可以去解决另一类完全不同的问题。

### 6.2 一个最基本的例子：最大化 $f(x) = x^2$

为了理解遗传算法的运作方式，可以先从最简单的例子入手。设目标是最大化

$$f(x) = x^2, \quad x \in [1, 31]$$

由于区间上的整数最多到 31，所以只需要 5 位二进制就足以表示一个候选解。也就是说，每个个体都可以写成一个长度为 5 的比特串：

$$x \in 0, 1^5$$

例如，初始种群可以随机取为：

01101, 11000, 01000, 10011

把它们解释成十进制后，分别得到 13、24、8、19。再代入适应度函数  $f(x) = x^2$ ，就得到对应适应度：

$$13^2 = 169, \quad 24^2 = 576, \quad 8^2 = 64, \quad 19^2 = 361$$

这一步已经体现出遗传算法的一个基本套路：先有**表示**，再有**解释**，然后才有**评价**。二进制串本身只是编码形式；只有当它被解释成问题中的某个候选解后，适应度函数才有意义。

这个例子虽然非常简单，却已经包含了遗传算法最关键的三个要素。第一，要想办法把解编码成可操作的字符串；第二，要定义一个评价标准，也就是适应度；第三，要基于适应度来驱动后续进化。后面所有更复杂的问题，其实都只是把这三件事做得更精细而已。

### 6.3 种群、适应度与轮盘赌选择

遗传算法和许多传统优化方法最大的不同，是它维护的不是单个解，而是一个**种群** (population)。种群中的每个个体都是一个候选解，而整个种群则表示系统当前对搜索空间的一种分布式探索。使用种群的好处在于，算法不会把全部希望都压在某一个当前位置上，而是可以在多个区域同时保留搜索机会，从而更容易跳出局部最优。

在种群中，个体质量通常用**适应度** (fitness) 来度量。适应度越高，说明这个个体对应的解越优，也就越值得在下一代中保留下来。以前面的例子为例，四个个体的适应度和为

$$169 + 576 + 64 + 361 = 1170$$

于是，每个个体被选中的概率可以取为其适应度占总适应度的比例：

$$p_i = \frac{f(i)}{\sum_{j=1}^n f(j)}$$

于是四个个体的选择概率分别约为：

$$0.14, \quad 0.49, \quad 0.06, \quad 0.31$$

这说明：适应度高的个体更可能被选中，但适应度低的个体也不一定完全没有机会。这样的设计非常重要。若系统只保留当前最优个体，那么搜索会过早收缩，种群多样性迅速丢失；而若完全不考虑适应度，搜索又会退化成纯随机尝试。因此，遗传算法要追求的是“偏向优秀，但不完全僵死”的平衡。

最经典的选择方式之一，就是**轮盘赌选择** (roulette wheel selection)。可以把整个概率分布想象成一个圆盘，每个个体按其选择概率占据圆盘上的一段扇形区域。区域越大，被指针命中的概率就越大。多次旋转轮盘，就可以从种群中抽取将要繁殖的个体。连续旋转的轮盘示意图，正是在动态展示这一过程：虽然 49% 的个体最容易被抽中，14%、6%、31% 的个体也并不是完全没有机会。这种随机而带偏好的采样，正是选择操作的本质。

因此，轮盘赌并不是一个花哨的可视化技巧，而是用最直观的方式告诉我们：遗传算法中的“优胜劣汰”并不是绝对淘汰，而是按适应度改变“留下后代的机会”。



## 6.4 交叉与变异：两种不同的搜索力量

完成选择之后，遗传算法还需要真正产生下一代个体。最重要的两个遗传算子就是**交叉** (crossover) 和**变异** (mutation)。

交叉的作用，是把两个父代个体的部分信息重新组合，生成新的子代。最简单的单点交叉，可以理解为在两个二进制串上各找一个切分点，把前半段和后半段交换。例如：

```
0110|1
1100|0
```

若按这个位置切开并交换尾部，就可能得到：

```
01100
11001
```

进一步地，若从另一对个体中切出不同位置，还可能形成：

```
11|000  -> 11011
10|011  -> 10000
```

交叉之所以重要，是因为它允许系统把两个相对不错的部分结构重新拼接起来。若一个父代在前半段很优秀，另一个父代在后半段很优秀，那么交叉就有机会把这两种优点合并到同一个后代身上。也正因为如此，人们常说交叉是遗传算法中最能体现“重组已有优秀片段”思想的操作。

而变异则不同。变异通常是在一个个体内部随机改变某一位或某几位。例如

```
01100 \to 11100
```

就是把某一位从 0 翻成了 1。变异的强度一般较小，出现概率也通常较低，但它有一个非常关键的作用：防止种群因为过早趋同而失去新的可能性。若没有变异，种群中没有出现过的基因位型就很难再被重新引入，搜索空间会越来越窄，算法也更容易停在某个并不理想的区域。

因此，交叉和变异并不是互相替代的，而是两种互补的搜索力量。交叉主要负责**重组已有信息**，把已有优秀片段重新拼起来；变异主要负责**引入新的扰动**，让搜索不至于彻底陷入僵化。一个设计合理的遗传算法，往往要在“利用已有好结构”和“继续保持探索能力”之间取得平衡。

## 6.5 遗传算法的标准流程

把前面的思想综合起来，就得到了遗传算法最经典的流程：

```
begin
  set time t = 0
  initialize the population P(t)
  while the termination condition is not met do
    begin
      evaluate fitness of each member of the population P(t);
      select members from population P(t) based on fitness;
      produce the offspring of these pairs using genetic operators;
```

```

    replace candidates of P(t) with these offspring;
    set time t = t + 1
end
end

```

如果把这段伪代码翻译成更自然的话，可以理解成这样：先随机生成一批初始解；然后计算每个解有多好；再按照“好解更有机会留下后代”的原则进行选择；接着对这些被选出的父代做交叉和变异，产生新个体；用新个体替换原来的部分种群；如此反复迭代，直到达到终止条件，例如代数足够多、适应度不再提升、已经找到满意解等。

初学时很容易忽略一个事实：遗传算法真正的难点，往往不在“会不会写这个循环”，而在于每一步的细节怎样设计。例如编码方式是否合理，适应度函数是否真正反映问题目标，约束条件如何处理，交叉和变异算子是否适合当前表示，终止条件该怎样设定，等等。也就是说，遗传算法的框架很统一，但不同问题之间真正拉开差距的，是这些具体设计。

## 6.6 遗传算法适合解决什么问题

遗传算法之所以广受欢迎，很大程度上是因为它可以处理多种经典困难问题。以下几个例子很能说明这一点。

第一个例子是**背包问题** (knapsack problem)。设有  $n$  个物品，第  $i$  个物品价值为  $v_i$ ，重量为  $w_i$ ，背包总承重上限为  $W$ 。目标是在不超过容量限制的前提下，尽量让总价值最大。这个问题很适合用 0-1 串编码：某一位为 1 表示拿这个物品，为 0 表示不拿。于是，一个比特串自然对应一个候选方案，适应度函数则可以围绕“总价值”来定义，同时对超过容量限制的个体施加惩罚。

第二类例子是**逻辑可满足性问题**。例如一个命题公式处于合取范式 (CNF) 时，它由若干个子句通过 AND 连接而成，而每个子句又是若干文字的 OR。像

$$(\neg a \vee c) \wedge (\neg a \vee c \vee \neg e) \wedge (\neg b \vee c \vee d \vee \neg e) \wedge (a \vee \neg b \vee c) \wedge (\neg e \vee v)$$

这样的公式，其核心任务是：能否给每个命题变量赋真值，使整个公式为真。对这种问题，可以把每个变量的真假编码成一个二进制位，再用“满足的子句数”作为适应度。这样，遗传算法就把一个逻辑搜索问题转化成了对比特串的进化过程。

这里，我们常常用**语义树** (semantic tree) 的示例来表达这个搜索过程。它本质上体现的是：逻辑推理和定理证明常常也会展开成极大的搜索树。树一大，穷举就会迅速爆炸。这类问题并不一定总是靠遗传算法来做，但它们确实提醒我们：只要一个问题可以编码成候选结构，再定义“好坏”标准，遗传算法就有机会介入。

第三个经典例子是**旅行商问题** (traveling salesman problem, TSP)。给定若干城市以及城市间的代价，希望找到一条最便宜的闭合路线，使得每个城市恰好访问一次。这个问题最能体现搜索空间爆炸的可怕程度。若有 19 个城市，则可能路线数达到

$$18! = 6.40237 \times 10^{15}$$

这个数量级意味着，即使计算机每秒检查一万条路线，想把所有可能全部看完，所需时间也会远远超出人的寿命。遗传算法在这里的价值就很明显：既然穷举不现实，就只能依靠启发式全局搜索来尽可能逼近好解。

## 6.7 旅行商问题中的编码、交叉与变异

旅行商问题与前面的二进制编码不同，它更适合用**排列编码**。例如：

$p1 = (1\ 9\ 2\ 4\ 6\ 5\ 7\ 8\ 3)$

$p2 = (4\ 5\ 9\ 1\ 8\ 7\ 6\ 2\ 3)$

这里每个个体表示一条访问城市的顺序。与二进制串相比，排列编码有一个额外难点：每个城市必须恰好出现一次，所以交叉和变异都不能随便做，否则很容易生成非法路径。

这里的交叉方式，本质上是在两个切分点之间保留一个父代的片段，再按另一个父代的访问顺序补全剩余城市。例如从某个切点开始保留父代中的一段，再把另一个父代中尚未出现的城市按原顺序填进去，于是可能得到：

$c1 = (2\ 3\ 9\ 4\ 6\ 5\ 7\ 1\ 8)$

$c2 = (3\ 9\ 2\ 1\ 8\ 7\ 6\ 4\ 5)$

在这个例子中，如果细心观察就不难发现：两个切分点，选在了第三座城市和第四座城市之间，以及第七座城市和第八座城市之间。因此，父代和子代的第 4 到 7 座城市是相同的。而剩下的几座城市是如何交叉计算的呢？以  $c1$  为例，即为 23918。它正是原始的父代  $p2$  中，从第八座城市（第二个切分点）开始循环访问，先得到 234591876，然后再去除  $p1$  的第 4 到 7 座城市中已有的 4657 后得到的结果。

这种交叉方式的重要意义在于：它不会破坏“每个城市恰好出现一次”的约束，同时又尽量保留父代路径中的局部顺序信息。

排列编码中的变异也与二进制不同。我们看一个例子：**反转** (inversion) 式变异。例如：

$$(239465718) \rightarrow (239756418)$$

这可以理解为把中间某一段子路径倒过来。这样的变异既能产生新结构，又比完全随机打乱更温和，更适合保留已有路线中的局部好特征。不难发现，这里的两个切分点，和前面的交叉法选择的位置是相同的，都是 4657。

这个例子很好地说明：遗传算法并不是固定不变的模板。面对不同编码方式，交叉与变异的具体设计也必须改变。若仍强行使用普通单点交叉去处理排列编码，往往会得到重复城市或遗漏城市的非法结果。因此，遗传算法虽然通用，但它的“通用”不是不动脑筋地套公式，而是要求我们针对问题结构去设计合适的表示和算子。

## 6.8 Gray 编码与编码方式的重要性

在遗传算法中，编码方式本身往往会极大影响搜索表现。**Gray 编码** (Gray coding) 的出现，正是在提醒我们：编码并不是无关紧要的技术细节。还记得在数字逻辑与部件设计中学过的“格雷码”吗？它正是这里的 Gray 编码。

普通二进制有一个问题：相邻整数在编码空间中未必只差一位。例如从 7 到 8，二进制会从 0111 跳到 1000，一下子变化了四位。对遗传算法来说，这意味着在数值空间里只是一个很小的邻域移动，但在编码空间中却表现成了很大的结构变化。这样一来，交叉和变异就不容易稳定地利用“相邻解往往有相近性质”这一点。

Gray 编码则试图缓解这个问题。它的特点是：相邻整数的编码通常只变化一位。例如二进制中的 0111 和 1000 (7 和 8) 的跳变，在 Gray 码中则对应：

$$0100 \leftrightarrow 1100$$

它们就只差一个比特位。这使得搜索空间中的“局部扰动”更接近数值空间中的“局部移动”，从而让变异和交叉在某些优化问题上表现得更平滑。

怎么计算格雷码呢？在数论中，我们就学过最经典的方法：二进制转换为格雷码时，第一位保留，之后每位等于原二进制当前位与前一位异或；格雷码转换为二进制时，第一位保留，之后每位等于原格雷码与相邻的前面高位二进制结果进行异或。

因此，Gray 编码并不是一个单独的技巧，而是在提醒我们：遗传算法操作的不是“问题本身”，而是“问题的编码”。如果编码方式和问题结构严重错位，即使选择、交叉、变异都写对了，算法效果也可能很差。

## 6.9 进化硬件、随机搜索与遗传算法的优势

遗传算法的应用并不局限于组合优化。还有一类很有代表性的方向叫做**进化硬件** (evolutionary hardware)，即把电路结构、逻辑配置乃至可编程硬件单元的连接方式都编码起来，让遗传算法自动搜索更优设计。像 PLA 那类复杂电子结构图，正是在提示：遗传算法甚至可以参与“设计机器本身”。

那么，遗传算法到底比单纯随机搜索强在哪里？若把两者的搜索过程做一个直观对比，就会发现这一点。随机搜索的初始化相当于在一个大解空间中撒下很多点；随后的选择、交叉、变异若没有结构性引导，搜索依然可能大面积停留在低质量区域。相比之下，遗传算法通过适应度驱动的选择，会逐渐把搜索重心推向更高质量区域；通过交叉，它还能把多个较优区域中的局部结构重新组合；通过变异，它又不会完全丧失探索新区域的能力。

因此，遗传算法的强项并不只是“能找到红点”，而是它会随着代数增加，逐渐把搜索资源集中到更有希望的区域中。从“The beginning search space”到“The search space after n generations”这类示意变化，正是在说明：随着进化推进，种群会越来越集中在高解质量区域附近。

当然，这并不意味着遗传算法总是优于其他方法。它仍然可能收敛慢、参数敏感、过早成熟，甚至在某些问题上不如专门设计的局部搜索或数学规划方法。但它最大的优势在于：不要求梯度信息，不依赖问题可导，适应面广，且在巨大复杂搜索空间中往往能以相对朴素的机制取得相当不错的结果。

## 6.10 模式、阶与定义长度

如果说前面的内容更多是在解释遗传算法“怎么做”，那么接下来要讨论的模式理论，则是在尝试说明它“为什么会有效”。这里最关键的观念是**模式** (schema)。

模式可以看作一种模板，它描述了一批具有某些共同位置特征的字符串。例如模式

$(1 * * 0 * 1)$

表示长度为 6 的所有二进制串中，第 1 位是 1，第 4 位是 0，第 6 位是 1，而第 2、3、5 位可以任意取 0 或 1。这里的 \* 是通配符，表示该位置不作限制。

模式最重要的两个量是**阶** (order) 和**定义长度** (defining length)。模式的阶，记为  $O(s)$ ，表示模式中被固定的位置个数。以上面的模式为例，固定了第 1、4、6 位，因此

$$O(s) = 3$$

模式的定义长度，记为  $\delta(s)$ ，表示第一个固定位置和最后一个固定位置之间的距离。对于模式  $(1 * * 0 * 1)$ ，最左和最右的固定位置分别是第 1 位和第 6 位，因此

$$\delta(s) = 5$$

为什么这两个量重要？因为在遗传算法中，交叉和变异对模式的破坏概率，与模式的长度和固定位置多少密切相关。首先，固定位置越多，变异时越容易被打乱。这是因为模式本身要求这些位置必须是这些固定值，一旦发生反转，那么很可能就不再符合这个模式。此外，跨越范围越长，交叉时越容易被切开。

模式的**适应度**，则定义为所有匹配该模式的字符串适应度的平均值。若一个模式中包含了許多较优个体，那么这个模式本身就可以被看作“一个值得传播的优良结构片段”。这也是模式理论最核心的直觉：遗传算法并不只是筛选完整个体，它还在无形中筛选那些隐藏在个体内部的优良子结构。

## 6.11 模式定理

设  $m(s, t)$  表示第  $t$  代中属于模式  $s$  的字符串个数。若种群大小为  $n$ ，个体  $i$  的适应度为  $f(i)$ ，则它被选中的概率是

$$p_i = \frac{f(i)}{\sum_{j=1}^n f(j)}$$

若模式  $s$  的平均适应度记为  $f(s)$ ，整代种群的平均适应度记为

$$\bar{f} = \frac{1}{n} \sum_{j=1}^n f(j)$$

那么在只有选择、暂时不考虑交叉和变异破坏的情况下，模式在下一代中的期望出现次数满足

$$E[m(s, t+1)] = m(s, t) \frac{f(s)}{\bar{f}}$$

这个式子已经非常重要了。它告诉我们：如果某个模式的平均适应度高于整代平均适应度，那么它在下一代的期望个数就会上升；反之则会下降。

若进一步设

$$f(s) = (1 + c)\bar{f}$$

其中， $c > 0$  表示该模式优于平均水平的程度，那么有

$$E[m(s, t + 1)] = m(s, t)(1 + c)$$

于是继续迭代可以得到一种指数增长趋势：

$$E[m(s, t + 1)] \approx m(s, 0)(1 + c)^{t+1}$$

这说明，只要某个模式平均来说优于整体，而且不被遗传操作大量破坏，那么它会在种群中越来越常见。

接下来还要考虑交叉的影响。若交叉概率为  $p_c$ ，字符串长度为  $l$ ，那么一个模式在交叉中不被破坏的概率至少满足

$$p_s \geq 1 - p_c \frac{\delta(s)}{l - 1}$$

这说明：模式跨越得越长，即  $\delta(s)$  越大，就越容易被切分点打断。

再考虑变异。若单个位的变异概率为  $p_m$ ，则一个模式中所有固定位都不被变异破坏的概率近似为

$$p_s \approx 1 - O(s)p_m$$

这里用到了  $p_m \ll 1$  的近似。它告诉我们：模式固定的位置越多，即阶越高，就越容易在变异中被打散。

把这些因素综合起来，就得到著名的**模式定理** (schema theorem)：

$$E[m(s, t + 1)] \geq m(s, t) \frac{f(s)}{\bar{f}} \left[ 1 - p_c \frac{\delta(s)}{l - 1} - O(s)p_m \right]$$

这个不等式表达的思想非常深刻：**短的、低阶的、平均适应度高于整体平均的模式，会在连续多代中得到指数式放大。**这类模式常被称为 “building blocks”，也就是遗传算法在演化过程中反复保留和重组的优良积木块。

从这里可以看出，遗传算法之所以有效，并不只是因为“随机试了很多次”，而是因为它会逐渐积累并重组这些局部优良结构。模式定理虽然不是解释遗传算法一切现象的最终理论，但它确实抓住了一个核心事实：搜索并不是完全盲目的，选择、交叉和变异在统计意义上会偏向传播某些高质量结构片段。

本章要建立的，是一种不同于传统解析优化的思维方式。遗传算法并不试图沿着一个明确梯度一步步走向最优，也不要求问题必须有好用的数学结构；它更像是在一个巨大的可能性空间中维护一个活的种群，让种群在选择、交叉和变异中不断重组，逐步把搜索压力推向更有希望的区域。

## 本章小练习

### 1. 判断题

遗传算法中的选择操作，意味着每一代只能保留当前适应度最高的那个个体，其余个体全部淘汰。（\_\_\_\_）

### 2. 计算题

设某一代种群中有四个个体，其适应度分别为 169、576、64、361。求它们按轮盘赌选择时的概率。

### 3. 简答题

请说明为什么在遗传算法中，交叉和变异缺一不可。

### 参考解答

1. 错。遗传算法中的选择通常是按适应度分配被选中的概率，而不是只保留唯一最优个体。适应度高的个体更容易留下后代，但适应度较低的个体往往也仍保留一定机会，这样才能维持种群多样性。
2. 先计算总适应度：

$$169 + 576 + 64 + 361 = 1170$$

因此四个个体的选择概率分别为

$$\frac{169}{1170} \approx 0.144, \quad \frac{576}{1170} \approx 0.492, \quad \frac{64}{1170} \approx 0.055, \quad \frac{361}{1170} \approx 0.309$$

3. 交叉和变异作用不同，因此不能互相替代。交叉的主要作用，是把不同父代中的局部优良结构重新组合起来，从而有机会形成更高质量的后代；这体现的是对已有信息的利用。变异的主要作用，则是在种群中重新引入新的位型和新的可能性，防止整个种群因为过早趋同而失去继续探索搜索空间的能力。若只有交叉没有变异，种群会越来越像，新的信息难以出现；若只有变异没有交叉，搜索又会退化成较弱的随机扰动，难以有效重组已有好结构。

## 第七章 逻辑与 Prolog

### 本章概要

人工智能中的逻辑系统有一种很特别的地位。它既像数学，因为它强调形式、语义和证明；又像程序设计，因为它可以被直接执行，甚至变成一种编程语言。若前面的知识表示一章讨论的是“知识怎样存”，那么这一章讨论的就是“知识怎样以逻辑形式写出来，并进一步用于推理、证明和程序执行”。

这一章的内容很多，但主线非常清楚。首先要弄懂命题逻辑和一阶逻辑分别能表达什么；然后要学会把自然语言翻译成逻辑公式；接着要掌握一些最重要的逻辑等价变换和推理规则；再往后，重点就是把公式一步步变成子句形式，利用归结反驳法完成证明；最后再进入 Horn 子句与 Prolog，理解为什么逻辑规则可以直接当程序运行，为什么子句顺序和目标顺序会影响结果，以及列表、回溯、剪枝、路径搜索这些经典例子又是在展示什么。只要抓住这条线，整章虽然长，却并不杂乱。

### 7.1 逻辑在人工智能中的位置

在人工智能中，逻辑并不是单纯为了训练形式化思维而存在。它真正的价值在于：能够把含义明确、结构清楚、可被严格检查的知识写出来，并且让系统自动推出新的结论。只要知识和推理规则都写得足够规范，机器就能按照固定方法进行证明，而不依赖人的直觉猜测。

从更高层次看，逻辑也是知识表示的重要基础之一。要让机器理解“所有篮球运动员都高”“有些人喜欢凤尾鱼”“John 有祖父”“某人如果幸运就会中彩票”这类命题，仅靠自然语言字符串远远不够，必须把它们转化成一种内部结构化表示。逻辑恰好提供了这种能力。

当然，不同层次的逻辑表达能力并不相同。命题逻辑可以处理真假命题之间的组合关系，但它看不见对象内部结构；一阶逻辑则能进一步描述对象、属性、关系、函数和量词，因此表达能力强得多。Prolog 则是在 Horn 子句这一特殊逻辑形式之上，发展出的一种可执行逻辑程序语言。也就是说，这一章并不是把“逻辑”和“程序”分开讲，而是在展示：某些逻辑表示经过适当限制之后，可以直接转化为程序执行过程。

## 7.2 命题逻辑：从真假命题到复合公式

**命题逻辑** (propositional logic) 研究的，是一些整体上可以判定真假的命题，以及这些命题如何通过连接词组合成更复杂的公式。最基本的符号包括命题符号  $P, Q, R, S, \dots$ ，真值符号 true 和 false，以及几个经典连接词：

$\wedge$  表示合取， $\vee$  表示析取， $\neg$  表示否定

$\rightarrow$  表示蕴含， $\equiv$  表示等值

命题逻辑中的“句子”是递归定义的。每一个命题符号本身就是句子，真值符号也是句子；若  $P$  是句子，那么  $\neg P$  也是句子；若  $P$  和  $Q$  都是句子，那么  $P \wedge Q$ 、 $P \vee Q$ 、 $P \rightarrow Q$ 、 $P \equiv Q$  也都是句子。合法句子常被称为 **良构公式** (well-formed formula, WFF)。

命题逻辑的语义，是给每个命题符号赋真假值。一个**解释** (interpretation)，就是对每个命题符号指定 true 或 false。在此基础上，各种复合公式的真值由定义决定。否定  $\neg P$  在  $P$  为真时取假，在  $P$  为假时取真；合取  $P \wedge Q$  只有在二者都真时才为真；析取  $P \vee Q$  只有在二者都假时才为假。

其中最值得特别强调的是**蕴含**  $P \rightarrow Q$ 。很多初学者会觉得“如果……那么……”一定很强，但在形式逻辑里，它只有在在前件为真而后件为假时才为假，其他三种情况都为真。例如“下过雨  $\rightarrow$  地面是湿的”这一命题，在“没下雨且地面不湿”“没下雨但地面是湿的”“下过雨且地面是湿的”这三种情况下都为真，只有“下过雨但地面不湿”时为假。这一点虽然初看有些反直觉，但它恰恰对应了蕴含的形式定义：只要没有出现“前件成立但后件不成立”的反例，蕴含就被视为成立。

命题逻辑中还有大量非常重要的等价变换。它们之所以重要，不只是因为化简过程中经常要用到，更因为后面把公式转成子句形式时，基本都要反复调用这些法则。例如：

$$\neg(\neg P) \equiv P$$

$$P \rightarrow Q \equiv \neg P \vee Q$$

$$P \rightarrow Q \equiv \neg Q \rightarrow \neg P$$



$$\neg(P \wedge Q) \equiv \neg P \vee \neg Q$$

$$\neg(P \vee Q) \equiv \neg P \wedge \neg Q$$

$$P \vee (Q \wedge R) \equiv (P \vee Q) \wedge (P \vee R)$$

$$P \wedge (Q \vee R) \equiv (P \wedge Q) \vee (P \wedge R)$$

这些法则中，尤其要熟练掌握蕴含消去、德摩根律、交换律、结合律和分配律。因为一旦进入归结法，很多看似复杂的问题，本质上就是在不停地利用这些等价变换把句子改写成适合机器操作的形式。

### 7.3 从命题逻辑到一阶逻辑

命题逻辑虽然整洁，但表达能力有限。它只能把“John is tall”当成一个不可再分的整体命题，却无法看出其中有对象 John 和性质 tall，也不能表达“所有人都……”或者“存在某个人……”。这就引出一阶逻辑（first-order logic）。

一阶逻辑比命题逻辑多出了几个关键成分。第一是**常量符号**（constant），用于表示具体对象，如 john、socrates。第二是**变量符号**（variable），如  $x, y, z$ 。第三是**函数符号**（function），表示从若干对象到另一个对象的映射，如 father(x)、sum(x,y)。第四是**谓词符号**（predicate），表示对象之间的性质或关系，如 man(x)、parent(x,y)、equal(x,y)。此外，一阶逻辑还引入了两个核心量词：

$\forall$  表示全称量词， $\exists$  表示存在量词

于是，我们终于可以表达结构化的知识。例如：

$$\forall x(man(x) \rightarrow mortal(x))$$

表示“所有人都会死”；而

$$man(socrates)$$

表示“Socrates 是人”。把二者结合，就可以推出

$$mortal(socrates)$$

这正是逻辑最经典的例子之一。

再比如，“所有篮球运动员都高”可以写成

$$\forall x(basketball_p layer(x) \rightarrow tall(x))$$

“没有人喜欢税收”则可以写成

$$\neg \exists x(like(x, taxes))$$

因此，一阶逻辑真正带来的进步，不只是“多了一些新符号”，而是让我们能把对象、属性、关系和量词都放进公式中，表达能力一下子从“命题之间的真假组合”提升到了“对象世界中的结构关系”。

#### 7.4 一阶逻辑的语义：解释、满足、模型、有效

要真正理解逻辑，不能只会写公式，还必须懂它的语义。设有一个非空集合  $D$  作为讨论域 (domain)。一个**解释**，就是把常量、变量、函数和谓词分别映射到这个域及其相关结构中去。常量对应域中的具体元素，函数对应  $D^m$  到  $D$  的映射，谓词对应  $D^n$  到  $\{true, false\}$  的映射。

比方说， $D$  是西游记里的所有角色，那么它就先限定了后续的讨论都只能针对这些人物展开。这时，常量就是指给  $D$  中的元素“起个名字”，比方说  $a$  是孙悟空， $b$  是猪八戒， $c$  是唐僧等。函数就是从  $D^m$  到  $D$  的映射，比方说我们可以令  $f(x)$  表示“ $x$  的师傅”。而谓词就是从  $D^n$  到  $\{true, false\}$  的映射，它和函数的区别就在于输出必须是一个布尔值。比方说，我们令  $P(x, y)$  表示“ $x$  打得过  $y$ ”，那么它就是一个谓词。在这个背景下， $P(a, f(b))$  实际就表示“孙悟空打得过唐僧”，如果单从武力值上看，那么显然应该输出 true。

对于一开始没有任何实质意义的一个公式（比方说  $x + y = z$ ），我们要先做出一个解释（比方说是实数域上的加法，或是钟表的刻度相加），它只包含定义域、常量、函数和谓词的确定，而不包含变量赋值。而变量赋值是独立于解释之外的动态过程，它解决的是变量的取值问题。因此，一个公式的真值必须由解释和变量赋值共同决定。

若一个公式在某个解释和某个变量赋值下为真，就说该解释**满足** (satisfy) 这个公式。若一个解释对某公式的所有变量赋值都使其为真，那么这个解释就是该公式的一个**模型** (model)。一个公式若至少存在一个解释使其为真，就叫做**可满足的** (satisfiable)；若不存在任何解释使其为真，就叫做**不可满足的** (unsatisfiable)。如果一个公式在所有可能解释下都为真，那么它叫做**有效式** (valid)。

这几个概念一定要分清。可满足表示“至少有一种可能世界让它成立”，有效表示“无论哪种可能世界它都成立”。例如：

$$P(x) \vee \neg P(x)$$

是一个典型有效式，因为无论怎样解释，排中律都成立。而

$$\exists x(P(x) \wedge \neg P(x))$$

则是不可满足的，因为它要求某个对象同时满足  $P$  和  $\neg P$ ，这在标准逻辑中不可能成立。若一组公式整体不可满足，就说这组公式是**不一致的** (inconsistent)。

这些定义非常重要，因为后面的归结证明，最终就是在利用“若加入目标的否定后得到不一致，则原目标可由前提推出”这一思想。换句话说，我们先假设前提成立，同时结论不成立，看是否会发生冲突。如果冲突，那么这个结论自然就可被推出。

## 7.5 自然语言到逻辑表达

逻辑最容易失分、也最需要真正理解的地方之一，就是**把自然语言翻译成逻辑表达式**。这一步看上去只是“换一种记号”，实际上是在训练对语义结构的把握能力。

例如，“如果星期一不下雨，Tom 就去山里”可以写作

$$\neg \text{weather}(\text{rain}, \text{monday}) \rightarrow \text{go}(\text{tom}, \text{mountains})$$

“Emma 是一只杜宾犬，而且是一只好狗”可以写作

$$\text{gooddog}(\text{emma}) \wedge \text{isa}(\text{emma}, \text{doberman})$$

“有些人喜欢凤尾鱼”使用存在量词：

$$\exists x(\text{person}(x) \wedge \text{like}(x, \text{anchovies}))$$

在这类翻译中，最关键的是先判断句子在语义上属于哪一类：它是在陈述一个普遍规律，还是在陈述某个个体事实；是在说“存在某个对象”，还是在说“所有对象都满足”；是在说一个属性，还是在说一个关系；是在说条件蕴含，还是在说合取或析取。若这些判断不清楚，翻译出来的逻辑表达往往就会错。

再比如家庭关系问题：

$$\forall x \forall y ((\text{father}(x, y) \vee \text{mother}(x, y)) \rightarrow \text{parent}(x, y))$$

$$\forall x \forall y \forall z ((\text{parent}(x, y) \wedge \text{parent}(x, z)) \rightarrow \text{sibling}(y, z))$$

再配合事实

$$\text{mother}(\text{eve}, \text{abel}), \quad \text{mother}(\text{eve}, \text{cain}), \quad \text{father}(\text{adam}, \text{abel}), \quad \text{father}(\text{adam}, \text{cain})$$

就可以推出

$$\text{sibling}(\text{cain}, \text{abel})$$

这个例子说明：逻辑表达不仅是写公式，更是在把“常识中的推理链”压缩成若干可机械应用的规则。

## 7.6 推理、逻辑后继与证明过程

若一个表达式  $x$  在所有满足一组前提  $S$  的解释中都为真，就说  $x$  是  $S$  的**逻辑后继** (logically follows)。这一定义非常严格，它告诉我们：真正的逻辑推理不是“看起来像对的”，而是在所有可能解释下都不会出错。

为把这种语义关系变成可计算过程，人们设计了各种**证明过程** (proof procedure)。一个证明过程由两部分组成：一套推理规则，以及一套决定何时、如何应用这些规则的算法。一个推理规则若推出的所有结论都真的是原前提的逻辑后继，就说它是**可靠的** (sound)；若对所有真的逻辑后继，它最终都有能力推出，则说它是**完全的** (complete)。

在这里， $\vdash$  符号的含义就是“可推导”。当它用于逻辑表达式时，通常写作  $\Gamma \vdash \alpha$ ，表示公式  $\alpha$  可以从前提集合  $\Gamma$  中，依据推理规则一步步证明出来。

在基本推理规则中，最重要的包括：

**肯定前件** (modus ponens)：

$$P, \quad P \rightarrow Q \vdash Q$$

**否定后件** (modus tollens)：

$$P \rightarrow Q, \quad \neg Q \vdash \neg P$$

**消去规则**：由  $P \wedge Q$  可以推出  $P$  和  $Q$ 。

**引入规则**：若已知  $P$  和  $Q$ ，则可推出  $P \wedge Q$ 。

**全称实例化** (universal instantiation)：由

$$\forall x P(x)$$

可推出

$$P(a)$$

其中  $a$  是讨论域中的适当对象。比如由

$$\forall x (man(x) \rightarrow mortal(x))$$

就可得到

$$man(socrates) \rightarrow mortal(socrates)$$

再结合

$$man(socrates)$$

最终我们就可以由 modus ponens 推出

$$mortal(socrates)$$

## 7.7 合一：让两个表达式“对上”

在一阶逻辑推理中，一个非常核心的技术是**合一** (unification)。它要解决的问题是：给定两个含变量的表达式，能否通过恰当替换变量，让它们变成完全相同的形式。

例如：

```
parents(x, father(x), mother(bill))
parents(bill, father(bill), y)
```

在这里，我们令  $parents(A, B, C)$  函数表示“ $A$  的父亲是  $B$ ，母亲是  $C$ ”。

要让它们一致，首先可令

$$\{bill/x\}$$

这个式子的含义就是“用 bill 去替换变量  $x$ ”。

于是第一个表达式变成

```
parents(bill, father(bill), mother(bill))
```

接着再令

$$\{mother(bill)/y\}$$

第二个表达式也变成一样。于是最一般的合一代换就是

$$\{bill/x, mother(bill)/y\}$$

这里特别重要的是**最一般合一子** (most general unifier, mgu)。所谓“最一般”，意思不是把式子对齐就行，而是要尽量保持代换的普适性，不做不必要的额外限制。若一个更具体的代换也能合一，那么它应当可以由最一般合一子再加进一步代换得到。

合一时还有几条必须牢记的限制。第一，常量不能被随便替换。第二，同一个变量在所有出现位置上必须一致替换。第三，一个变量不能和包含它自己的项合一，例如不能令  $x = f(x)$ ，否则会产生循环结构。这些规则保证了合一过程既

一致，又不会滑向荒谬。

合一之所以重要，是因为一阶逻辑推理中的许多步骤，实质上都依赖“找到一组恰当变量替换，使两个公式在某个位置恰好互补或匹配”。比方说在医疗中，可能有一种疾病，如果母亲患有，那么孩子患病概率也很高。此时如果我们得知了  $mother(bill, mary)$  和  $has\_disease(mary, some\_disease)$ ，那么只有通过和证据表达式的合一，才能够证明  $bill$  很可能患病。

## 7.8 归结反驳法的基本思想

一旦进入自动定理证明，最重要的方法之一就是**归结反驳法** (resolution refutation)。它的核心思想非常经典：

1. 先把全部前提写成子句形式。
2. 把要证明的目标取否定，也写成子句形式，并加入前提集合。
3. 通过归结不断推出新的子句。
4. 若最终推出空子句  $\square$ ，就说明前提加目标否定不可满足，从而原目标成立。

这个方法之所以强大，是因为它把“证明某个结论”为真，转化成了“证明加入其否定后会发生矛盾”。从逻辑上说，这是非常自然的：若目标的否定与原前提无法共存，那么原目标就必然是前提的逻辑后继。

例如，设有前提：

$$\forall x(dog(x) \rightarrow animal(x))$$

$$\forall y(animal(y) \rightarrow die(y))$$

$$dog(fido)$$

目标是证明

$$die(fido)$$

按归结反驳法，要加入目标否定：

$$\neg die(fido)$$

再把前提改写成子句：

$$\neg dog(x) \vee animal(x)$$

$$\neg animal(y) \vee die(y)$$

$$dog(fido)$$

$$\neg die(fido)$$

接着不断归结。第一步由

$$\neg dog(x) \vee animal(x)$$

和

$$dog(fido)$$

在代换  $\{fido/x\}$  下归结, 得到

$$animal(fido)$$

第二步再由

$$\neg animal(y) \vee die(y)$$

和

$$animal(fido)$$

在代换  $\{fido/y\}$  下归结, 得到

$$die(fido)$$

最后再与

$$\neg die(fido)$$

归结, 得到空子句

□

这就完成了证明。有时，我们也把上述归结过程画成一棵分支与合并不断推进的证明树。

## 7.9 从自然语言到子句形式

这一部分是整章里最需要认真掌握的地方之一，因为许多自动推理方法都要求输入先化成**子句形式** (clause form) 或合取范式 (CNF) 近似结构。对复杂一阶逻辑公式而言，这一变换通常按固定步骤进行。

设原公式较复杂，含有蕴含、否定、量词和嵌套结构。那么常见流程如下。

第一步，**消去蕴含**。因为归结法不直接处理  $\rightarrow$ ，所以要用等价式

$$a \rightarrow b \equiv \neg a \vee b$$

把所有蕴含改写掉。

第二步，**把否定尽量往里推**，直到否定只直接作用在原子公式上。这一步依赖双重否定律和德摩根律，以及量词的否定变换：

$$\neg \exists x(a(x)) \equiv \forall x(\neg a(x))$$

$$\neg \forall x(b(x)) \equiv \exists x(\neg b(x))$$

$$\neg(a \wedge b) \equiv \neg a \vee \neg b$$

$$\neg(a \vee b) \equiv \neg a \wedge \neg b$$

第三步，**标准化变量名**。也就是把不同量词约束下的变量改名，保证彼此不冲突。例如：

$$\forall x(a(x)) \vee \forall x(b(x))$$

应改写为

$$\forall x(a(x)) \vee \forall y(b(y))$$

第四步，**把量词前移**。把全部量词移到公式最左侧，但不改变原有依赖次序。



第五步，**Skolem 化**。这是非常关键的一步。所有存在量词都要被消去，用新的 Skolem 常量或 Skolem 函数替换。若存在量词不依赖任何前面的全称变量，就用一个新常量代替；若依赖某些全称变量，就用这些变量的函数来代替。比如“每个人都有一个父母”

$$\forall x \exists y \text{ parent}(x, y)$$

经过 Skolem 化后，可以写成

$$\text{parent}(x, pa(x))$$

这里的  $pa(x)$  就是“ $x$  的某个父母”的函数化表示。它不要求真的知道是谁，只要求逻辑上给这个存在对象一个统一名字。

第六步，**删去全称量词**。因为经过前面处理后，剩下的量词默认都可以看作对整个子句的全称约束，于是在子句层面通常直接省略。

第七步，**用分配律把公式化成合取的析取式**，即把公式写成多个括号的合取  $\wedge$ ，且要求每个括号内只能包含析取  $\vee$ 。例如：

$$a \vee (b \wedge c) \equiv (a \vee b) \wedge (a \vee c)$$

如此反复，把公式改写成若干析取子句的合取。

第八步，**把每个合取项单独看成一个子句**。例如：

$$[\neg a(x) \vee \neg b(x) \vee c(x, i) \vee e(w)]$$

和

$$[\neg a(u) \vee \neg b(u) \vee \neg c(f(u), g(u)) \vee d(u, f(u)) \vee e(v)]$$

就是两个独立子句。

第九步，**再次变量标准化**，确保不同子句之间的变量名也互不混淆。

这套流程要形成机械的记忆。也就是说，看到一个自然语言结论或复杂逻辑公式后，应当立刻知道：先消蕴含，再推否定，再改变量名，再前移量词，再做 Skolem 化，再化成合取析取式，再拆子句。

## 7.10 两个经典归结例子

为了真正理解归结反驳法，还需要看几个完整故事型例子。

第一个例子是“John 是否快乐”。自然语言知识如下：通过历史考试并中彩票的人会快乐；学习或者幸运的人都能通过考试；John 没学习，但他很幸运；幸运的人会中彩票。问题是：John 快乐吗？

把它们写成逻辑形式：

$$\forall x(\text{pass}(x, \text{history}) \wedge \text{win}(x, \text{lottery}) \rightarrow \text{happy}(x))$$

$$\forall x \forall y(\text{study}(x) \vee \text{lucky}(x) \rightarrow \text{pass}(x, y))$$

$$\neg \text{study}(\text{john}) \wedge \text{lucky}(\text{john})$$

$$\forall x(\text{lucky}(x) \rightarrow \text{win}(x, \text{lottery}))$$

目标是证明  $\text{happy}(\text{john})$ ，所以加入否定目标

$$\neg \text{happy}(\text{john})$$

化为子句后，关键几步是：由  $\text{lucky}(\text{john})$  和  $\neg \text{lucky}(u) \vee \text{win}(u, \text{lottery})$  推出  $\text{win}(\text{john}, \text{lottery})$ ；由  $\text{lucky}(\text{john})$  和“学习或幸运  $\rightarrow$  通过考试”推出  $\text{pass}(\text{john}, \text{history})$ ；再和“通过历史考试且中彩票的人会快乐”合起来推出  $\text{happy}(\text{john})$ ；最后与  $\neg \text{happy}(\text{john})$  归结得到空子句。这个过程也可以用归结树表示。

第二个例子是“谁拥有令人兴奋的生活”。知识是：不贫穷且聪明的人快乐；会读书的人不愚蠢，而这里又给出假设

$$\forall x(\text{smart}(x) \equiv \neg \text{stupid}(x))$$

$$\forall y(\text{wealthy}(y) \equiv \neg \text{poor}(y))$$

已知 John 会读书且富有，快乐的人有令人兴奋的生活。问题是：能否找到一个拥有令人兴奋生活的人？

这种题的重点不只是推出结论，还要会做**答案提取** (answer extraction)。在归结反驳过程中，系统并不只是证明“存在某人”，还会通过合一代换逐步把那个存在变量落实现成具体对象，例如最后得到代换链表明那个对象就是 john。通过归结路径上的代换合成，可以把“存在谁”具体提取出来。

再比如祖父问题。若有

$$\forall x \exists y \text{parent}(x, y)$$

以及

$$\forall x \forall y \forall z (parent(x, y) \wedge parent(y, z) \rightarrow grandparent(x, z))$$

要证明 John 有祖父。经过 Skolem 化后，前者变成

$$parent(x, pa(x))$$

于是进一步可推出

$$grandparent(john, pa(pa(john)))$$

这个结果非常值得注意，因为它展示了 Skolem 函数的真实作用：即使不知道 John 的祖父到底是谁，逻辑系统仍然可以用一个函数项清楚地表示“John 的父母的父母”。

### 7.11 归结策略与 Herbrand 观点

归结法虽然强大，但并不是所有归结顺序都同样高效。于是就有了各种**归结策略**。常见策略包括广度优先、支持集策略、单子句优先策略、线性输入形式等。它们的目标，都是减少无效搜索，尽快把推理引向矛盾。

其中，线性输入形式比较容易理解：每一步新归结出的子句，都必须和原输入集中的某个子句继续归结，而不是允许任意两个中间子句自由组合。它的优点是结构比较简单，但并不总是完全的，也就是说，有些本来可证明的结论，在这种受限策略下未必能推出来。

还需要提到 **Herbrand 结构** 与 **语义树**。Herbrand 观点的核心，是把一阶逻辑中的讨论域限制在由常量和函数反复生成出来的所有基项上，从而把一阶逻辑问题拉回到某种“可枚举结构”的层面。例如 Herbrand 域可写为

$$H = \bigcup_{i=0}^{\infty} H_i$$

其中，若原子句中已有常量，就从这些常量起步；若没有常量，就引入一个新常量。之后不断把函数作用到已有项上，得到越来越大的域。语义树则是另一种直观工具，它通过树形分支去检查一组子句是否可能被满足，或者展示在归结过程中各条路径怎样被逐步关闭。各种树状示意图，正是这类思想的图形化体现。

### 7.12 逻辑系统的层次与扩展

经典命题逻辑和一阶逻辑并不是逻辑世界的全部。不同逻辑系统在表达能力和可判定性之间存在权衡。

表达能力极强的逻辑，如高阶逻辑或某些一阶逻辑扩展形式，往往会面临不可判定问题。表达能力较弱但结构受限的逻辑，则可能计算复杂度较低。介于命题逻辑和一阶逻辑之间，还有一类可判定但量词受限的逻辑系统，例如模态逻辑、描述逻辑、命题时序逻辑等。

这里，我们再学习两个名词：**默认推理** (default reasoning) 与**情景限制** (circumscription)。例如默认逻辑中的规则可以写成

$$\frac{p : \neg r_1 \wedge \cdots \wedge \neg r_n}{q}$$

其含义是：若已知  $p$ ，且没有证据表明  $r_1, \dots, r_n$  成立，那么默认推出  $q$ 。这和前面不确定性一章中的非单调推理有密切关系。模态逻辑则引入必然性和可能性：

$$\Box p, \quad \Diamond p$$

并满足如

$$\Box p \leftrightarrow \neg \Diamond \neg p$$

$$\Box(p \rightarrow q) \rightarrow (\Box p \rightarrow \Box q)$$

这些内容在这一章中属于拓展视野，但应当知道：逻辑并不是一种单一语言，而是一整族系统，每一类都在表达能力与计算代价之间作不同取舍。

### 7.13 Horn 子句：让逻辑变成程序

若一个子句中**至多只有一个正文字**，那么它就叫做 **Horn 子句** (Horn clause)。一般形式可写成

$$a \vee \neg b_1 \vee \neg b_2 \vee \cdots \vee \neg b_n$$

为了突出这个唯一正文字的结论地位，通常把它写成蕴含形式：

$$a \leftarrow b_1 \wedge b_2 \wedge \cdots \wedge b_n$$

这正是 Prolog 规则最熟悉的样子。

Horn 子句通常有三种形式。第一种是**事实**：

$$a \leftarrow$$

它表示  $a$  无条件成立。第二种是**规则**：

$$a \leftarrow b_1 \wedge b_2 \wedge \cdots \wedge b_n$$

它表示若  $b_1, \dots, b_n$  都成立，则推出  $a$ 。第三种是**无头子句**或**目标子句**：

$$\leftarrow a_1 \wedge a_2 \wedge \cdots \wedge a_n$$

它表示我们要去证明这些目标。这是什么意思呢？我们知道，无头子句  $\leftarrow a_1 \wedge a_2 \wedge \cdots \wedge a_n$  的含义，本质上就是对它取反。而底层推理机所使用的归结反证法，套路就是先假定结论是错的，如果最后推导出了空子句，就说明结论其实是对的。因此，当我们去问推理机一个问题是否正确时，系统拿到目标后会立刻在前面加一个逻辑否定符号，然后将它作为一个“待消去的目标列表”扔进机器里。至于消去的手段，自然就是当前知识库中的规则或知识。

Horn 子句之所以特别重要，是因为在这种受限形式下，逻辑推理会变得非常适合程序执行。而下面的 Prolog 正是建立在这一思想之上：知识库是事实和规则，查询是目标，而解释器则通过不断把当前目标替换成新的子目标来尝试求解。

## 7.14 Prolog 的执行机制

Prolog 的执行过程，可以看成一种受控的逻辑归结。给定目标

$$\leftarrow a_1 \wedge a_2 \wedge \cdots \wedge a_n$$

解释器会从左到右，先处理最左边的目标  $a_1$ 。它在程序中寻找第一个能够与  $a_1$  合一的规则头。若找到某条规则

$$a' \leftarrow b_1 \wedge b_2 \wedge \cdots \wedge b_m$$

并且合一替换为  $\xi$ ，则原目标就被替换为

$$\leftarrow (b_1 \wedge b_2 \wedge \cdots \wedge b_m \wedge a_2 \wedge \cdots \wedge a_n)\xi$$

然后系统继续处理新的最左目标。若最终所有目标都被消去，得到空目标  $\square$ ，那么查询成功；若中途失败，则回溯，尝试其他规则或其他合一方式。

因此，Prolog 的执行顺序有两个鲜明特点。第一，**目标总是按从左到右的顺序处理**。第二，**规则总是按自上而下的顺序尝试**。这两个顺序看似只是实现细节，实际上会极大影响程序能否成功、效率怎样，甚至是否陷入无限递归。

## 7.15 Prolog 的基本语法与简单例子

Prolog 语法和逻辑记号之间有一套比较清楚的对应关系：

```
and      -> ,
or       -> ;
only if  -> :-
not      -> not
```

例如事实库

```
likes(george, kate).
likes(george, susie).
```

```
likes(george, wine).
likes(susie, wine).
likes(kate, gin).
```

对应的查询:

```
?- likes(george, susie).
```

会成功; 而

```
?- likes(george, gin).
```

会失败。若查询

```
?- likes(george, x).
```

则系统会依次返回

```
x = kate
x = susie
x = wine
```

这说明 Prolog 的变量本质上是在求“让目标成立的代换”。它并不是只会回答“是”或“否”，而是会把满足条件的对象逐个枚举出来。

若再加入规则

```
friends(x, y) :- likes(x, z), likes(y, z).
```

那么它表示: 若  $x$  和  $y$  都喜欢同样的东西  $z$ , 则它们是朋友。由此可见, Prolog 程序其实就是把事实和逻辑规则直接写出来, 而求值过程就是自动搜索能让目标成立的变量代换。

## 7.16 列表匹配与递归程序

Prolog 的另一个经典主题是列表。因为列表特别适合展示“模式匹配 + 递归”这种逻辑程序设计方式。

例如:

```
[tom, dick, harry]
```

若与

```
[x | y]
```

匹配, 则可得  $x = tom$ ,  $y = [dick, harry]$ 。若与

```
[x, y | z]
```

匹配, 则可得  $x = tom$ ,  $y = dick$ ,  $z = [harry]$ 。这种“头尾拆分”是 Prolog 列表处理最核心的机制。

经典的“检验一个元素是否在表中”的 member 程序写作:

```
member(x, [x | t]).
member(x, [y | t]) :- member(x, t).
```

这里的  $x$  表示我们要判断“是否在这个表中”的目标元素， $y$  表示表头元素，而  $t$  则表示切去表头后，表的剩余部分。显然，第一条规则表示：若目标元素正好是表头，则成功。第二条规则表示：若不是表头，就递归到尾表中继续找。

例如查询

```
?- member(c, [a, b, c]).
```

第一次尝试第一条规则失败，因为  $c \neq a$ ；于是转到第二条规则，把目标化为

```
member(c, [b, c])
```

再次失败后继续递归，最终在

```
member(c, [c])
```

时，第一条规则匹配成功，于是整个查询层层返回成功。

这个例子非常重要，因为它展示了 Prolog 程序的本质：规则本身既是知识描述，也是控制搜索的手段。一个看似极短的递归程序，其实背后已经包含了模式匹配、变量绑定、递归调用和回溯机制。

### 7.17 子句顺序、目标顺序与回溯

很多初学者以为，逻辑规则只要内容等价，写法顺序就无所谓。对纯逻辑而言，这种想法部分成立；但对 Prolog 来说，却往往不成立。因为 Prolog 的执行顺序是固定的，所以**子句顺序**与**目标顺序**都会影响程序表现。

例如祖先关系可写成：

```
predecessor(Parent, Child) :- parent(Parent, Child).
predecessor(Predecessor, Successor) :-
    parent(Predecessor, Child),
    predecessor(Child, Successor).
```

这种写法通常比较安全，因为先给出基例，再给出递归情况。若把顺序改成先递归、后基例，就可能让系统一开始就不断展开递归，而迟迟碰不到可终止的情况。再进一步，若递归体内部的目标顺序也写得<sub>不当</sub>，例如先递归调用 predecessor 再去查 parent，就更容易陷入无限回溯。

这说明一个很深刻的事实：**Prolog 看上去是在写逻辑，但实际上也在写控制结构**。程序员不仅要关心“规则是否正确”，还要关心“解释器会按什么顺序使用这些规则”。

### 7.18 九宫格马步搜索与路径问题

九宫格和路径搜索这类例子，正是在展示 Prolog 处理搜索问题的能力。在象棋中，马走的是“日字”。那么如何判断马能否走到某个位置呢？我们可以利用 Prolog 来实现。

设路径关系写成：

```

path(z, z, l).
path(x, y, l) :-
    move(x, z),
    not(member(z, l)),
    path(z, y, [z | l]).

```

在这里， $x$  代表当前点， $y$  代表终点， $z$  则代表下一个目标位置。而  $l$  最为关键，它代表已访问列表。如果不维护，那么无法避免死循环的出现。

上述 Prolog 语句中，第一条规则是基例：若起点和终点相同，则路径已经找到。第二条规则表示：若能从当前点  $x$  走到某个中间点  $z$ ，而且  $z$  尚未在已访问列表  $l$  中，那么就递归地从  $z$  继续走向终点，并把  $z$  加入访问记录中。而这里的  $[z|l]$ ，则表示：如果访问到了新的位置  $z$ ，则把它加入访问记录  $l$  中。

例如查询

```
?- path(1, 3, [1]).
```

系统会先尝试所有满足  $\text{move}(1, z)$  的候选。如果某一步走到的格子已经访问过，那么  $\text{not}(\text{member}(z, l))$  失败，系统立即回溯，换另一条路线继续试。于是，这个程序本质上就是一个**深度优先搜索 + 访问标记 + 回溯**的组合。

### 7.19 cut：控制搜索的剪枝

Prolog 中还有一个非常特殊的符号，叫做 **cut**，记作 **!**。它的作用不是增加新知识，而是**控制回溯范围**。一旦程序执行到 cut，系统就会承诺：到目前为止作出的某些选择不再回头尝试别的分支。

例如：

```
path2(x, y) :- move(x, z), move(z, y).
```

查询

```
?- path2(1, w).
```

可能给出多个答案，因为  $\text{move}(1, z)$  和  $\text{move}(z, y)$  的各种组合都会被回溯枚举。

而若写成

```
path3(x, y) :- move(x, z), !, move(z, y).
```

则在确定了第一个满足条件的  $z$  后，就不再回头考虑别的  $\text{move}(1, z)$  选择了。于是可得到的答案会更少，但搜索更受控制。

cut 的价值在于提高效率、限制无意义的回溯，但它也会破坏程序原本的纯逻辑含义。

### 7.20 农夫、狼、羊、菜与状态空间搜索

“农夫、狼、羊、菜过河”是 Prolog 教学中的经典例子。问题的约束是：船每次最多载两个对象，其中一个必须是农夫；若狼和羊单独留在一边，狼会吃羊；若羊和菜单单独留在一边，羊会吃菜。目标是找到一系列安全过河步骤。在小学奥数中我们就学过策略：农夫先和羊过河，然后回来再把狼接过去，然后再把羊带回来，随后再把菜带过去，最后再回来把羊接过去。



这类问题最自然的表示，是把一个局面写成状态，例如

```
state(Farmer, Wolf, Goat, Cabbage)
```

其中每个位置可以取东西岸两种值。如此我们就可以定义初始状态： $state(e, e, e, e)$  和目标状态  $state(w, w, w, w)$ 。而非法状态则可以表示为：比如  $state(w, e, e, w)$ ，此时农夫在西岸，而狼和羊单独留在东岸。这在底层的 move 规则里会被判定为非法，系统会直接将其拦截并触发回溯。

然后定义若干 move 规则，分别表示“农夫带狼”“农夫带羊”“农夫带菜”“农夫单独划船”四种动作，并在每一步都检查新状态是否安全。

随后，我们就可以写出一个通用的搜索过程了：

```
path(goal, goal, been_stack).
path(state, goal, been_stack) :-
    move(state, next_state),
    not(member_stack(next_state, been_stack)),
    stack(next_state, been_stack, new_been_stack),
    path(next_state, goal, new_been_stack), !.
```

在这里，path 中的三个参数分别代表当前状态、目标状态和已访问列表。第一句表示如果当前状态和目标状态相同，那么路径已经找到。而第二句则有 5 个要求被 And 起来，即必须同时满足：下一个状态需要符合 move 的规则，必须不能和之前重复；随后把这个状态压入已访问列表中，接着递归搜索下一个状态；最后，只要找到了一条路径，就立即 cut 方法返回。

这个结构和前面的马步搜索几乎是同一个思想：生成后继状态、检查合法性、避免重复访问、递归搜索、必要时回溯。最终得到的一串状态序列，就是问题的解答。

这个例子的意义非常大。它告诉我们，逻辑程序不只是会做公式推理和数据库查询，还可以把谜题、路径问题、状态转移问题都转化成“状态 + 合法移动 + 搜索控制”的程序结构。也就是说，Prolog 同时是一种逻辑推理工具，也是一种搜索编程语言。

本章真正想建立的，不只是对若干逻辑名词的记忆，而是一个贯穿到底的认识：**逻辑既是一种知识表示语言，也是一种推理机制，还可以在适当限制下直接变成程序执行方式。**

## 本章小练习

### 1. 判断题

若一个公式在某个解释下为真，那么它一定是有效式。（\_\_\_\_）

### 2. 化简题

将

$$\forall x(dog(x) \rightarrow animal(x))$$

化成适合归结使用的子句形式。

### 3. 简答题

请说明为什么在 Prolog 中，子句顺序和目标顺序都可能影响程序行为。

### 参考解答

1. 错。某个公式在某一个解释下为真，只说明这个解释满足它；只有当它在所有可能解释下都为真时，才能称为有效式。
2. 先消去蕴含：

$$\text{dog}(x) \rightarrow \text{animal}(x) \equiv \neg \text{dog}(x) \vee \text{animal}(x)$$

原公式是全称量化的。而在子句形式中，全称量词通常可以省略，因此最后得到的子句为

$$\neg \text{dog}(x) \vee \text{animal}(x)$$

3. 在 Prolog 中，解释器总是先处理最左边的目标，并且按照程序中从上到下的顺序尝试规则，因此不同的目标顺序和子句顺序会改变搜索展开方式。若先尝试一个递归规则而不是基例，程序可能陷入无限递归；若先处理某个会产生大量分支的目标，也可能导致回溯开销剧增。因此，虽然逻辑上等价的规则在数学意义上可能没有区别，但在 Prolog 的实际执行中，它们的顺序会直接影响搜索路径、效率，甚至是否能成功返回结果。

## 第八章 CLIPS 与基于规则的专家系统

### 本章概要

前一章讨论逻辑与 Prolog 时，我们已经看到：知识如果写成合适的逻辑形式，就可以被程序直接执行。到了这一章，视角会进一步转向更工程化的专家系统平台。CLIPS 是一个典型的**产生式系统**（production system）环境，它把“事实”“规则”“推理引擎”这些要素结合起来，使系统能够像一个规则驱动的专家系统那样运行。

这一章的重点，不是死记若干语法，而是弄清楚一个规则系统究竟是怎样工作的。什么是事实，什么是模板，什么是规则的左部和右部，推理引擎怎样在大量事实和规则之间做匹配，为什么 agenda 和优先级会影响系统行为，为什么模式顺序会显著影响效率，为什么模块化、控制模式和事实加载机制对大型系统很重要。只要把这些问题想清楚，CLIPS 就不再只是某种奇怪的语言，而会变成一个非常清晰的规则系统框架。

### 8.1 规则系统的基本结构

基于规则的专家系统通常由三个核心部分组成：**事实列表**（fact list）、**知识库**（knowledge base）和**推理引擎**（inference engine）。事实列表存放系统当前已知的具体信息，例如“某人年龄 21”“当前火警类型是 fire”“某个传感器读数是 125”；知识库存放一般规则，例如“如果紧急情况是火灾，则应当启动喷淋系统”；推理引擎负责检查当前事实是否满足某些规则的前提，并在满足时触发相应动作。

这种结构非常重要，因为它把“知识本身”和“推理过程”分开了。若知识发生变化，常常只需要修改事实或规则，而不需要整体重写程序控制流程。这也是专家系统长期以来很受重视的一个原因：它的知识库具有较强的可修改性和可解释性。

从整体风格上看，基于规则的系统往往具有几个鲜明特点。第一，它是**模块化的**，不同规则可以分别负责不同局部知识；第二，它通常具备一定的**解释能力**，因为系统的结论可以追溯到哪些规则被触发；第三，它在某种程度上模仿了人类专家“看到某种情况就应用某条经验规则”的认知方式。CLIPS 正是在这样的背景下发展起来的一个成熟平台。

## 8.2 CLIPS 是什么：从语法外形看规则系统

CLIPS 的名字来自 **C Language Integrated Production System**。从名字就能看出，它的核心并不是传统意义上的过程式编程，而是“产生式规则系统”。所谓产生式，可以理解为“如果满足某条件，就执行某动作”的结构，这和前面学过的产生式规则系统是一脉相承的。

CLIPS 的代码表面上看大量使用括号，因此初学时常会觉得它有点像 Lisp。事实上，这种外形确实有助于把知识写成非常规则的树形结构。它的记号习惯中，圆括号 ( ) 是最基本的界定符；尖括号 < > 常表示占位符；方括号 [ ] 表示可选成分；+ 表示至少一个；\* 表示零个或多个；竖线 | 表示可选其一。

CLIPS 提供了若干原始数据类型。最基本的包括整数、浮点数、符号和字符串。例如：

```
1
+3
-1
1.5
9e+1
fire
activate-sprinkler-system
"John Smith"
```

需要特别注意的是，符号和字符串不是一回事。符号更像未加引号的标识符，而字符串则必须用双引号括起来。变量通常以 ? 或 \$? 开头，因此这些符号不能随便拿来当普通标识符开头使用。在用途上，变量就是用于存储和操作数据的占位符；字符串则是不可变的纯文本数据；而符号则是 CLIPS 中最核心的数据类型，用来作为唯一的标识符、关键字或函数名。比如一个事实 (*animal duck*)，说明了鸭子是一种动物，那么这里的 animal 和 duck 就都是符号。

## 8.3 事实与模板：让知识有结构地存起来

在 CLIPS 中，最基础的知识单位就是**事实** (fact)。事实可以看成工作记忆中的一条记录。例如：

```
(person (name "John Smith") (age 21) (eye-color black) (hair-color black))
```

按照我们前面所学，这里的 person、name、age、eye-color、hair-color、black 都是符号。而 "John Smith" 是一个字符串，21 则是一个普通的数值。

这条事实表示一个具体的人，并给出了姓名、年龄、眼睛颜色和头发颜色。显然，这种写法比单纯一串无结构文本更适合让机器检索和匹配。

为了让事实结构更清楚，CLIPS 使用 **deftemplate** 定义模板。例如：

```
(deftemplate person
  "A person template" ; 可选的注释部分，补充说明
  (slot name)
  (slot age)
  (slot eye-color)
  (slot hair-color))
```

不难猜测，这里的分号；是 CLIPS 的注释符号。

模板可以理解成“事实的类型说明书”。它告诉系统：person 这种事实通常有哪些槽位 (slot)，每个槽位应该装什么类型的值。这样一来，事实不再是任意拼接的对象，而是受模板约束的数据结构。

这里要区分两类槽位。**单值槽位** (slot) 只容纳一个值，例如年龄；**多值槽位** (multislot) 可以容纳多个值，例如名字由多个部分组成，或者一个人有多个子女。比如：

```
(deftemplate number-list
  (multislot values) ; 定义 values 为多值槽位
)
```

就可以对应这样的事实：

```
(number-list (values 7 9 3 4 20))
```

#### 8.4 模板属性：类型、取值范围、基数与默认值

CLIPS 的模板系统之所以实用，是因为每个槽位还可以带各种**属性约束**。这些约束非常像数据库中的模式说明，它们能显著提高数据质量，也让规则匹配更加可靠。

第一类是**类型属性** (type)。例如：

```
(slot age (type INTEGER))
```

表示年龄必须是整数。CLIPS 支持的类型约束包括 SYMBOL、STRING、INTEGER、FLOAT、NUMBER、LEXEME 等。若不给出类型，系统默认允许更一般的变量形式。

第二类是**允许值属性** (allowed values)。例如：

```
(slot gender (allowed-values male female))
```

表示 gender 只能取 male 或 female。这在分类取值很有限的场景里非常有用。

第三类是**范围属性** (range)。例如：

```
(slot age (type INTEGER) (range 0 ?VARIABLE))
```

表示年龄至少为 0，上界不固定。范围约束特别适合数值监控和工程类事实。

第四类是**基数属性** (cardinality)，它只对 multislot 特别重要。例如：

```
(multislot name (type SYMBOL) (cardinality 1 6))
```

表示名字槽位里至少要有 1 个字段，最多有 6 个字段。

第五类是**默认值属性** (default)。例如：

```
(slot gender (allowed-values male female) (default female))
```

表示若没有显式给出 gender，系统会自动填入默认值 female。更一般地，CLIPS 还有 ?DERIVE 和 ?NONE 这样的机制，分别表示“按系统规则推导默认值”和“不提供默认值”。例如某个模板中，整数槽位的默认值可能自动变成 0，受限枚举槽位的默认值可能自动取第一个合法值。这种“自动补全”机制在实际知识工程里很方便。

因此，模板属性的意义在于：让事实不仅“有结构”，而且“结构有约束”。一旦系统规模变大，这种约束会极大减少错误传播。

## 8.5 有序事实与显式模板

除了带命名槽位的模板事实，CLIPS 还支持**有序事实** (ordered facts)。例如：

```
(number-list 7 9 3 4 20)
```

这种事实没有显式写出槽名，而是按位置组织内容。可以把它理解成一个更轻量的写法。其背后可视为隐含了某种模板结构，所以有时也称之为**隐式模板** (implied deftemplate) 创建的事实。其与前面的区别就在于，不需要用 deftemplate 提前声明，直接在系统里写下这句话，CLIPS 就会直接自动生成一个隐式的模板来兼容它。

与之对应，前面那种基于 deftemplate 明确定义槽位而创建出来的事实，属于更严格的显式模板事实。两种方式都能用，但当知识结构比较复杂、需要长期维护或需要很多规则共享时，显式模板通常更清晰、更安全。此外，我们也不难注意到，隐式模板在写具体事实时，强制不能写槽位名，只能纯写数据，这些数据就相当于“按位置死板排列的符号”。比如

```
(person-info "John Smith" age 21 male)
```

这种写法是比较危险的。因为如果不小心把位置写反了，系统不会报错，但数据就全乱了。

如果把整个关系画出来，可以得到一张很典型的层次图：DEFTEMPLATE 是一种构造；FACT 是具体实例；FACT 可以由显式模板创建，也可以由隐式模板创建；槽位和字段共同构成事实内容。

## 8.6 事实操作：添加、显示、删除与修改

一个规则系统如果只能声明事实而不能更新事实，就很难真实运行起来。因此 CLIPS 提供了若干最基本的事实操作。

添加事实使用 assert：

```
(assert (emergency (type fire)))
```

显示事实可以用 facts，删除事实可用 retract，修改事实则常用 modify。例如：

```
(modify <fact-index> (address "37 Cherry Lane"))
```

表示在原事实基础上改写某个槽位内容。这里的 <fact-index> 翻译过来叫事实 ID。当你在 CLIPS 中输入或产生一条事实时，系统会自动给它分配一个以 f- 开头的独一无二的编号，如 f-0、f-1 等。同一种模板在内存中可以同时存在成百上千条事实，如果没有编号，那么就不知道要修改哪个。

要注意一点：规则系统中的“修改”通常不是原地直接操作，而是以工作记忆中的事实为对象进行受控更新。所谓受控更新，可以理解成系统实际上连续偷偷做了两件事：先撤销，把旧的 f-1 事实从内存中彻底删掉；然后再做新的断言，即把修改后的数据，当成一条全新的事实重新塞进内存，并分配一个新的编号，如 f-4。之所以要做“先删再建”这么复杂的受控更新，就是为了让工作记忆发生一次剧烈震荡（少了一条事实，多了一条事实），从而才能重新激活所有相关规则，促使系统去检查有没有新的规则被触发，或者原本成立的规则现在是不是该失效了，引发整个规则库的连锁反应。

另外，CLIPS 还支持 defacts，用于批量声明一组初始事实。例如：

```
(defacts people
  "Some people we know"
  (person (name "Sean Smith") (age 21) (eye-color black) (hair-color black))
  (person (name "John Riley") (age 24) (eye-color blue) (hair-color black))
  (person (name "Bruce Lenny") (age 35) (eye-color blue) (hair-color blond)))
```

这相当于把一批初始工作记忆一次性装进系统里，非常适合示例系统和测试环境。

## 8.7 规则：左部匹配，右部动作

在最开始我们就提到：基于规则的专家系统需要包含事实列表、知识库和推理引擎这三个部分。前面我们说了事实的储存和修改方法，这里我们再来看看知识库，也就是规则。

CLIPS 中最核心的构造之一是 **defrule**。一条规则通常由左右两部分组成：左部 (LHS) 写出触发条件，右部 (RHS) 写出触发后的动作。基本语法是：

```
(defrule <rule-name>
  <patterns>*
  =>
  <actions>*)
```

例如最经典的火灾应对规则可以写成：

```
(deftemplate emergency (slot type))
(deftemplate response (slot action))

(defrule fire-emergency
  (emergency (type fire))
  =>
  (assert (response (action activate-sprinkler-system))))
```

这条规则的意思非常直接：如果工作记忆中存在 (emergency (type fire)) 这一事实，那么就断言一条新的响应事实 (response (action activate-sprinkler-system))。

因此，规则系统的运行方式和普通程序很不一样。普通程序通常是“按写好的顺序一步步执行语句”；而规则系统更像是在等待条件被满足，一旦满足就触发相应行为。这也是为什么 CLIPS 特别适合事件响应、故障诊断、监控、配置与分类这类问题。

如果想真正启动推理引擎，则常用

```
(run)
```

或带上步数限制的

```
(run <limit>)
```

这表示让系统开始从 agenda，即当前所有被激活的规则集合中，按照优先级来依次执行。

## 8.8 变量、事实地址与规则中的绑定

规则若想写得灵活，就不能把所有常量都写死，因此必须使用**变量**。CLIPS 中普通变量以 ?name 形式出现。例如：

```
(defrule find-blue-eyes
  (person (name ?name) (eyes blue))
  =>
  (printout t ?name " has blue eyes." crlf)) ; 这里的 crlf 是 CLIPS 的换行符号
```

这里 ?name 会在匹配时绑定成具体姓名，规则右部则把这个值打印出来。

变量还可以在多条模式之间共享。例如：

```
(defrule find-eyes
  (find (eyes ?eyes))
  (person (name ?name) (eyes ?eyes))
  =>
  (printout t ?name " has " ?eyes " eyes." crlf))
```

这表示：先从一条 find 事实里取出想查的眼睛颜色，再去找所有拥有同色眼睛的人。共享变量 ?eyes 就起到了“把两条事实对上”的作用。

除了绑定值，CLIPS 还允许绑定**事实地址**。例如：

```
?f1 <- (moved (name ?name) (address ?address)) ; 表示某人搬家到了某个地址
?f2 <- (person (name ?name))
```

这里 ?f1 和 ?f2 不是普通值，而是匹配到的那两条事实本身的地址。于是右部就可以写：

```
(retract ?f1)
(modify ?f2 (address ?address))
```

为什么需要 (retract ?f1) 呢？很显然，当一个人已经搬家后，这个事实就不再需要了。而我们已经通过前面的 ?f2 <- (person (name ?name)) 找到了“这个人”所在的事实，因此接下来就可以直接把这个事实中，他的“地址”属性更新为新的地址，即 (modify ?f2 (address ?address))。

这在“旧信息到来后更新已有记录”这类任务中非常常见。它说明 CLIPS 规则不只是做判断，还能对工作记忆进行精细操作。

## 8.9 单字段通配、多字段通配与未指定槽位

我们经常想要查找一些“符合特定要求”的事实，那么就需要去匹配。模式匹配系统若想足够灵活，就必须支持“有些位置我只关心部分内容”。比方说，我们想要查找一个人的身份证号，但只知道他的 last-name，而不知道 first-name。此时，就需要通配机制的参与。

必须注意的是，这里带问号 ? 和 \$? 的通配语句，已经不再是前面所说的事实了，而是只能用于规则左部的“模式”，也就是用来筛选事实的“网眼”。事实永远只能是确定的。

最简单的是**单字段通配符** ?。例如：

```
(person (name ? ? ?last-name) (identity-card-number ?id-number))
```

表示姓名槽位中前两个字段不关心，只关心最后一个字段是 ?last-name。如此便可查到他的身份证号。这种写法适合处理固定长度但只关心部分字段的情况。

若某个槽位根本没写出来，CLIPS 往往将其视为“任意值都可”。例如：

```
(person (name John Q. Smith))
```

在匹配意义上，可以理解为等价于

```
(person (name John Q. Smith) (identity-card-number ?))
```

也就是说，未指定槽位默认不构成限制条件。

更强大的是**多字段通配符** \$?。例如：

```
(person (name $?name) (children $?before ?child $?after))
```

表示在 children 这个多值槽位中，系统会寻找某个子女 ?child，并把它前面的部分记为 \$?before，后面的部分记为 \$?after。例如对 (children Jane Joe Paul) 查询 Joe 时，就会得到 before = (Jane)，after = (Paul)。

这种机制非常灵活，但也意味着匹配组合会迅速增多。后面讲效率时会再次看到，多字段通配符用得越多，搜索空间通常越大。

## 8.10 约束：否定、选择、合取与测试

CLIPS 的模式不只是“把值取出来”，还可以在模式内部直接写各种约束。

例如，用 ~ 表示**否定约束**：

```
(hair ~black)
```

表示头发不是黑色。

用 | 表示**析取约束**：

```
(hair brown|black)
```

表示头发是棕色或黑色。

用 & 则可以把变量绑定与约束组合起来：



```
(hair ?color&brown|black)
```

表示既要把颜色绑定成变量 ?color，又要求它只能是棕色或黑色。

还可以配合谓词测试函数写出更复杂限制，例如：

```
(age ?age&:(> ?age 18)) ; 冒号表示谓词约束，后接一个需要被执行和求值的表达式
```

这相当于模式匹配和数值条件检查同时进行。与单独写成

```
(age ?age)
(test (> ?age 18)) ; test 是特殊用法，它是 CLIPS 中的关键字，只有表达式为真时，才允许匹配
```

相比，这种内嵌约束往往更紧凑，也更利于高效匹配。

类似地，对于复杂逻辑条件，还可以使用 and、or、not、test 等条件元素。例如：

```
(or (emergency (type flood))
    (extinguisher-system (type water-sprinkler) (status on)))
```

表示满足其中任一条件即可。

这种内嵌约束机制，是 CLIPS 模式语言非常强大的地方。它让规则左部不再只是“事实模板的列表”，而是可以写成带逻辑组合和数值约束的匹配条件。

### 8.11 条件元素：or、and、not、exists、forall、logical

在 CLIPS 中，规则左部可以使用若干专门的**条件元素** (conditional elements)，它们构成了非常重要的一套表达能力。

or 表示多个前提中只要有一个成立即可。例如“若发生洪水，或洒水灭火系统已打开，则关闭电力”就可以把原本两条相似规则合成一条。

and 表示同时满足多个条件。虽然多个普通模式连写本身就有合取意味，但显式 and 在嵌套结构中更清楚。例如“若是 C 类火灾，并且电源已关闭，则使用二氧化碳灭火器”。

not 表示某种事实不存在。例如：

```
(not (number ?y&:(> ?y ?x)))
```

配合

```
(number ?x)
```

就可以找到“没有任何数比它更大”的那个数，于是规则能识别出最大值。这是一个很典型的“通过不存在的反例来刻画极值”的写法。

exists 表示“至少存在一个满足条件的事实”。若不用 exists，某些规则可能会因每个匹配事实都各自触发而执行多次；加入 exists 后，就能把语义改成“只要存在即可”。例如“系统中只要存在某个 emergency，就向操作员发一次警报”，这时 exists 就非常自然。

forall 表示“对所有满足第一部分条件的事实，后续条件都成立”。例如：

```
(forall (emergency (type fire) (location ?where))
  (fire-squad (location ?where))
  (evacuated (building ?where)))
```

其含义是：对每个着火地点，都必须同时有消防队在场并且建筑已疏散。forall 的语义很强，它是在规则左部直接表达一种全称条件。

logical 则和依赖管理有关。若一个结论事实是通过某些前提“逻辑支持”得来的，那么当前提消失时，这个结论也应自动消失。logical 就是为了表达这种依赖关系。例如氧气面罩的使用可能依赖“有毒烟雾存在”和“气体灭火器正在使用”，若这两个条件后来不再成立，逻辑支持下产生的结论也应被撤销。例如：

```
(logical (hazard smoke)          ; 前提 1: 存在有毒烟雾
  (extinguisher gas))           ; 前提 2: 气体灭火器正在使用
```

如果不用 logical，那么当“有毒烟雾存在”或“气体灭火器正在使用”不再成立时，逻辑支持下产生的结论不会自动撤销。那么，我们可能就只能憋屈地再写一条相反的规则：“如果发现面罩在外面，但是没有烟雾或者没有灭火器正在使用，那么撤销面罩事实。”当业务逻辑非常复杂时，写这种反向清理规则会让人崩溃，而且极易漏掉边界条件导致系统逻辑混乱。

因此，这些条件元素并不只是“语法花样”，而是在让规则系统表达存在、全称、缺失条件和依赖关系这些非常核心的知识结构。

## 8.12 规则系统中的决策树程序

下面来看一个很典型的例子：用 CLIPS 写一个**基于规则的决策树程序**。这个例子特别有价值，因为它展示了规则系统不只是静态匹配事实，还能和用户交互、逐步推进流程，甚至在回答错误时进行学习。

先定义一个节点模板：

```
(deftemplate node
  (slot name) ; 节点的名字，比如 node1、node2 等
  (slot type) ; 是决策节点还是叶子节点
  (slot question) ; 如果是决策节点，则提问；叶子节点则为 nil
  (slot yes-node) ; 如果用户回答 yes，进入的下一个节点
  (slot no-node) ; 如果用户回答 no，进入的下一个节点
  (slot answer) ; 如果是叶子节点，则给出猜测结果；决策节点则为 nil
)
```

然后把决策树中的每个节点都表示成事实。比如根节点问“这个动物是温血的吗？”，回答 yes 就进入 node1，回答 no 就得到答案 snake；在 node1 再问“这个动物会打呼噜吗？”，回答 yes 得到 cat，回答 no 得到 dog。这样，一棵树就被翻译成了一组结构化节点事实。

接下来，系统会用规则驱动整个问答流程。若当前节点是一个决策节点，就提问并读取 yes/no；若用户输入非法答案，就由 bad-answer 规则撤销该答案并重新要求输入；若当前节点是叶子节点，就给出猜测结果，并再问“我猜对了吗？”。

真正有意思的地方在于：若系统猜错了，它不会简单报错退出，而是通过 replace-answer-node 等规则要求用户告诉它正确答案，并让用户提供一个新的区分性问题。然后再用 gensym\* 生成新节点名，把原本的答案节点修改成一个新的

的决策节点，同时挂上两个新的答案节点。这样一来，系统就在运行过程中“学会了”一个新的分类分支。

这个例子非常值得重视，因为它把“规则系统 + 事实更新 + 人机交互 + 简单增量学习”几件事结合到了一起。也就是说，CLIPS 不只是会响应已有知识，还能在一定程度上扩展自己的知识结构。

### 8.13 推理引擎、agenda 与冲突消解

现在，我们已经了解了基于规则的专家系统中的事实列表和知识库，接下来，我们再来看一看第三大模块——推理引擎。

规则系统真正运行起来时，并不是所有规则都会立刻执行。首先，推理引擎要检查哪些规则左部已经被当前事实满足；这些可执行的规则实例被称为**激活项** (activation)，它们会进入一个待处理队列，通常称为 **agenda**。换句话说，agenda 是特指那些左部已经被当前事实满足的规则实例。

推理引擎的大致过程可以概括为一个循环：

1. 先做 **Match**：检查规则左部与当前事实是否匹配，更新 agenda。最开始 agenda 中为空，而 Match 的意义就在于把现在已经满足要求、能被执行的规则加入到其中。
2. 再做 **Conflict Resolution**：若 agenda 中有多个可执行规则，根据优先级挑选一个。
3. 接着 **Act**：执行被选中规则右部的动作。
4. 然后再次匹配，因为事实可能已经被修改。这是因为上一条规则执行所得到的结论，可能会导致当前 agenda 中的某些规则的条件失效，要被踢出；或者是有新的规则要被加入 agenda。
5. 若执行了 halt 或没有规则可执行，则停止。

这说明规则系统是一个持续的 **匹配 - 选择 - 执行 - 再匹配** 循环。只要工作记忆在变，agenda 就会跟着变，系统的下一步行为也会变化。在基于规则的系统中，真正控制整体运行的，不只是知识库本身，还有 agenda 的管理与冲突消解策略。

### 8.14 salience、控制模式与模块化

当多个规则同时可触发时，就会出现“到底先执行哪一条”的问题。CLIPS 提供了 salience 机制来给规则设定优先级。取值范围大致在 -10000 到 10000 之间，数值必须为整数。如果未明确声明，系统默认为所有规则设定的优先级为 0。而例如：

```
(declare (salience 10))
```

就表示该规则优先级为 10，比默认优先级 0 的规则更早考虑。

然而，优先级虽然有用，却很容易被**滥用**。例如若系统里有三条规则分别处理 emergency、important、normal 三种情况，单纯靠高低不同的 salience 去强压执行顺序，往往会让规则之间出现隐含耦合：表面上每条规则都简单，实际上它们是否能正常工作取决于别人有没有先跑、跑了几次。这种写法维护起来会很痛苦。

更好的办法，通常是把控制逻辑显式写出来。例如在普通规则左部直接加上：

```
(not (situation emergency))
```

也就是在普通和重要（非紧急）规则的左部，显式地加上对紧急状态的否定。这保证了如果处在紧急状态，那么这些普通和重要规则根本都不会被加入到 agenda 中。

或使用专门的**控制模式** (control pattern)，给系统加入阶段事实。这使得在每个阶段，只有符合要求的规则被加入 agenda，从而也避免了混杂。当高优先级阶段的所有事情都干完了，agenda 里没有相关的规则了，一条接力规则才会触发，负责更新当前状态到低优先级的阶段。比如：

```
(defrule 诊断故障
  (phase detection) ; 只有处于检测阶段，这条规则才生效
  => ... 处理诊断...
)

(defrule 隔离故障
  (phase isolation) ; 只有处于隔离阶段，这条规则才生效
  => ... 处理隔离...
)

(defrule 切换到隔离阶段
  ?p <- (phase detection) ; 现在处于旧阶段
  =>
  (retract ?p)
  (assert (phase isolation)) ; 把系统推向下一个阶段
  (printout t "--- 检测完毕，现在进入隔离阶段 ---" crlf)
)
```

然后让不同规则只在对应阶段触发。这样一来，系统的控制逻辑就不再藏在 salience 数字里，而是显式地体现在工作记忆和规则条件中，可读性和可维护性都会显著提高。

进一步地，大型系统还会用 **defmodule** 做模块化。可以建立 DETECTION、ISOLATION、RECOVERY 等模块，把不同阶段的规则分开放置，再利用 focus 控制当前激活哪个模块。例如现在需要聚焦于 DETECTION，那么就执行：

```
(focus DETECTION)
```

模块之间还能通过 export 和 import 共享模板。例如 RECOVERY 模块可以导入 DETECTION 模块的 fault 模板和 ISOLATION 模块的 possible-failure 模板。

```
(deftemplate fault ...) ; 写在 `DETECTION` 模块中
(export deftemplate fault) ; 允许其他模块导入
```

```
(deftemplate possible-failure ...) ; 写在 `ISOLATION` 模块中
(export deftemplate possible-failure) ; 允许其他模块导入
```

```
(import DETECTION deftemplate fault) ; 写在 `RECOVERY` 模块中
(import ISOLATION deftemplate possible-failure)
```

因此，这一部分真正想说明的是：规则系统不只是一要“会推理”，还要“能被组织”。没有控制模式和模块化，大型知识库很快就会变得难以维护。

### 8.15 监控问题：一个完整规则系统的拼装方式

在本章，我们最值得细看的例子，是一个完整的**监控系统**。这个例子几乎把前面所有内容都串起来了。

系统中有若干设备 D1 到 D4，以及与之相连的一组传感器 S1 到 S6。每个传感器都有低红线、低警戒线、高警戒线和高红线等阈值。例如，若读数小于等于低红线，或大于等于高红线，就应立即停机；若读数进入警戒区，则要发警告，若持续若干周期仍未恢复，则也应停机。

为了实现这一系统，需要先在 MAIN 模块中定义设备和传感器模板，并建立初始事实。设备模板记录设备名称和开关状态；传感器模板记录所属设备、当前原始读数、当前状态以及各种阈值。再用 defacts 录入设备初始状态和各传感器阈值信息。

接着要解决输入问题。系统中通过 INPUT 模块读取传感器数据。这里的数据源可以是事实文件，每个周期把下一批原始数值依次填到各个传感器的 raw-value 槽位中。若数据读完了，系统就会打印信息并 halt。

然后，在 TRENDS 模块中，系统根据每个传感器的 raw-value 与阈值关系，把当前状态判断为 normal、high-guard-line、high-red-line、low-guard-line 或 low-red-line。这一部分实际上就是一个基于规则的状态分类器。

仅仅知道当前状态还不够，还要知道状态有没有持续。于是系统再维护一种 sensor-trend 事实，记录某传感器当前连续处于某状态的起始周期和结束周期。若某个周期里状态没变，就更新结束时间；若状态改变，就重置开始和结束时间。

最后，在 WARNINGS 模块中，根据 sensor-trend 和设备状态决定是否发警告或停机。若传感器落入红区，则立刻关闭相应设备；若传感器落入警戒区但持续时间尚未达到要求，只给出警告；若持续周期达到阈值，就执行停机。

这个例子非常能体现规则系统的风格。它不是把全部逻辑塞进一个大函数，而是拆成了设备事实、传感器事实、输入模块、趋势模块、警告模块、周期控制模块等多个部分。每个部分各做一件事，最后通过事实和规则自然联动起来。这正是专家系统工程化实现的典型方式。

### 8.16 模式匹配网络

规则系统之所以好用，靠的正是自动模式匹配；但规则系统也可能因为模式匹配过多而变得非常慢。在这里，我们可以把 **pattern network / joint network** 理解成一种把单个模式检查与多个模式联结统一组织起来的匹配结构，它正是理解效率问题的关键。

可以这样理解：当规则左部有多个模式时，系统不是简单地“一个规则从头扫描到尾”，而是会把不同模式拆进一张共享的匹配网络里。某些节点负责检查单个事实的局部条件，例如“x 槽位不是绿色”“b 槽位等于红色”；而更高层的联合节点则负责把多个模式之间共享变量的约束合起来，例如“第一条模式里的 a 要等于第二条模式里的 y”。当多个规则共享相似前缀时，这种网络结构能够复用中间结果，避免重复计算。

这类思想与 Rete 网络很接近。即使不去深究底层实现，也一定要明白：规则系统的高效并不是白来的，而是靠精细的模式共享和部分匹配缓存实现的。

### 8.17 模式顺序

在 CLIPS 中，**模式顺序** 是一个非常关键的效率问题。规则左部并不是随便排排都一样。常见的 good-match 与 bad-match 对比，正是为了说明这一点。

在 good-match 规则里，最前面先匹配

```
(find-match ?x ?y ?z ?w)
```

这条事实非常具体，它立刻把四个变量都绑定住了。在事实列表中，可能只有很少的几个 find-match 事实。因此，后面再匹配 (item ?x)、(item ?y)、(item ?z)、(item ?w) 时，搜索空间已经被大大缩小，所以只会产生很少的部分匹配组合。

而在 bad-match 规则里，前面先写了四个

```
(item ?x)
```

```
(item ?y)
```

```
(item ?z)
```

```
(item ?w)
```

它们都很宽松，于是系统会先产生海量候选组合所有的 x、y、z、w，直到最后才拿 (find-match ?x ?y ?z ?w)，即“必须要在 find-match 中存在对应的变量”，来筛选。这样会导致部分匹配数呈爆炸式增长，甚至可能耗尽内存。

因此，有几条非常重要的效率经验需要牢牢记住：

第一，**越具体的模式越应该靠前。**

第二，**匹配事实数最少的模式应尽量靠前。**

第三，**变化频繁的事实尽量靠后**，因为它们一变就会使大量匹配失效。

第四，**尽量减少多字段通配符和变量数量**，因为它们会显著扩大匹配组合。

第五，**测试条件应尽量早放**，越早淘汰无效候选越省计算。

第六，能用内建约束表达的，就尽量不要改写成更笨重的等价结构。

这些原则不是琐碎优化，而是规则系统写大以后能不能跑得动的关键。

## 8.18 多字段通配与匹配爆炸

多字段通配符 \$? 特别方便，但也特别危险。比如模式

```
(list (items $?a $?b $?c))
```

若去匹配事实

```
(list (items x 5 y 9))
```

系统可以有大量不同拆分方式：\$?a 取 x、\$?b 取 5、\$?c 取 y 9；或者 \$?a 取 x 5、\$?b 取 y、\$?c 取 9；还可以让某些多字段变量匹配空列表。于是短短四个元素，就能产生十几种匹配尝试。这说明，规则系统的搜索复杂度有时并不来自事实数量本身，而来自模式允许的组合自由度。多字段通配符越多、变量越松，系统需要枚举的可能拆分就越多。因此，在实际设计中，若不是确实需要，千万不要滥用 \$?。

## 8.19 test 条件与内建约束的效率

当规则中需要额外检查某些关系时，test 条件元素非常有用。例如，要找三个彼此不同的点，可以写成：

```
(defrule three-distinct-points
  ?point-1 <- (point (x ?x1) (y ?y1))
  ?point-2 <- (point (x ?x2) (y ?y2))
  ?point-3 <- (point (x ?x3) (y ?y3))
  (test (and (neq ?point-1 ?point-2)
              (neq ?point-2 ?point-3)
              (neq ?point-1 ?point-3)))

  =>
  ...)
```

但若把全部 test 都放到最后，系统会先把大量三元组组合出来，再统一做筛选，效率可能不高。更好的做法是把一部分 test 提前放到模式之间。例如在匹配第二个点之后，立刻检查它和第一个点是否相同；匹配第三个点后，再检查它和前两个点是否不同。这样就能更早剪掉无效组合。

类似地，若某个条件本来可以直接写成连接约束，就尽量不要写成更复杂的谓词测试。例如：

```
(color ?x&red|green|blue)
```

通常就比

```
(color ?x&:(or (eq ?x red) (eq ?x green) (eq ?x blue)))
```

更简洁，也更容易被高效处理。因为前者直接利用了 CLIPS 的内建模式约束机制。

## 8.20 initial-fact、过程函数与交互控制

除了事实、规则和模板，CLIPS 还提供了一些特别实用的辅助机制。

其中一个 **initial-fact**。我们知道，CLIPS 规则是靠事实来触发的。如果系统一启动，内存里空空如也，那谁来注入第一条事实、触发第一条规则呢？很多时候，我们想在系统刚启动时打印一句欢迎词，或者初始化一些基础数据，此时规则左部不需要任何业务条件。像这些没有显式前提的规则，或者以 not 开头的规则，在系统启动时往往依赖一个特殊初始事实来触发，才能使得那些“看上去左部为空”的规则在系统一启动就运行。它就是 (initial-fact)。

另一个是**过程函数**，例如 if 和 while。例如：

```
(if <predicate-expression> then <expression>+ [else <expression>+]) ; 如果满足这个谓词表达式，就执行
```

以及

```
(while <predicate-expression> [do] <expression>+)
```

这说明 CLIPS 虽然本质上是规则系统，但并不排斥在规则右部写少量过程式控制。典型例子就是询问用户是否继续，若输入既不是 yes 也不是 no，就通过 while 一直要求重新输入。这类机制让 CLIPS 不只是“被动推理”，还能够一定程度上完成交互流程控制。

本章真正要建立的，是一种比纯逻辑程序更贴近工程系统的理解：**一个基于规则的专家系统，不只是若干 if-then 句子的堆积，而是由事实、模板、规则、agenda、模式匹配网络、控制模式和模块化机制共同构成的运行系统。**

模板决定了事实长什么样，事实构成工作记忆，规则描述在什么条件下该做什么动作，推理引擎通过匹配和冲突消解驱动系统前进，agenda 组织了当前可执行的规则实例，模块和控制模式负责把大系统拆成可维护的阶段结构，而各种模式约束、通配符和 test 条件又决定了系统运行时究竟快还是慢。

因此，学习 CLIPS 时最值得掌握的，不只是若干具体命令，而是三层理解。第一层，能看懂并书写模板、事实和规则。第二层，能理解系统怎样通过匹配和 agenda 运行起来。第三层，能意识到规则系统有非常强的工程性，写法的优劣会直接决定系统的可维护性与效率。

## 本章小练习

### 1. 判断题

在 CLIPS 中，规则左部的模式顺序通常不会影响程序运行效率，因为只要逻辑上等价，系统总能自动优化到一样的程度。（\_\_\_\_\_）

### 2. 填空题

若一个槽位只能容纳一个值，应使用 \_\_\_\_\_；若一个槽位可以容纳多个值，应使用 \_\_\_\_\_。

### 3. 简答题

请说明为什么大型规则系统不应过度依赖 salience 来组织整体流程，并说明使用阶段事实或模块化控制的好处。

## 参考解答

1. 错。虽然逻辑意义上某些模式重排后可能等价，但在实际执行中，模式顺序会显著影响部分匹配数量和搜索规模。若把宽松、匹配面很大的模式放在前面，往往会造成组合爆炸，从而严重拖慢系统，甚至耗尽内存。
2. 若一个槽位只能容纳一个值，应使用 slot；若一个槽位可以容纳多个值，应使用 multislot。
3. 过度依赖 salience 的问题在于：系统控制逻辑会被隐藏在一堆优先级数字之中，规则之间形成隐蔽耦合，后续维护和修改都很困难。相比之下，若使用阶段事实，如 phase detection、phase isolation、phase recovery，或者使用 defmodule 把不同功能拆开，那么系统控制逻辑会显式出现在工作记忆与模块结构中，更容易理解、调试和扩展。

---

## 第九章 强化学习

### 本章概要

前面几章中，我们已经见过多种人工智能方法。有些方法依赖显式规则，有些方法依赖样本数据，还有一些方法强调搜索与优化。**强化学习** (reinforcement learning, RL) 则把注意力集中到另一类问题上：一个智能体处在环境中，必须一边行动，一边观察后果，并通过奖赏或惩罚逐步学会怎样做得更好。这里没有人手把手告诉它每一步的标准答案，系统只能根据长期效果不断修正自己的行为。

这一章要解决的核心问题是：什么叫状态、动作、奖赏与策略；为什么强化学习特别强调“长期回报”而不是眼前得失；贝尔曼方程怎样把“当前决策”和“未来收益”联系起来；多臂老虎机、蒙特卡洛方法、时序差分学习、Sarsa、



Q-learning、策略梯度、资格迹等方法分别在解决什么问题；以及深度神经网络与强化学习结合后，为什么会产生更强的决策能力。读完后，读者应当能够把强化学习看成一条清晰的方法链，而不是若干彼此割裂的算法名称。

## 9.1 从机器学习的整体图景看强化学习

在机器学习中，最常见的两类任务是监督学习与无监督学习。监督学习有带标签的数据，系统要做的是从输入到输出之间建立映射，例如根据样本判断图片属于哪一类；无监督学习没有标签，系统更像是在数据中寻找结构，例如聚类、降维或发现潜在模式。强化学习与这两者都不同，因为它研究的是**目标导向的行动者如何在不确定环境中学习决策**。

这一区别看上去只是表述不同，实则非常深刻。有监督学习默认“正确答案”已经给出，模型只需尽量逼近它；强化学习中却往往没有现成答案，系统必须通过一次次尝试，自己摸索“怎样的行为序列最终更有利”。而无监督学习虽然也没有预先标注的“标准答案”，但它旨在理解数据结构、发现隐藏的模式或规律，而强化学习却旨在指导行动的策略，通过反复试错来最大化长期累计奖励。

从中我们就可以看出，强化学习不是单纯地学一个静态映射，而是在学一种**行动策略**。

还要注意，强化学习通常不是只关心单步选择，而是关心连续交互。一个动作的好坏，常常并不只看它眼前得到的奖赏，还要看它会不会把系统带到更有利或更糟糕的后续状态。也正因为如此，强化学习的核心不只是“选动作”，而是“在长期回报意义下选动作”。

若用一个直观例子来理解，可以想象一只老鼠在迷宫中寻找出口。它会观察当前位置，选择向左、向右、向前还是向后；环境则根据它的动作改变位置，可能给出食物，也可能让它撞墙。老鼠并不知道最优路线，但如果系统能够根据成功与失败逐渐调整行为，那么它就表现出了一种典型的强化学习过程。这类问题往往同时包含内部状态、外部环境、观察信息、动作选择与奖赏反馈，是强化学习最自然的应用场景。

## 9.2 强化学习的基本框架

强化学习最基本的交互结构可以概括成一个循环：在时刻  $t$ ，智能体观察到环境状态  $S_t$ ，根据某种策略选择动作  $A_t$ ，环境在下一时刻反馈一个奖赏  $R_{t+1}$ ，并转移到新状态  $S_{t+1}$ 。随后系统再次根据新状态作出决策。这样就形成了

$$S_0, A_0, R_1, S_1, A_1, R_2, S_2, A_2, \dots$$

这样的交互序列，也常被称为一条**轨迹** (trajectory) 或一个**回合** (episode) 中的经历。

在这一框架中，有几个符号几乎处处都会出现。通常用  $S$  表示状态集合，用  $A$  表示动作集合，用  $R$  表示可能奖赏的集合。若记策略为  $\pi$ ，那么

$$\pi(a \mid s)$$

表示在状态  $s$  下采取动作  $a$  的概率。这里必须特别理解：**策略就是行为规则**。它回答的不是“某个状态值是多少”，而是“在这个状态下该怎么做”。

除了策略之外，强化学习还特别关心**价值**。如果把状态  $s$  在策略  $\pi$  下的价值记为  $v_\pi(s)$ ，那么它表示：系统从状态  $s$  出发，今后按照策略  $\pi$  行动时，平均能得到多大的长期回报。若把状态动作价值记为  $q_\pi(s, a)$ ，那么它表示：先在状态  $s$

采取动作  $a$ ，之后继续按照  $\pi$  行动时，平均能得到多大的长期回报。

你可能会问：既然我们都有一个策略  $\pi$  了，老老实实用  $v_\pi(s)$  不就行了，为什么非要搞出一个允许第一步“乱来”的  $q_\pi(s, a)$  呢？这很重要，因为系统想要进化，寻找更好的出路。正是后者允许系统去假设和假设：要是我今天不按常理出牌，试着去做个动作  $a$ ，而之后依然保持我的老习惯，平均价值会变好还是变坏？

因此，策略和价值分工很明确。策略管“怎么做”，价值管“这样做长期值不值”。许多强化学习算法本质上都在做两件事中的一种，或者同时做两件事：要么估计价值，要么改进策略。

### 9.3 奖赏、回报与折扣因子

初学强化学习时，一个很容易混淆的地方是：**奖赏** (reward) 与**回报** (return) 并不是同一个东西。奖赏通常是环境在一步之后立刻给出的反馈，例如走对一步得到 0 分，撞墙得到 -1 分，到达目标得到 +10 分。回报则是从当前时刻开始，后续奖赏的累积。

若任务是有限回合的，并且终止时刻为  $T$ ，那么从时刻  $t$  开始的回报可以写为

$$G_t = R_{t+1} + R_{t+2} + \cdots + R_T$$

但在很多问题中，更常见的是带折扣的回报：

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

这里， $\gamma$  称为**折扣因子** (discount factor)，满足  $0 \leq \gamma < 1$ 。它的作用是把更远未来的奖赏适当减弱。这样做至少有三层意义。第一，现实中“越早得到回报通常越有价值”，折扣正好体现这一点。第二，若任务可能无限持续，折扣有助于让无穷和收敛。第三，折扣也反映了系统对未来不确定性的谨慎程度。

从直觉上看，若  $\gamma$  很接近 0，系统就更短视，几乎只在乎眼前奖赏；若  $\gamma$  很接近 1，系统则更重视长期影响。也就是说，折扣因子其实在决定“智能体到底有多看重未来”。

### 9.4 多臂老虎机问题与探索利用矛盾

在进入一般强化学习之前，先看一个比完整环境更简单、但极具代表性的问题：**多臂老虎机问题** (multi-armed bandit problem)。可以把它想成一排老虎机，每一台拉杆机背后都有某种未知的平均收益。问题是：在有限尝试次数下，应当怎样选择拉哪一台，才能让总收益尽可能大。

这个问题之所以重要，在于它抽出了强化学习中最经典的矛盾：**利用** (exploitation) 与**探索** (exploration) 的平衡。如果系统总是选择当前看起来最好的机器，那就是利用，但它可能错过尚未充分尝试、实际上更好的机器；如果系统总在到处乱试，那就是探索，但它又可能浪费太多机会，没有把已知较优的机器充分利用起来。

设某个动作已经被选择了  $n - 1$  次，前面得到的奖赏为  $R_1, R_2, \dots, R_{n-1}$ ，则它的动作价值估计可以写成样本平均：

$$Q_n = \frac{R_1 + R_2 + \cdots + R_{n-1}}{n - 1}$$

也就是说，当前对某个动作值的判断，就是它历史表现的平均水平。这是最朴素也最自然的估计方式。

若进一步把更新写成递推形式，则有

$$Q_{n+1} = Q_n + \frac{1}{n} [R_n - Q_n]$$

这个式子非常值得熟悉。它说明新估计等于旧估计加上一个修正项，而修正项正比于“新奖赏与旧估计之间的误差”。因此，强化学习中的许多更新公式，本质上都可以看成“旧值 + 学习率 × 误差”的形式。

当环境是非平稳的，也就是机器的真实收益会随时间变化时，单纯做简单平均往往反应太慢。此时常采用常数步长更新：

$$Q_{n+1} = Q_n + \alpha [R_n - Q_n]$$

其中， $\alpha$  是学习率。若把这个公式不断展开，会发现它对较新的奖赏赋予更大权重，对更久远的奖赏赋予指数衰减的权重。因此，它实际上形成了一种**指数加权平均**。

在动作选择上，最经典的简单方法是  $\epsilon$ -**贪心策略**。它的思想很直接：以较大概率  $1 - \epsilon$  选择当前估计值最大的动作，以较小概率  $\epsilon$  随机探索。这样一来，系统既能主要利用当前已知较优动作，又不至于完全失去探索新机会的可能。

这类方法虽然简单，却把强化学习中的一个根本难题清楚地暴露出来：好的决策不只是“选当前最优”，还包括“为了未来更优而适度试错”。

## 9.5 贝尔曼方程：把当前与未来联系起来

强化学习真正进入理论核心，通常从**贝尔曼方程** (Bellman equation) 开始。它之所以重要，是因为它把“长期回报”拆解成了“当前一步奖赏 + 未来状态的价值”。

对任意策略  $\pi$ ，状态价值函数定义为

$$v_{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid S_t = s]$$

根据回报的定义， $G_t = R_{t+1} + \gamma G_{t+1}$ ，所以

$$v_{\pi}(s) = \mathbb{E}_{\pi}[R_{t+1} + \gamma G_{t+1} \mid S_t = s]$$

若再把动作选择与环境转移都展开，就得到

$$v_{\pi}(s) = \sum_a \pi(a \mid s) \sum_{s', r} p(s', r \mid s, a) [r + \gamma v_{\pi}(s')]$$

这里， $p(s', r \mid s, a)$  表示在状态  $s$  下执行动作  $a$  后，转移到状态  $s'$  并得到奖赏  $r$  的概率。这个式子里每个部分都必须看懂：最外层对动作求和，是因为策略可能是随机的；内层对后继状态和奖赏求和，是因为环境可能是随机的；方括号里则是“一步立即收益 + 未来折扣价值”。

初次看到时，你可能会感到有点困惑：每次要“做选择”的好像只有动作，为什么后面还要加一次概率求和呢？这是因为在强化学习的问题中，即便做出了一个决定，下一步的状态和奖赏也未必是确定的。在“老鼠走迷宫”的例子中，我们可能会觉得每个格子上是否有奶酪是确定的，但很多例子中都并非如此。例如，选择同一台老虎机游戏，每轮的结果未必相同，我们只能用一个概率来衡量。正因如此，贝尔曼方程中，还需要对“在状态  $s$  下执行动作  $a$  后，转移到状态  $s'$  并得到奖赏  $r$ ”进行概率求和。

综上所述，贝尔曼方程最核心的思想只有一句话：**一个状态的价值，不是孤立决定的，而是由它通向的下一步及更远未来共同决定的。**这正是动态规划与强化学习能够成立的基础。

## 9.6 广义策略迭代与策略迭代

理解了价值函数之后，下一个自然问题是：怎样得到更好的策略？强化学习里有一个极其重要的总框架，叫作**广义策略迭代** (generalized policy iteration, GPI)。它可以理解为两类过程的交替进行：一类是**策略评估** (policy evaluation)，另一类是**策略改进** (policy improvement)。

策略评估的任务是：给定当前策略  $\pi$ ，估计它的价值函数  $v_\pi$  或  $q_\pi$ 。策略改进的任务则是：根据已经估计出来的价值，构造一个不比原来更差的新策略  $\pi'$ 。于是，系统会在“评估当前策略”和“让策略变得更贪心一些”之间不断往返，直到最终收敛到较优甚至最优策略。

在模型已知、状态动作空间较小的情形下，一个最典型的方法是**策略迭代** (policy iteration)。其过程可以概括为：

1. 任意初始化一个策略  $\pi$  和价值函数  $V(s)$ 。这里的  $V(s)$  初始化的结果，我们可以形象地理解成“迷宫中所有格子的价值都赋初值为 0”。
2. 固定当前策略，反复更新  $V(s)$ ，直到价值近似收敛。
3. 利用当前价值函数，对每个状态改选更优动作，得到新策略。
4. 若策略不再变化，则停止；否则继续回到策略评估阶段，即第 2 步。

在策略评估中，常见更新形式为

$$V(s) \leftarrow \sum_{s', r} p(s', r \mid s, \pi(s)) [r + \gamma V(s')]$$

而策略改进常写成

$$\pi(s) \leftarrow \arg \max_a \sum_{s', r} p(s', r \mid s, a) [r + \gamma V(s')]$$

在这里， $\arg \max$  的意思是不取这个最大的数值，而是获取那个能带来最大值的动作的名字。

上述公式说明：一方面，价值函数是在给定策略下算出来的；另一方面，策略又会根据价值函数变得更优。这种相互促进关系，正是强化学习许多方法背后的共通思想。

有些图示会把这一过程画成两条彼此推动的曲线：一条代表价值函数逐渐接近最优价值，另一条代表策略逐渐接近最优策略。即使不去深究图形细节，也一定要明白：**强化学习中的“学价值”和“学策略”往往不是完全分开的两件事，而是在相互牵引中一起向前走。**

## 9.7 一个直观例子：为什么局部动作要服从长期价值

有些例子会用高尔夫击球来解释最优策略的意义。球员站在某个位置时，可以选择不同球杆，例如短杆、长杆或推杆。某种动作也许本次打得很远，但可能把球送进沙坑；另一种动作看似保守，却能把球更稳定地带向果岭。若只看眼前距离，系统可能倾向于猛打一杆；但若从“最终多少杆完成”这一长期目标看，最优动作就不一定是眼前推进最多的动作。

这个例子说明，强化学习中的动作价值并不只是“这一步有多爽快”，而是“这一步会不会把自己带到更好的后续局面”。许多初学者一开始对价值函数的理解之所以模糊，正是因为总把它想成当前奖赏，而忽略了状态转移后的长期影响。

因此，强化学习与静态分类的差异就更加清楚了。分类问题里，一个输入通常只对应一个输出；强化学习里，一次动作往往会改变整个后续问题的结构。决策之所以难，不仅因为环境不确定，更因为今天的选择会改变明天的选项。

## 9.8 蒙特卡洛方法：从完整经历中学习

若不知道环境模型，但能够反复采样得到完整回合，那么一种自然思路是：先把一整局玩完，再根据最终结果回头估计各状态或状态动作对的价值。这就是**蒙特卡洛方法** (Monte Carlo methods)。

它的核心思想很直接。若某个状态动作对在多次回合中反复出现，就收集每次出现之后得到的回报  $G$ ，把这些回报取平均，作为该状态动作对的价值估计。于是，蒙特卡洛方法不是依赖显式转移概率推导价值，而是依赖**实际采样回报的平均**。

例如在 **Monte Carlo ES** 方法中，系统会从某个起始状态动作对出发生成一条回合，然后对于回合中出现的每个  $(s, a)$ ，取其第一次出现之后的回报，累积到  $\text{Returns}(s, a)$  中，再用平均回报更新  $Q(s, a)$ 。最后，对回合中出现过的状态，把策略改成当前  $Q$  值最大的动作。

这类方法的优点是概念直接，而且不需要环境模型；但它也有明显局限。最大的问题是：**必须等整个回合结束后才能更新**。如果任务特别长，或者根本没有明确终点，那么这种“等到最后再回看”的做法就会非常慢。

## 9.9 时序差分学习：边走边学

为了解决蒙特卡洛方法必须等待回合结束的问题，强化学习中发展出了极其重要的一类方法，即**时序差分学习** (temporal-difference learning, TD learning)。它常被认为是强化学习中最核心、也最有代表性的思想之一。

TD 方法兼有两类优点。一方面，它像蒙特卡洛方法那样，可以直接从原始经验中学习，而不需要已知环境模型；另一方面，它又像动态规划那样，不必等到最终结果，而是可以利用已有价值估计对当前值进行**自举** (bootstrap) 更新。

若要估计某策略下的状态价值，蒙特卡洛更新会使用完整回报：

$$V(S_t) \leftarrow V(S_t) + \alpha [G_t - V(S_t)]$$

而 TD(0) 则把完整回报替换成一步目标：

$$V(S_t) \leftarrow V(S_t) + \alpha [R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$

方括号里的

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$$

常被称为**时序差分误差** (TD error)。它刻画了“当前估计”与“一步奖赏加下一状态估计”之间的差别。若这个误差为正，说明当前状态价值被低估了；若为负，则说明被高估了。

TD 学习最值得抓住的思想是：系统不必等未来全部发生完毕，只要看一步之后发生了什么，以及自己当前对下一状态的估计如何，就能立刻开始修正。这种“边走边学”的能力，使它在许多连续决策问题中都非常重要。

### 9.10 Sarsa 与 Q-learning：控制问题的两条典型路线

前面讨论的 TD 方法主要是估计状态价值。若进一步希望直接学会如何选动作，就会进入控制问题。这里有两条最经典的路线：**Sarsa 与 Q-learning**。

Sarsa 是一种**同策略** (on-policy) 时序差分控制方法。所谓同策略，可以理解为：它评估和改进的，就是当前实际用来产生行为的那条策略。Sarsa 的一次更新写为

$$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma Q(S', A') - Q(S, A)]$$

其中， $A'$  是在新状态  $S'$  下，按当前行为策略实际选出来的动作。也就是说，如果它采用了  $\epsilon$ -贪心策略，那么它完全有可能去选择一些其他动作。因为更新目标里真的使用了“下一步实际会怎么选”，所以它反映的是当前行为策略本身的长期效果。

Q-learning 则是一种**异策略** (off-policy) 方法。它的更新形式为

$$Q(S, A) \leftarrow Q(S, A) + \alpha \left[ R + \gamma \max_a Q(S', a) - Q(S, A) \right]$$

这里不管下一步实际执行了什么动作，更新目标都直接采用下一状态中估计值最大的动作。也就是说，Q-learning 一边可能用带探索的策略去采样，一边却按照“理想上应当选最优动作”来更新。因此，它学的是朝向最优策略的价值函数。

Sarsa 与 Q-learning 的差别，不应只背成“一个 on-policy，一个 off-policy”，更要从直觉上理解。Sarsa 更贴近“我按照当前带探索的行为方式行动，那么这种真实行为方式长期效果如何”；Q-learning 更贴近“哪怕我现在还在探索，我也要把价值朝着理想最优动作的方向拉”。因此，在某些风险较高的环境中，Sarsa 往往更保守，因为它会把探索动作本身带来的风险也计入价值；而 Q-learning 则更激进一些。

### 9.11 基于模型的方法与现实挑战

到目前为止，前面介绍的许多方法都可以看成**无模型方法** (model-free methods)，也就是不显式学习环境的转移规律，只根据真实经验直接更新价值或策略。然而，在很多情形下，若能够学习一个环境模型，再利用模型进行模拟、规划和搜索，系统就可能更高效。这就是**基于模型的方法** (model-based methods) 的思想。

可以把它理解成这样一种循环：真实环境提供真实经验，系统一方面直接用这些经验更新策略或价值，另一方面又用这些经验去学习一个“世界模型”；随后，系统还可以利用模型生成模拟经验，再做额外的规划更新。这样，学习就不再只依赖真实交互，还能借助内部模拟加速。

但强化学习真正落到现实系统时，困难往往很多。首先，状态与动作空间可能是高维、连续的，如何表示状态本身就不容易。其次，奖赏函数未必天然清晰，有时还存在多目标、风险敏感或延迟反馈问题。再次，真实系统能提供的交互样本常常很有限，而强化学习却常常需要大量试验。除此之外，还要处理探索与利用的权衡、已学模型的规划质量、复杂任务的自动分解，以及多个智能体并存时的协作与竞争等问题。

从这个角度看，强化学习远不只是几条更新公式。公式告诉我们怎样更新，但真正把系统做出来，还要解决表示、采样效率、稳定性与工程安全等许多问题。

## 9.12 策略梯度：直接优化策略

前面许多方法都在先学价值，再由价值间接决定动作。还有一条重要路线则是：**直接把策略本身参数化，然后直接优化策略参数**。这就是**策略梯度方法** (policy gradient methods)。

设对每个状态动作对都定义一个参数化偏好函数  $h(s, a, \theta)$ ，其中  $\theta$  是策略参数。和前面的深度学习一样，这个参数是一个很多维的向量。

通过引入神经网络，我们将原先的机械查表变成了连续的数学函数。在这里，状态  $s$  不再是一个单纯的格子编号，而是一组特征向量，可以描述当前状态的方方面面。在强化学习过程中，我们根据一个动作后的回报，计算梯度，并据此更新策略参数  $\theta$ 。

当我们让神经网络去调整参数  $\theta$ ，使得系统在“状态 1”下大幅提高“向右走”的概率时，由于神经网络的连续性，所有跟这个“状态 1”长得很像的“状态 2”、“状态 3”，所计算出来的“向右走”的概率也会被顺带一起推高。这就让 AI 拥有了人类一样的泛化能力，在相似的地方也能享受到经验。

在这个过程中，类似深度学习，常见做法也是用 softmax 形式把偏好转成概率：

$$\pi(a \mid s, \theta) = \frac{\exp(h(s, a, \theta))}{\sum_b \exp(h(s, b, \theta))}$$

这个式子非常重要。它表示：策略不再是查表式的死规定，而是由参数控制的概率分布。偏好越高，动作被选中的概率通常越大。

一个经典算法是 **REINFORCE with Baseline**。它会先根据当前策略采样整条回合，然后对每个时刻计算回报  $G_t$ ，再用

$$\delta = G_t - \hat{v}(S_t, w)$$

作为优势信号。其中， $\hat{v}(S_t, w)$  是一个基线，通常由参数  $w$  表示的状态价值函数给出。之后，一方面更新价值参数  $w$ ，另一方面沿着

$$\nabla_{\theta} \ln \pi(A_t \mid S_t, \theta)$$

的方向更新策略参数  $\theta$ 。

初学者可以先抓住最核心的一层意思：若某个动作最终带来了高于预期的回报，就提高它在相似状态下再次被选中的概率；若某个动作带来的回报低于预期，就降低它的概率。所谓“基线”的引入，则主要是为了降低方差，使学习更稳定。

### 9.13 n 步 TD、 $\lambda$ -回报与资格迹

一旦理解了蒙特卡洛与 TD(0) 的差别，就会自然想到一个问题：能不能在“完全等回合结束”和“只看一步”之间取中间方案？答案当然可以，这就得到 **n 步 TD** 的思想。

若从时刻  $t$  开始往前看  $n$  步，则对应的  $n$  步回报可写为

$$G_{t:t+n} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{n-1} R_{t+n} + \gamma^n \hat{v}(S_{t+n}, w)$$

这里，前  $n$  步奖赏是真实采样得到的，而第  $n$  步之后的未来，则仍用当前价值估计来补上。于是，当  $n = 1$  时，它退化成 TD(0)；当  $n$  足够大并覆盖整个回合时，它就越来越接近蒙特卡洛方法。

进一步地，若把不同步长的  $n$  步回报按权重加权平均，就得到  **$\lambda$ -回报**：

$$G_t^\lambda = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_{t:t+n}$$

其中， $0 \leq \lambda \leq 1$ 。当  $\lambda = 0$  时，方法更像单步 TD；当  $\lambda$  接近 1 时，方法更像蒙特卡洛。因此， $\lambda$  实际上在控制“更新到底更依赖短期自举，还是更依赖长期真实回报”。

这类方法还有两个经常一起出现的视角。**前向视角** (forward view) 直接从将来多个时间步的回报组合来理解更新目标；**后向视角** (backward view) 则从当前误差如何向过去状态传播来理解。后向视角中最核心的工具就是**资格迹** (eligibility traces)。

资格迹可以理解为：某个状态或参数分量最近是否“活跃过”，以及这种活跃痕迹还剩多少。若记迹向量为  $z$ ，则常见更新为

$$z \leftarrow \gamma \lambda z + \nabla \hat{v}(S, w)$$

然后再用 TD 误差更新参数：

$$w \leftarrow w + \alpha \delta z$$

它的直观意义是：当前一步产生的误差，不只修正此刻的状态估计，还会按衰减方式回传给之前那些“刚刚参与过决策”的状态或参数。正因为如此，资格迹常被看作连接 TD 与蒙特卡洛的一座桥梁。

### 9.14 深度强化学习与多智能体方向

当状态空间很大、甚至是图像、棋盘、连续控制信号这类高维输入时，表格式强化学习很快就不够用了。这时就需要把神经网络引入强化学习，用网络去近似策略函数、价值函数或环境模型，这便形成了**深度强化学习** (deep reinforcement



learning)。

一个非常著名的例子是 AlphaGo。围棋之所以长期被视作极具挑战性的任务，是因为它的搜索空间极其庞大，而且局面好坏并不容易直接评价。AlphaGo 的关键思想之一，就是用**策略网络** (policy network) 帮助选点，用**价值网络** (value network) 评估局面，再把监督学习、自我博弈与强化学习结合起来。这样，系统就不再只依赖手工设计的评估函数，而是能够从大量对局经验中逐步学到更强的落子能力。

强化学习还可以走向**多智能体** (multi-agent) 场景。此时环境中不再只有一个行动者，而是有多个主体同时学习、协作或竞争。这样一来，每个智能体面对的环境都会因为其他智能体的变化而不断改变，问题难度显著上升。但与此同时，也可能涌现出更复杂、更丰富的群体行为，例如追逐、协作建造、工具使用和策略演化等。

有些研究会讨论多智能体自动课程 (autocurricula) 现象，意思是多个智能体在长期互动中，会自发形成越来越复杂的任务压力 and 能力需求，从而推动新行为不断出现。这个方向提醒我们：强化学习不仅能研究单个行动者怎样变聪明，也能研究复杂系统中的集体智能怎样逐步形成。

这一章真正要建立的，是对强化学习主线的一种整体把握：**智能体在环境中不断交互，通过奖赏信号学习长期决策，而价值函数、策略、回报、自举更新和探索机制共同构成了这条学习链。**

## 本章小练习

### 1. 判断题

强化学习与监督学习的本质区别只在于是否使用神经网络；若都使用神经网络，那么二者就没有原则差异。(\_\_\_\_)

### 2. 填空题

在强化学习中，描述“在状态  $s$  下采取动作  $a$  的概率”的函数通常记为 \_\_\_\_\_；把从当前时刻开始未来奖赏累积而成的量通常记为 \_\_\_\_\_。

### 3. 简答题

请说明 Sarsa 与 Q-learning 的更新目标有什么差别，并解释为什么这会导致二者在行为风格上常常有所不同。

## 参考解答

1. 错。强化学习与监督学习的根本区别不在于是否使用神经网络，而在于问题结构不同。监督学习通常从带标签样本中学习输入到输出的映射，系统面对的是已经给出的标准答案；强化学习则是在环境交互中根据奖赏信号学习策略，重点是长期回报、状态转移以及探索与利用之间的平衡。即使二者都使用神经网络，学习目标与反馈方式也仍然不同。
2. 该函数通常记为  $\pi(a | s)$ ，它表示策略在状态  $s$  下选择动作  $a$  的概率；未来奖赏累积而成的量通常记为  $G_t$ ，称为回报。前者回答“该怎么行动”，后者回答“这样行动最终值不值”。
3. Sarsa 的更新目标是  $R + \gamma Q(S', A')$ ，其中  $A'$  是下一状态下按照当前行为策略实际选出的动作。因此，Sarsa 评估的是“当前实际行为策略”的效果，属于同策略方法。Q-learning 的更新目标则是  $R + \gamma \max_a Q(S', a)$ ，也就是直接把下一状态中估计值最大的动作当成目标，不管真实采样时下一步到底选了什么，因此它属于异策略方法。正因为如此，Sarsa 往往会把探索动作可能带来的风险也计入价值估计，行为相对更保守；Q-learning 则更倾向于朝理想贪心策略的方向更新，因此常显得更激进一些。